Re-practical Practice Solutions

The Menu and Binary scenario: A_1 to B_3

COS 110 | Swift Tuition | 2026 edition

Model answers for the practice questions. Try each question first. Each answer is the one file you would upload. It compiles with `g++ -std=c++98 -Wall -Werror` and runs clean under valgrind with the given files. Other correct answers are possible. After each answer comes an explanation: what the function must do, how the answer works, and the mistakes that lose marks.

Contents

- **A_1 Exceptions**
  1. A1-01. Exceptions: the constructors of the exception classes and `printException` (100)
  2. A1-02. Exceptions: `Menu::findFirst` throws `InvalidType` or the right not-found exception (100)
  3. A1-03. Exceptions: `Menu::findLast` searches from the back and throws (100)
  4. A1-04. Exceptions: `Menu::averagePrice` throws `NothingMatchingFilter` (100)
  5. A1-05. Exceptions: `Menu::numberAbove` counts items and throws `NothingAboveMinimumValue` (100)
  6. A1-06. Exceptions: `Menu::numberBelow` counts items and throws `NothingBelowMaximumValue` (100)
  7. A1-07. Exceptions: `Menu::numberEqual` counts items and throws `NothingMatchingFilter` (100)
  8. A1-08. Exceptions: `Menu::printSummary` catches each exception type in the right order (100)
- **A_2 Operator overloading**
  1. A2-01. Operator overloading: `operator<<` for `Item` (100)
  2. A2-02. Operator overloading: `operator<<` for `Menu` (prints empty slots) (100)
  3. A2-03. Operator overloading: `operator<<` for `Exception` (100)
  4. A2-04. Operator overloading: `Menu::operator+(Item*)` adds an item or deletes it (100)
  5. A2-05. Operator overloading: `Menu::operator[]` returns `Item*&` (100)
  6. A2-06. Operator overloading: `Item::operator==` and `Item::operator<` (100)
  7. A2-07. Operator overloading: `Menu::operator+=` grows the array when the menu is full (100)
  8. A2-08. Operator overloading: `Menu::operator-(string)` removes an item by name (100)
- **A_3 Assignment operator**
  1. A3-01. Assignment operator: `Item::operator=` (100)
  2. A3-02. Assignment operator: `Menu::operator=` (100)
  3. A3-03. Copy constructor: `Menu::Menu(const Menu&)` (100)
  4. A3-04. Assignment operator: `Menu::operator=` that keeps its own size (100)
  5. A3-05. Assignment operator: `Binary::operator=` (a `bool*` array) (100)
- **B_1 Recursion**
  1. B1-01. Recursion: `Binary::toDecimal` (100)
  2. B1-02. Recursion: `Binary::countOnes` (100)
  3. B1-03. Recursion: `Binary::isPalindrome` (100)
  4. B1-04. Recursion: `BinaryStrings::countNegative` (100)
  5. B1-05. Recursion: `BinaryStrings::calculatePos` with a node and a bit index (100)
  6. B1-06. Recursion: `BinaryStrings::printReverse` (100)
  7. B1-07. Recursion: `Binary::setBits` (100)
- **B_2 Linked lists**

1. B2-01. Linked lists: `BinaryStrings::append` (100)
2. B2-02. Linked lists: `BinaryStrings::insertAt` (100)
3. B2-03. Linked lists: `BinaryStrings::removeAt` (100)
4. B2-04. Linked lists: `BinaryStrings::remove` the first equal string (100)
5. B2-05. Linked lists: `BinaryStrings::reverse` (100)
6. B2-06. Linked lists: `BinaryStrings::sort` by decimal value (100)
7. B2-07. Linked lists: `BinaryStrings::calculatePos` (100)
8. B2-08. Linked lists: `BinaryStrings` copy constructor (100)

- **B_3 Template specialisation**
  1. B3-01. Template specialisation: `printValue<Binary>` (100)
  2. B3-02. Template specialisation: `largest<Binary>` (100)
  3. B3-03. Template specialisation: `isEqual<Binary>` (100)
  4. B3-04. Template specialisation: `addition<Binary, Binary>` throws (100)
  5. B3-05. Template specialisation: member `Node<Binary>::toInt` (100)
  6. B3-06. Template specialisation: `countTrue<Binary>` in a file that a header includes (100)

A_1 Exceptions: solutions

A1-01. Exceptions: the constructors of the exception classes and `printException` (100)

File: A_1.cpp

```
#include "Exception.h"

// msg is private, so every subclass sets it through the constructor of its direct base class.
Exception::Exception(string msg) : msg(msg) {
}

void Exception::printException() {
    cout << "Exception: " << msg << endl;
}

InvalidType::InvalidType(string type) : Exception("Invalid type: " + type) {
}

NothingMatchingFilter::NothingMatchingFilter(string filter) : Exception("Nothing matching filter: " +
filter) {
}

NothingAboveMinimumValue::NothingAboveMinimumValue() : NothingMatchingFilter("above minimum value") {
}

NothingBelowMaximumValue::NothingBelowMaximumValue() : NothingMatchingFilter("below maximum value") {
}

NotFoundException::NotFoundException() : Exception("Item not found") {
}

StarterNotFound::StarterNotFound() : NotFoundException() {
}

MainNotFound::MainNotFound() : NotFoundException() {
}

DessertNotFound::DessertNotFound() : NotFoundException() {
}
```

Expected output of the given main.cpp

```
Exception: Kitchen closed
Exception: Kitchen closed
Other: Exception: Invalid type: Drink
Filter: Exception: Nothing matching filter: Dessert
Filter: Exception: Nothing matching filter: above minimum value
Filter: Exception: Nothing matching filter: below maximum value
Not found: Exception: Item not found
Not found: Exception: Item not found
Not found: Exception: Item not found
```

Explanation

What the functions must do

The file `A_1.cpp` holds ten short functions: the constructor of `Exception`, the member function `printException()`, and one constructor for each of the eight subclasses. Each constructor only has to put the right text into `msg`. `printException()` prints that text after `Exception:`.

The one idea behind all ten functions: `msg` is `private` in `Exception`. Only the code of `Exception` itself may touch it. A subclass sets it by passing its text to the constructor of the class directly above it.

How the solution works

The base constructor.

```
Exception::Exception(string msg) : msg(msg) {
}
```

The part after the colon is the member initialiser list. `msg(msg)` means: initialise the member `msg` with the parameter `msg`. Inside the brackets, the name `msg` means the parameter. Outside, before the brackets, it means the member. The body is empty because there is nothing else to do.

`printException()`.

```
void Exception::printException() {
    cout << "Exception: " << msg << endl;
}
```

It prints to `cout`, not to a stream parameter. It ends with `endl`. The given `operator<<` prints the same text without the newline. In `main.cpp`, the first two lines show the difference: both print `Exception: Kitchen closed`, and the second needs its own `endl`.

The two direct subclasses that take a string.

```
InvalidType::InvalidType(string type) : Exception("Invalid type: " + type) {
}

NothingMatchingFilter::NothingMatchingFilter(string filter) : Exception("Nothing matching filter: " +
filter) {
}
```

`"Invalid type: " + type` joins a text and a `string`. The result is a new `string`. This string goes to the `Exception` constructor, which stores it in `msg`. The subclass never names `msg`.

The two classes below `NothingMatchingFilter`.

```
NothingAboveMinimumValue::NothingAboveMinimumValue() : NothingMatchingFilter("above minimum value") {
}
```

`NothingAboveMinimumValue` is derived from `NothingMatchingFilter`, not from `Exception`. So its initialiser list calls the `NothingMatchingFilter` constructor. That constructor adds `Nothing matching`

`filter:` in front. The stored message is `Nothing matching filter: above minimum value`. The class passes only the end of the message. `NothingBelowMaximumValue` works the same way with `below maximum value`.

The not-found classes.

```
NotFoundException::NotFoundException() : Exception("Item not found") {
}

StarterNotFound::StarterNotFound() : NotFoundException() {
}
```

`NotFoundException` stores `Item not found`. `StarterNotFound`, `MainNotFound` and `DessertNotFound` call the `NotFoundException` constructor, which stores that same message. The three classes differ only in their type. A `catch` block can tell them apart; their text cannot.

Writing `: NotFoundException()` is optional. If you leave it out, C++ calls `NotFoundException()` for you. Writing it makes the chain visible.

The chain of constructor calls. When `main` runs `throw NothingAboveMinimumValue();`, these steps happen:

1. `NothingAboveMinimumValue()` calls `NothingMatchingFilter("above minimum value")`.
2. `NothingMatchingFilter` puts `Nothing matching filter:` in front and calls `Exception("Nothing matching filter: above minimum value")`.
3. The `Exception` constructor stores the text in `msg`. Its body ends.
4. The body of `NothingMatchingFilter` runs. It is empty.
5. The body of `NothingAboveMinimumValue` runs. It is empty.

Trace

The calls below are in `main.cpp`. Open it to see the full code.

`main.cpp` throws one object of each class in a loop. The helper `throwOne(which)` throws a different object for each value of `which`. The three `catch` blocks are ordered with each derived class before its base class.

`which`	thrown object	first `catch` that fits	printed line
0	`InvalidType("Drink")`	`Exception&` (an `InvalidType` object is not a `NothingMatchingFilter` and not a `NotFoundException`)	`Other: Exception: Invalid type: Drink`
1	`NothingMatchingFilter("Dessert")`	`NothingMatchingFilter&`	`Filter: Exception: Nothing matching filter: Dessert`
2	`NothingAboveMinimumValue()`	`NothingMatchingFilter&` (its base class)	`Filter: Exception: Nothing`

which	thrown object	first catch that fits	printed line
			matching filter: above minimum value
3	NothingBelowMaximumValue()	NothingMatchingFilter&	Filter: Exception: Nothing matching filter: below maximum value
4	StarterNotFound()	NotFoundException&	Not found: Exception: Item not found
5	MainNotFound()	NotFoundException&	Not found: Exception: Item not found
6	DessertNotFound()	NotFoundException&	Not found: Exception: Item not found

For `which` 6, none of the `if` conditions in `throwOne` is true, so the last line throws `DessertNotFound()`. After `which` 6, the loop condition `which <= 6` is false and the loop ends.

Each line has two parts. The `catch` block prints the prefix, for example `Filter:`. Then `e.printException()` prints `Exception:`, the message and the newline. The call works on a base-class reference: `e` has the type `NothingMatchingFilter&`, but the object is a `NothingAboveMinimumValue`. Its `msg` was set by the constructor chain, so the right text appears.

Common mistakes

- **Assigning to `msg` in the body of a subclass constructor**, for example `InvalidType::InvalidType(string type) { msg = "Invalid type: " + type; }`. This fails twice. `msg` is private, so the subclass may not use it. Also, without an initialiser list C++ calls `Exception()`, and `Exception` has no default constructor. The build stops with an error, and the mark is 0.
- **Calling the `Exception` constructor from `NothingBelowMaximumValue`**, as in `: Exception("Nothing matching filter: below maximum value")`. A constructor may call only the constructor of its direct base class. `Exception` is not the direct base of `NothingBelowMaximumValue`. The build stops with the error "type 'Exception' is not a direct base of 'NothingBelowMaximumValue'".
- **Passing the whole message to `NothingMatchingFilter`**, as in `: NothingMatchingFilter("Nothing matching filter: above minimum value")`. The `NothingMatchingFilter` constructor adds its prefix again. The line becomes `Filter: Exception: Nothing matching filter: Nothing matching filter: above minimum value`. With this mistake, the output of `main.cpp` does not match the `ExpectedOutput` block.

- **Leaving out `endl` in `printException()`.** The next output then continues on the same line. In `main.cpp`, the first two lines join into `Exception: Kitchen closedException: Kitchen closed`, and every caught exception runs into the next line. With this mistake, the output of `main.cpp` does not match the `ExpectedOutput` block.
- **Spelling a message differently**, for example `Invalid Type:` or `Item not found.` with a full stop. The output must match the table in the question exactly.

A1-02. Exceptions: `Menu::findFirst` throws `InvalidType` or the right not-found exception (100)

File: A_1.cpp

```
#include "Menu.h"

Item Menu::findFirst(string type) {
    // Check the type name before the loop. An invalid name throws even for an empty menu.
    char code;
    if (type == "Starter") {
        code = 'S';
    } else if (type == "Main") {
        code = 'M';
    } else if (type == "Dessert") {
        code = 'D';
    } else {
        throw InvalidType(type);
    }
    for (int i = 0; i < maxSize; i++) {
        if (items[i] != NULL && items[i]->getType() == code) {
            return *items[i];
        }
    }
    // The loop ended, so no item of this type is on the menu.
    if (code == 'S') {
        throw StarterNotFound();
    }
    if (code == 'M') {
        throw MainNotFound();
    }
    throw DessertNotFound();
}
```

Expected output of the given main.cpp

```
First Starter: Garlic bread (S) R45.5
First Main: Beef burger (M) R120
First Dessert: Malva pudding (D) R60
InvalidType caught: Exception: Invalid type: Drink
First Dessert: Ice cream (D) R35
StarterNotFound caught: Exception: Item not found
MainNotFound caught: Exception: Item not found
DessertNotFound caught: Exception: Item not found
```

Explanation

What the function must do

`findFirst(type)` returns a copy of the first item of one type. The type arrives as a type name: `"Starter"`, `"Main"` or `"Dessert"`. Each `Item` stores its type as one character: `'S'`, `'M'` or `'D'`. So the function first turns the type name into the type code, then searches the array.

There are two ways to fail, and each has its own exception:

- the type name is not valid: `InvalidType(type)`;
- the type name is valid, but no item of that type is on the menu: `StarterNotFound()`, `MainNotFound()` or `DessertNotFound()`.

How the solution works

Step 1: turn the type name into a type code.

```
char code;
if (type == "Starter") {
    code = 'S';
} else if (type == "Main") {
    code = 'M';
} else if (type == "Dessert") {
    code = 'D';
} else {
    throw InvalidType(type);
}
```

- `type == "Starter"` compares the whole `string` with the text. The comparison is exact. `"starter"` and `"Starters"` are not equal to `"Starter"`.
- Only one branch of the `if` / `else if` chain runs. The first condition that is true sets `code`.
- The final `else` runs when none of the three words matched. `throw` leaves the function at once. Nothing after it in the function runs.
- This step comes before the loop. So `findFirst("Drink")` throws `InvalidType` even on an empty menu. The question requires this order.

Step 2: search from the front.

```
for (int i = 0; i < maxSize; i++) {
    if (items[i] != NULL && items[i]->getType() == code) {
        return *items[i];
    }
}
```

- The loop visits the slots 0, 1, ..., `maxSize - 1`. It stops before `maxSize`, which is outside the array.
- `items[i] != NULL` checks that the slot holds an item. An empty slot holds `NULL`.
- `items[i]->getType() == code` checks the type. The arrow `->` calls a member function through a pointer.
- `&&` stops at the first false part. For a `NULL` slot the left part is false, so `items[i]->getType()` never runs.
- `return *items[i];` follows the pointer and returns the item. The return type is `Item`, not `Item&`, so the caller gets a copy made by the copy constructor. The item in the menu stays where it is.
- `return` ends the function inside the loop. So the function returns the first match and never looks at later slots.

Step 3: nothing matched.

```
if (code == 'S') {
    throw StarterNotFound();
}
if (code == 'M') {
    throw MainNotFound();
```

```
    }
    throw DessertNotFound();
```

The loop ends without a `return` only when no slot matched. The code still holds the type, so the function picks the matching not-found class. If `code` is neither `'S'` nor `'M'`, it must be `'D'`, so the last line needs no `if`.

Every path of the function ends in a `return` or a `throw`. So g++ gives no "control reaches end of non-void function" warning.

`throw StarterNotFound();` creates the object and throws it by value. The caller catches it by reference, `catch (StarterNotFound& e)`.

Trace

The calls below are in `main.cpp`. Open it to see the full code.

The first menu in `main.cpp` has `maxSize` 5:

Slot	0	1	2	3	4
Item	Garlic bread (S)	Beef burger (M)	Veg curry (M)	Malva pudding (D)	`NULL`

`showFirst(menu, "Main").` Step 1 sets `code` to `'M'`.

i	items[i] != NULL	getType() == 'M'	action
0	true	`'S'`: false	next slot
1	true	`'M'`: true	return Beef burger

Slots 2 to 4 are never visited. The line is `First Main: Beef burger (M) R120`.

`showFirst(menu, "Dessert").` `code` is `'D'`. Slots 0, 1 and 2 hold other types. Slot 3 matches, and the function returns Malva pudding. Slot 4 (`NULL`) is never reached.

`showFirst(menu, "Drink").` Step 1 finds no match and throws `InvalidType("Drink")`. The loop never runs. `showFirst` catches it in `catch (InvalidType& e)` and prints `InvalidType caught: Exception: Invalid type: Drink`.

`showFirst(dessertsOnly, "Starter").` This menu has `maxSize` 3: slot 0 is Ice cream (D), slots 1 and 2 are `NULL`. `code` is `'S'`.

i	items[i] != NULL	getType() == 'S'	action
0	true	`'D'`: false	next slot
1	false	not evaluated	next slot
2	false	not evaluated	next slot

`i` becomes 3, `3 < 3` is false, and the loop ends. `code` is `'S'`, so the function throws `StarterNotFound()`. The output is `StarterNotFound caught: Exception: Item not found`.

`showFirst(empty, "Dessert").` Every slot is `NULL`. The loop finds nothing, and the function throws `DessertNotFound()`.

Common mistakes

- **Throwing the base class `NotFoundException()` for every type.** The message is the same, so it looks right. But the caller's `catch (StarterNotFound& e)` does not accept a `NotFoundException`. In `main.cpp` the object goes to `catch (Exception& e)`, because the `catch (StarterNotFound& e)` block does not fit. The line reads `Exception caught: Exception: Item not found` instead of `StarterNotFound caught: ...`. With this mistake, the output of `main.cpp` does not match the `ExpectedOutput` block.
- **No `NULL` check**, as in `if (items[i]->getType() == code)`. The call goes through a `NULL` pointer as soon as the search reaches an empty slot. In `main.cpp` this happens for `showFirst(dessertsOnly, "Starter")`. Valgrind reports `Invalid read`, and the program stops with a segmentation fault. The mark is 0.
- **Using the first letter of the type name as the code**, as in `char code = type[0];`. Then `"Drink"` gives `'D'`, and the function returns Malva pudding instead of throwing `InvalidType`. `"main"` gives `'m'`, which matches nothing. With this mistake, the output of `main.cpp` does not match the `ExpectedOutput` block.
- **Letting an invalid type name reach the not-found code.** For example, after the loop: `if (type == "Starter") throw StarterNotFound(); else if (type == "Main") throw MainNotFound(); else throw DessertNotFound();`. Then `findFirst("Drink")` throws `DessertNotFound` instead of `InvalidType`. In `main.cpp`, the line for `"Drink"` reads `DessertNotFound caught: Exception: Item not found`. Checking the type name first, before the loop, rules this out.
- **Returning a pointer**, as in `return items[i];`. The return type is `Item`, so the build fails.
- **Catching the exception inside `findFirst`.** Then it never reaches the caller. Printing in `findFirst` also adds lines that are not in the expected output.

A1-03. Exceptions: `Menu::findLast` searches from the back and throws (100)

File: A_1.cpp

```
#include "Menu.h"

Item Menu::findLast(string type) {
    char code;
    if (type == "Starter") {
        code = 'S';
    } else if (type == "Main") {
        code = 'M';
    } else if (type == "Dessert") {
        code = 'D';
    } else {
        throw InvalidType(type);
    }
    // Start at the last slot and move to the front. The first match found is the last item.
    for (int i = maxSize - 1; i >= 0; i--) {
        if (items[i] != NULL && items[i]->getType() == code) {
            return *items[i];
        }
    }
    if (code == 'S') {
        throw StarterNotFound();
    }
    if (code == 'M') {
        throw MainNotFound();
    }
    throw DessertNotFound();
}
```

Expected output of the given main.cpp

```
Last Starter: Soup (S) R50
Last Main: Veg curry (M) R95.5
Last Dessert: Malva pudding (D) R60
InvalidType caught: Exception: Invalid type: Drinks
Last Main: Beef burger (M) R120
StarterNotFound caught: Exception: Item not found
DessertNotFound caught: Exception: Item not found
MainNotFound caught: Exception: Item not found
```

Explanation

What the function must do

`findLast(type)` returns a copy of the last item of one type: the matching item with the highest index. It throws `InvalidType(type)` for a type name that is not `"Starter"`, `"Main"` or `"Dessert"`. It throws `StarterNotFound()`, `MainNotFound()` or `DessertNotFound()` when no item of that type is on the menu.

The function is `findFirst` with one change: the loop runs from the back of the array to the front. The first match it meets is then the last item of that type.

How the solution works

Step 1: turn the type name into a type code.

```
char code;
if (type == "Starter") {
    code = 'S';
} else if (type == "Main") {
    code = 'M';
} else if (type == "Dessert") {
    code = 'D';
} else {
    throw InvalidType(type);
}
```

The comparison `type == "Main"` is exact and case-sensitive. If none of the three words matches, the `else` branch throws `InvalidType(type)` and the function ends. This step runs before the loop, so an invalid type name throws `InvalidType` even when the menu is empty.

Step 2: search from the back.

```
for (int i = maxSize - 1; i >= 0; i--) {
    if (items[i] != NULL && items[i]->getType() == code) {
        return *items[i];
    }
}
```

- `int i = maxSize - 1` starts at the last valid index. The index `maxSize` is outside the array.
- `i >= 0` keeps slot 0 in the search. With `i > 0`, slot 0 would never be checked.
- `i--` moves one slot to the front each time.
- `items[i] != NULL` comes first in the `&&`. The empty slots are at the back of the array, so a backward loop meets them first. For a `NULL` slot the left part is false, and C++ does not run `items[i]->getType()`.
- `return *items[i];` returns a copy of the item and ends the function. Because the loop runs backward, this is the match with the highest index.
- For a menu of size 0, `i` starts at -1. The condition `-1 >= 0` is false, and the loop body never runs.

Step 3: nothing matched.

```
if (code == 'S') {
    throw StarterNotFound();
}
if (code == 'M') {
    throw MainNotFound();
}
throw DessertNotFound();
```

The function picks the not-found class that belongs to the type. Every path ends in a `return` or a `throw`, so g++ gives no warning about a missing return value.

Trace

The calls below are in `main.cpp`. Open it to see the full code.

The first menu in `main.cpp` has `maxSize` 6:

Slot	0	1	2	3	4	5
Item	Garlic bread (S)	Beef burger (M)	Veg curry (M)	Malva pudding (D)	Soup (S)	`NULL`

`showLast(menu, "Main").` `code` is `'M'`. The loop starts at `i = 5`.

i	items[i] != NULL	getType() == 'M'	action
5	false	not evaluated	next slot
4	true	`'S'`: false	next slot
3	true	`'D'`: false	next slot
2	true	`'M'`: true	return Veg curry

The output is `Last Main: Veg curry (M) R95.5`. Beef burger in slot 1 is also a main, but the loop never reaches it.

`showLast(menu, "Starter").` `code` is `'S'`. The loop starts at `i = 5`.

i	items[i] != NULL	getType() == 'S'	action
5	false	not evaluated	next slot
4	true	`'S'`: true	return Soup

The function returns Soup, not Garlic bread.

`showLast(menu, "Drinks").` Step 1 throws `InvalidType("Drinks")` before the loop.

`showLast(mainsOnly, "Main").` This menu has `maxSize` 3 and one item, Beef burger, in slot 0.

i	items[i] != NULL	getType() == 'M'	action
2	false	not evaluated	next slot
1	false	not evaluated	next slot
0	true	`'M'`: true	return Beef burger

This call needs `i >= 0`: the only match is in slot 0.

`showLast(mainsOnly, "Starter").` The loop visits slots 2, 1 and 0 and finds no starter. Then `i` becomes -1, `-1 >= 0` is false, and the loop ends. The function throws `StarterNotFound()`.

Common mistakes

- **Starting at `i = maxSize`.** The first step reads `items[maxSize]`, one slot past the end of the array. Valgrind reports `Invalid read of size 8` on every call that reaches the loop (7 of the 8 calls in `main.cpp`; the `"Drinks"` call throws before the loop). The value read there can be 0, so the printed output can look right. Valgrind still reports the errors, and memory errors cost marks.

- **Writing `i > 0`.** Slot 0 is never checked. In `main.cpp`, `showLast(mainsOnly, "Main")` then throws `MainNotFound` instead of returning Beef burger. With this mistake, the output of `main.cpp` does not match the `ExpectedOutput` block.
- **A forward loop that returns at the first match.** It gives the first match: `Last Starter: Garlic bread` instead of `Last Starter: Soup`. With this mistake, the output of `main.cpp` does not match the `ExpectedOutput` block.

A forward loop that remembers the last match (a variable `Item* last = NULL;` set in the loop) is also correct, but it must then return `*last` after the loop and throw when `last` is still `NULL`. - **No `NULL` check.** The backward loop meets an empty slot first on any menu that is not full, so the program stops with a segmentation fault on the first call. - **Throwing `NotFoundException()`** for every type. The caller's `catch (StarterNotFound& e)` does not catch a plain `NotFoundException`, so the wrong `catch` block prints the line.

A1-04. Exceptions: `Menu::averagePrice` throws `NothingMatchingFilter` (100)

File: A_1.cpp

```
#include "Menu.h"

float Menu::averagePrice(string type) {
    char code;
    if (type == "Starter") {
        code = 'S';
    } else if (type == "Main") {
        code = 'M';
    } else if (type == "Dessert") {
        code = 'D';
    } else {
        throw InvalidType(type);
    }
    float total = 0;
    int count = 0;
    for (int i = 0; i < maxSize; i++) {
        if (items[i] != NULL && items[i]->getType() == code) {
            total += items[i]->getPrice();
            count++;
        }
    }
    // Throw before the division, so the function never divides by 0.
    if (count == 0) {
        throw NothingMatchingFilter(type);
    }
    return total / count;
}
```

Expected output of the given main.cpp

```
Average Starter price: R47.75
Average Main price: R107.75
Average Dessert price: R60
InvalidType caught: Exception: Invalid type: Drink
Average Main price: R120
NothingMatchingFilter caught: Exception: Nothing matching filter: Dessert
NothingMatchingFilter caught: Exception: Nothing matching filter: Starter
```

Explanation

What the function must do

`averagePrice(type)` returns the average price of the items of one type: the sum of their prices divided by the number of items. It throws in two cases:

- the type name `type` is not `"Starter"`, `"Main"` or `"Dessert"`: `InvalidType(type)`;
- the menu has no item of that type: `NothingMatchingFilter(type)`. Without this exception, the function would divide by 0.

How the solution works

Step 1: turn the type name into a type code.

```
char code;
if (type == "Starter") {
    code = 'S';
} else if (type == "Main") {
    code = 'M';
} else if (type == "Dessert") {
    code = 'D';
} else {
    throw InvalidType(type);
}
```

The type name must match exactly. For any other type name, the function throws `InvalidType(type)` and stops. This runs before the loop, so an invalid type name throws `InvalidType` even on an empty menu.

Step 2: add up and count.

```
float total = 0;
int count = 0;
for (int i = 0; i < maxSize; i++) {
    if (items[i] != NULL && items[i]->getType() == code) {
        total += items[i]->getPrice();
        count++;
    }
}
```

- `total` is a `float`, like the prices. An `int` total would drop the decimals of each price.
- `count` is an `int`. It counts the matching items.
- The loop visits every slot. It must not stop at the first match.
- `items[i] != NULL` comes first in the `&&`, so the function never calls `getType()` through `NULL`.
- For a matching item, the price is added to `total` and `count` grows by 1. Both statements are inside the same `if`.

Step 3: throw or divide.

```
if (count == 0) {
    throw NothingMatchingFilter(type);
}
return total / count;
```

- The check uses `count`, not `total`. An item can cost 0, and it still counts as an item.
- The check comes before the division. `throw` ends the function, so the division never runs with `count` equal to 0.
- `NothingMatchingFilter(type)` gets the type name. The message is, for example, `Nothing matching filter: Dessert`.
- `total / count` divides a `float` by an `int`. C++ converts `count` to `float`, so the result keeps its decimals.

Trace

The calls below are in `main.cpp`. Open it to see the full code.

The first menu in `main.cpp` has `maxSize` 6:

Slot	0	1	2	3	4	5
Item	Garlic bread (S) R45.5	Beef burger (M) R120	Veg curry (M) R95.5	Malva pudding (D) R60	Soup (S) R50	`NULL`

`showAverage(menu, "Starter").` `code` is `'S'`.

`i`	not `NULL`	type `'S'`	`total` after	`count` after
0	true	true	45.5	1
1	true	false (`'M'`)	45.5	1
2	true	false (`'M'`)	45.5	1
3	true	false (`'D'`)	45.5	1
4	true	true	95.5	2
5	false	not evaluated	95.5	2

`count` is 2, so there is no throw. The result is 95.5 / 2 = 47.75. The line is `Average Starter price: R47.75`.

`showAverage(menu, "Main").` Slots 1 and 2 match. `total` is 120 + 95.5 = 215.5 and `count` is 2. The result is 107.75.

`showAverage(menu, "Dessert").` Only slot 3 matches. The result is 60 / 1 = 60, printed as `R60`.

`showAverage(menu, "Drink").` Step 1 throws `InvalidType("Drink")`.

`showAverage(small, "Dessert").` The menu `small` has `maxSize` 2. Slot 0 holds Beef burger (M), slot 1 is `NULL`. No item has the type `'D'`, so after the loop `count` is 0. The function throws `NothingMatchingFilter("Dessert")`, and `main` prints `NothingMatchingFilter caught: Exception: Nothing matching filter: Dessert`.

Common mistakes

- **No `count == 0` check.** For a type with no items, the function computes 0 / 0. For `float` values this does not crash. The result is "not a number". `cout` prints `nan` or `-nan`, depending on the system. With g++ on Linux, the line becomes `Average Dessert price: R-nan` instead of the `NothingMatchingFilter` line. With this mistake, the output of `main.cpp` does not match the `ExpectedOutput` block.
- **Declaring `int total`.** Each price is cut to a whole number when it is added: 45.5 becomes 45. The starter average becomes 95 / 2 = 47 (whole-number division), printed as `R47` instead of `R47.75`. g++ gives no warning for this with `-Wall`. With this mistake, the output of `main.cpp` does not match the `ExpectedOutput` block.
- **Checking `total == 0` instead of `count == 0`.** The output of `main.cpp` looks right, because no item there costs 0. `main.cpp` does not show this mistake. A menu whose only starter costs R0 does: `averagePrice("Starter")` then throws `NothingMatchingFilter` instead of returning 0.

- **Dividing by `maxSize`.** The empty slots and the items of other types are counted as well. The starter average becomes 95.5 / 6, printed as `R15.9167`. With this mistake, the output of `main.cpp` does not match the `ExpectedOutput` block.
- **Checking the count before the type name.** If the `count == 0` test comes first, an invalid type name (count 0) throws `NothingMatchingFilter("Drink")` instead of `InvalidType("Drink")`. The caller's `catch (InvalidType& e)` is skipped, and the line reads `NothingMatchingFilter caught: Exception: Nothing matching filter: Drink`. Check the type name first.

A1-05. Exceptions: `Menu::numberAbove` counts items and throws `NothingAboveMinimumValue` (100)

File: A_1.cpp

```
#include "Menu.h"

int Menu::numberAbove(string type, float value) {
    char code;
    if (type == "Starter") {
        code = 'S';
    } else if (type == "Main") {
        code = 'M';
    } else if (type == "Dessert") {
        code = 'D';
    } else {
        throw InvalidType(type);
    }
    int count = 0;
    for (int i = 0; i < maxSize; i++) {
        // The NULL check comes first: && stops at the first false part.
        if (items[i] != NULL && items[i]->getType() == code && items[i]->getPrice() > value) {
            count++;
        }
    }
    // No item is above the value.
    if (count == 0) {
        throw NothingAboveMinimumValue();
    }
    return count;
}
```

Expected output of the given main.cpp

```
Main above R100: 1
Main above R90: 2
NothingAboveMinimumValue caught: Exception: Nothing matching filter: above minimum value
Starter above R45.5: 1
NothingAboveMinimumValue caught: Exception: Nothing matching filter: above minimum value
InvalidType caught: Exception: Invalid type: Drink
```

Explanation

What the function must do

`numberAbove(type, value)` counts the items of one type whose price is greater than `value`. It returns the count. It throws in two cases:

- the type name `type` is not `"Starter"`, `"Main"` or `"Dessert"`: `InvalidType(type)`;
- the count is 0: `NothingAboveMinimumValue()`.

"Greater than" is strict. An item that costs exactly `value` does not count.

How the solution works

Step 1: turn the type name into a type code.

```
char code;
if (type == "Starter") {
    code = 'S';
} else if (type == "Main") {
    code = 'M';
} else if (type == "Dessert") {
    code = 'D';
} else {
    throw InvalidType(type);
}
```

The type name is compared exactly. If it is none of the three names, the function throws `InvalidType(type)` and stops. This happens before the loop, so an invalid type name throws even when the menu is empty.

Step 2: count.

```
int count = 0;
for (int i = 0; i < maxSize; i++) {
    if (items[i] != NULL && items[i]->getType() == code && items[i]->getPrice() > value) {
        count++;
    }
}
```

- `count` starts at 0 before the loop.
- The loop visits every slot from 0 to `maxSize - 1`. It does not stop early: every matching item must be counted.
- The condition has three parts joined by `&&`. All three must be true to count the item:
- `items[i] != NULL`: the slot holds an item. This part is first, so for an empty slot C++ stops here and never calls a function through `NULL`.
- `items[i]->getType() == code`: the item has the right type. Without this part, items of other types would be counted.
- `items[i]->getPrice() > value`: the price is strictly greater. An equal price gives false.

Step 3: throw or return.

```
if (count == 0) {
    throw NothingAboveMinimumValue();
}
return count;
```

A count of 0 throws `NothingAboveMinimumValue()`. This class takes no argument. Its message is always `Nothing matching filter: above minimum value`. Otherwise the function returns the count.

The check `count == 0` covers two different situations: no item of that type at all, and items of that type that are all too cheap. Both throw the same exception.

Trace

The calls below are in `main.cpp`. Open it to see the full code.

The menu in `main.cpp` has `maxSize` 6:

Slot	0	1	2	3	4	5
Item	Garlic bread (S) R45.5	Beef burger (M) R120	Veg curry (M) R95.5	Malva pudding (D) R60	Soup (S) R50	`NULL`

`show(menu, "Main", 100).` `code` is `'M'`, `value` is 100.

i	not NULL	type 'M'	price > 100	count after
0	true	false (`'S'`)	not evaluated	0
1	true	true	120: true	1
2	true	true	95.5: false	1
3	true	false (`'D'`)	not evaluated	1
4	true	false (`'S'`)	not evaluated	1
5	false	not evaluated	not evaluated	1

The count is 1, so the function returns 1. The line is `Main above R100: 1`.

`show(menu, "Main", 120).` Slot 1: `120 > 120` is false. Slot 2: `95.5 > 120` is false. The count stays 0, and the function throws `NothingAboveMinimumValue()`. The caller's `catch (NothingAboveMinimumValue& e)` prints `NothingAboveMinimumValue caught: Exception: Nothing matching filter: above minimum value`.

`show(menu, "Starter", 45.5).` Slot 0: Garlic bread costs 45.5, and `45.5 > 45.5` is false. Slot 4: Soup costs 50, and `50 > 45.5` is true. The count is 1.

`show(menu, "Dessert", 100).` Only slot 3 is a dessert, and `60 > 100` is false. The count is 0, so the function throws.

`show(menu, "Drink", 10).` Step 1 throws `InvalidType("Drink")`. The loop never runs.

Common mistakes

- **Writing `>=` instead of `>`.** Equal prices are then counted. `show(menu, "Main", 120)` returns 1 instead of throwing, and `show(menu, "Starter", 45.5)` returns 2. With this mistake, the output of `main.cpp` does not match the `ExpectedOutput` block.
- **Leaving out the type check in the condition.** There are two versions of this mistake.
- The solution's Step 1 is kept, but `items[i]->getType() == code` is removed from the condition. Then `code` is set and never read. g++ reports "variable 'code' set but not used", and `-Werror` makes this a build error. The mark is 0.
- The type name is only checked (`if (type != "Starter" && type != "Main" && type != "Dessert") throw InvalidType(type);`), and the loop counts every item. Then every item above the value counts, of any type. `show(menu, "Starter", 45.5)` counts Beef burger, Veg curry, Malva pudding and Soup and returns 4. `show(menu, "Dessert", 100)` counts Beef burger and returns 1. With this mistake, the output of `main.cpp` does not match the `ExpectedOutput` block.
- **Throwing `NothingMatchingFilter(type).`** It is the base class of `NothingAboveMinimumValue`. The caller's `catch (NothingAboveMinimumValue& e)` does not accept a base-class object. A later `catch` block prints `NothingMatchingFilter caught: Exception: Nothing matching filter: Main`. With this mistake, the output of `main.cpp` does not match the `ExpectedOutput` block.

- **No `NULL` check.** Slot 5 of the main menu is empty. The first call already reaches it and calls `getType()` through `NULL`. Valgrind reports an invalid read, and the program stops with a segmentation fault before it prints anything. The mark is 0.
- **Checking the type name only when the count is 0.** For an invalid type name the count is always 0, so the output can be the same. But the loop then reads every slot first, and one `if` does two jobs. Checking the type name before the loop is clearer and is what the question asks.

A1-06. Exceptions: `Menu::numberBelow` counts items and throws `NothingBelowMaximumValue` (100)

File: A_1.cpp

```
#include "Menu.h"

int Menu::numberBelow(string type, float value) {
    char code;
    if (type == "Starter") {
        code = 'S';
    } else if (type == "Main") {
        code = 'M';
    } else if (type == "Dessert") {
        code = 'D';
    } else {
        throw InvalidType(type);
    }
    int count = 0;
    for (int i = 0; i < maxSize; i++) {
        // The NULL check comes first: && stops at the first false part.
        if (items[i] != NULL && items[i]->getType() == code && items[i]->getPrice() < value) {
            count++;
        }
    }
    // No item is below the value.
    if (count == 0) {
        throw NothingBelowMaximumValue();
    }
    return count;
}
```

Expected output of the given main.cpp

```
Main below R100: 1
Main below R200: 2
NothingBelowMaximumValue caught: Exception: Nothing matching filter: below maximum value
Starter below R50: 1
NothingBelowMaximumValue caught: Exception: Nothing matching filter: below maximum value
InvalidType caught: Exception: Invalid type: Desserts
```

Explanation

What the function must do

`numberBelow(type, value)` counts the items of one type whose price is less than `value`. It returns the count. It throws in two cases:

- the type name `type` is not `"Starter"`, `"Main"` or `"Dessert"`: `InvalidType(type)`;
- the count is 0: `NothingBelowMaximumValue()`.

"Less than" is strict. An item that costs exactly `value` does not count.

The function has the same shape as `numberAbove`, with two changes: the comparison (`<`) and the exception class (`NothingBelowMaximumValue`). If you copy `numberAbove` and miss one of these changes, the output is wrong.

How the solution works

Step 1: turn the type name into a type code.

```
char code;
if (type == "Starter") {
    code = 'S';
} else if (type == "Main") {
    code = 'M';
} else if (type == "Dessert") {
    code = 'D';
} else {
    throw InvalidType(type);
}
```

The type name is compared exactly. Any other type name throws `InvalidType(type)` before the loop, so an invalid type name throws even on an empty menu.

Step 2: count.

```
int count = 0;
for (int i = 0; i < maxSize; i++) {
    if (items[i] != NULL && items[i]->getType() == code && items[i]->getPrice() < value) {
        count++;
    }
}
```

- The loop visits every slot from 0 to `maxSize - 1`.
- The three parts of the condition must all be true:
- `items[i] != NULL`: the slot holds an item. It is first, so C++ never calls a function through `NULL`.
- `items[i]->getType() == code`: the item has the right type.
- `items[i]->getPrice() < value`: the price is strictly less than `value`.
- `&&` stops at the first false part. For an item of another type, the price is not read.

Step 3: throw or return.

```
if (count == 0) {
    throw NothingBelowMaximumValue();
}
return count;
```

`NothingBelowMaximumValue()` takes no argument. Its message is `Nothing matching filter: below maximum value`. The class is derived from `NothingMatchingFilter`, so a caller may also catch it as a `NothingMatchingFilter&` or an `Exception&`.

Trace

The calls below are in `main.cpp`. Open it to see the full code.

The menu in `main.cpp` has `maxSize` 6:

Slot	0	1	2	3	4	5
Item	Garlic bread (S) R45.5	Beef burger (M) R120	Veg curry (M) R95.5	Malva pudding (D) R60	Soup (S) R50	`NULL`

`show(menu, "Main", 100).` `code` is `'M'`, `value` is 100.

`i`	not `NULL`	type `'M'`	price `< 100`	count after
0	true	false (`'S'`)	not evaluated	0
1	true	true	120: false	0
2	true	true	95.5: true	1
3	true	false (`'D'`)	not evaluated	1
4	true	false (`'S'`)	not evaluated	1
5	false	not evaluated	not evaluated	1

The function returns 1. The line is `Main below R100: 1`.

`show(menu, "Main", 200).` Both mains are below 200. The count is 2.

`show(menu, "Main", 95.5).` Slot 1: `120 < 95.5` is false. Slot 2: `95.5 < 95.5` is false. The count stays 0, so the function throws `NothingBelowMaximumValue()`. `main` prints `NothingBelowMaximumValue caught: Exception: Nothing matching filter: below maximum value`.

`show(menu, "Starter", 50).` Garlic bread: `45.5 < 50` is true. Soup: `50 < 50` is false. The count is 1.

`show(menu, "Dessert", 60).` Malva pudding costs exactly 60, and `60 < 60` is false. The count is 0, so the function throws.

`show(menu, "Desserts", 60).` `"Desserts"` is not `"Dessert"`. Step 1 throws `InvalidType("Desserts")`.

Common mistakes

- **Writing `<=` instead of `<`.** Equal prices are counted. `show(menu, "Main", 95.5)` returns 1 instead of throwing, and `show(menu, "Starter", 50)` returns 2. With this mistake, the output of `main.cpp` does not match the `ExpectedOutput` block.
- **Throwing `NothingAboveMinimumValue()`**, for example after copying `numberAbove`. Both classes are derived from `NothingMatchingFilter`, but they are not related to each other. The caller's `catch (NothingAboveMinimumValue& e)` comes first in `main.cpp`, so the line reads `NothingAboveMinimumValue caught: Exception: Nothing matching filter: above minimum value`. With this mistake, the output of `main.cpp` does not match the `ExpectedOutput` block.
- **No `NULL` check.** Slot 5 of the main menu is empty, so the first call reaches a `NULL` pointer. Valgrind reports an invalid read, and the program stops with a segmentation fault. The mark is 0.
- **Leaving out the type part of the condition.** If Step 1 still sets `code`, `code` is never read, and `-Werror` stops the build with "variable 'code' set but not used". If the type name is only validated, items of every type are counted, and the counts are wrong.
- **Returning 0 instead of throwing.** The question asks for an exception when nothing is below the value. A return of 0 prints `Main below R95.5: 0`, and no `catch` block runs.

A1-07. Exceptions: `Menu::numberEqual` counts items and throws `NothingMatchingFilter` (100)

File: A_1.cpp

```
#include "Menu.h"

int Menu::numberEqual(string type, float value) {
    char code;
    if (type == "Starter") {
        code = 'S';
    } else if (type == "Main") {
        code = 'M';
    } else if (type == "Dessert") {
        code = 'D';
    } else {
        throw InvalidType(type);
    }
    int count = 0;
    for (int i = 0; i < maxSize; i++) {
        // The NULL check comes first: && stops at the first false part.
        if (items[i] != NULL && items[i]->getType() == code && items[i]->getPrice() == value) {
            count++;
        }
    }
    // No item has this price. The message names the type.
    if (count == 0) {
        throw NothingMatchingFilter(type);
    }
    return count;
}
```

Expected output of the given main.cpp

```
Main equal to R95.5: 2
Main equal to R120: 1
NothingMatchingFilter caught: Exception: Nothing matching filter: Main
NothingMatchingFilter caught: Exception: Nothing matching filter: Dessert
Starter equal to R45.5: 1
InvalidType caught: Exception: Invalid type: Snack
```

Explanation

What the function must do

`numberEqual(type, value)` counts the items of one type whose price is exactly `value`. It returns the count. It throws in two cases:

- the type name `type` is not `"Starter"`, `"Main"` or `"Dessert"`: `InvalidType(type)`;
- the count is 0: `NothingMatchingFilter(type)`.

The second exception is the base class `NothingMatchingFilter`, not one of its two subclasses. It takes the type name, so the message names the type, for example `Nothing matching filter: Main`.

How the solution works

Step 1: turn the type name into a type code.

```
char code;
if (type == "Starter") {
    code = 'S';
} else if (type == "Main") {
    code = 'M';
} else if (type == "Dessert") {
    code = 'D';
} else {
    throw InvalidType(type);
}
```

Any type name other than the three names throws `InvalidType(type)` before the loop. An invalid type name therefore throws `InvalidType` even on an empty menu.

Step 2: count.

```
int count = 0;
for (int i = 0; i < maxSize; i++) {
    if (items[i] != NULL && items[i]->getType() == code && items[i]->getPrice() == value) {
        count++;
    }
}
```

- The loop visits every slot.
- `items[i] != NULL` is checked first, so C++ never calls `getType()` or `getPrice()` through `NULL`.
- `items[i]->getType() == code` keeps only items of the requested type.
- `items[i]->getPrice() == value` compares two `float` values exactly. In general, `==` on `float` values is risky, because many decimals (such as 0.1) cannot be stored exactly. The prices in this question (45.5, 95.5, 120, 35, 0) can be stored exactly, so `==` gives the expected result here.

Step 3: throw or return.

```
if (count == 0) {
    throw NothingMatchingFilter(type);
}
return count;
```

The type name `type` goes into the exception. The caller in `main.cpp` catches it with `catch (NothingMatchingFilter& e)`.

Why the catch order in `main.cpp` matters here. The helper `show` has these `catch` blocks, in this order: `InvalidType&`, `NothingAboveMinimumValue&`, `NothingBelowMaximumValue&`, `NothingMatchingFilter&`, `Exception&`.

A `NothingMatchingFilter` object is not an `InvalidType`, not a `NothingAboveMinimumValue` and not a `NothingBelowMaximumValue`. So the first three blocks are skipped, and the `NothingMatchingFilter&` block catches it. A subclass object would stop at its own block first. So the output shows exactly which class the function threw.

Trace

The calls below are in `main.cpp`. Open it to see the full code.

The menu in `main.cpp` has `maxSize` 7:

Slot	0	1	2	3	4	5	6
Item	Garlic bread (S) R45.5	Beef burger (M) R120	Veg curry (M) R95.5	Malva pudding (D) R60	Soup (S) R50	Pap and wors (M) R95.5	`NULL`

`show(menu, "Main", 95.5).` `code` is `'M'`, `value` is 95.5.

i	not `NULL`	type `'M'`	price `== 95.5`	count after
0	true	false (`'S'`)	not evaluated	0
1	true	true	120: false	0
2	true	true	95.5: true	1
3	true	false (`'D'`)	not evaluated	1
4	true	false (`'S'`)	not evaluated	1
5	true	true	95.5: true	2
6	false	not evaluated	not evaluated	2

The function returns 2. The line is `Main equal to R95.5: 2`.

`show(menu, "Main", 100).` No main costs 100. The count is 0, and the function throws `NothingMatchingFilter("Main")`. The line is `NothingMatchingFilter caught: Exception: Nothing matching filter: Main`.

`show(menu, "Dessert", 45.5).` Garlic bread costs 45.5, but it is a starter. The type part of the condition is false for it. Malva pudding is the only dessert, and it costs 60. The count is 0, and the message is `Nothing matching filter: Dessert`.

`show(menu, "Starter", 45.5).` Garlic bread matches. Soup costs 50. The count is 1.

`show(menu, "Snack", 45.5).` Step 1 throws `InvalidType("Snack")`.

Common mistakes

- **Throwing `NothingBelowMaximumValue()`**, for example after copying `numberBelow`. In `main.cpp`, the block `catch (NothingBelowMaximumValue& e)` comes before `catch (NothingMatchingFilter& e)`, so it catches the object. The line reads `NothingBelowMaximumValue caught: Exception: Nothing matching filter: below maximum value`. With this mistake, the output of `main.cpp` does not match the `ExpectedOutput` block.
- **Throwing `NothingMatchingFilter` with a fixed text**, for example `NothingMatchingFilter("equal value")`. The right `catch` block runs, but the message is `Nothing matching filter: equal value` instead of naming the type. With this mistake, the output of `main.cpp` does not match the `ExpectedOutput` block.
- **Counting items of every type.** If the type name is only validated and the loop has no type check, `show(menu, "Dessert", 45.5)` counts Garlic bread and returns 1 instead of throwing. With this

mistake, the output of `main.cpp` does not match the `ExpectedOutput` block. If Step 1 still sets `code` but the loop never reads it, `-Werror` stops the build with "variable 'code' set but not used".
- **No `NULL` check.** Slot 6 is empty. The first call reaches it and calls a function through `NULL`. The program stops with a segmentation fault, and the mark is 0.

A tolerance such as `fabs(price - value) < 0.01` gives the same result here. But it needs `<cmath>`, and the question asks for equal prices. Use `==`.

A1-08. Exceptions: `Menu::printSummary` catches each exception type in the right order (100)

File: A_1.cpp

```
#include "Menu.h"

bool Menu::printSummary(string type, float minPrice) {
    try {
        Item first = findFirst(type);
        int count = numberAbove(type, minPrice);
        // Both calls worked, so print the whole line now.
        cout << type << ": first is " << first << ", " << count << " above R" << minPrice << endl;
        return true;
    } catch (DessertNotFound& e) {
        // A derived class must come before its base class (NotFoundException).
        cout << "No desserts yet. ";
        e.printException();
    } catch (NotFoundException& e) {
        cout << "Not on the menu. ";
        e.printException();
    } catch (NothingAboveMinimumValue& e) {
        cout << "Nothing above R" << minPrice << ". ";
        e.printException();
    } catch (Exception& e) {
        // The base class of all the others comes last.
        cout << "Request failed. ";
        e.printException();
    }
    return false;
}
```

Expected output of the given main.cpp

```
Main: first is Beef burger (M) R120, 1 above R100
  returned true
Nothing above R45.5. Exception: Nothing matching filter: above minimum value
  returned false
No desserts yet. Exception: Item not found
  returned false
Request failed. Exception: Invalid type: Drink
  returned false
Not on the menu. Exception: Item not found
  returned false
Not on the menu. Exception: Item not found
  returned false
```

Hints

A1-08 hints

Hint 1

Put both calls, `findFirst` and `numberAbove`, inside one `try` block. Print the summary line after both calls, still inside the `try` block. If a call throws, the rest of the `try` block is skipped.

Hint 2

A `DessertNotFound` object is also a `NotFoundException` object, and both are `Exception` objects. A `catch (Exception& e)` block accepts every one of them. C++ tries the `catch` blocks from top to bottom and uses the first one that fits.

Hint 3

Write the derived class first: `DessertNotFound`, then its base class `NotFoundException`. `NothingAboveMinimumValue` can go anywhere before `Exception`. `Exception` comes last. In the opposite order, g++ warns "exception of type 'DessertNotFound' will be caught by earlier handler", and `-Werror` turns the warning into a build error.

Explanation

What the function must do

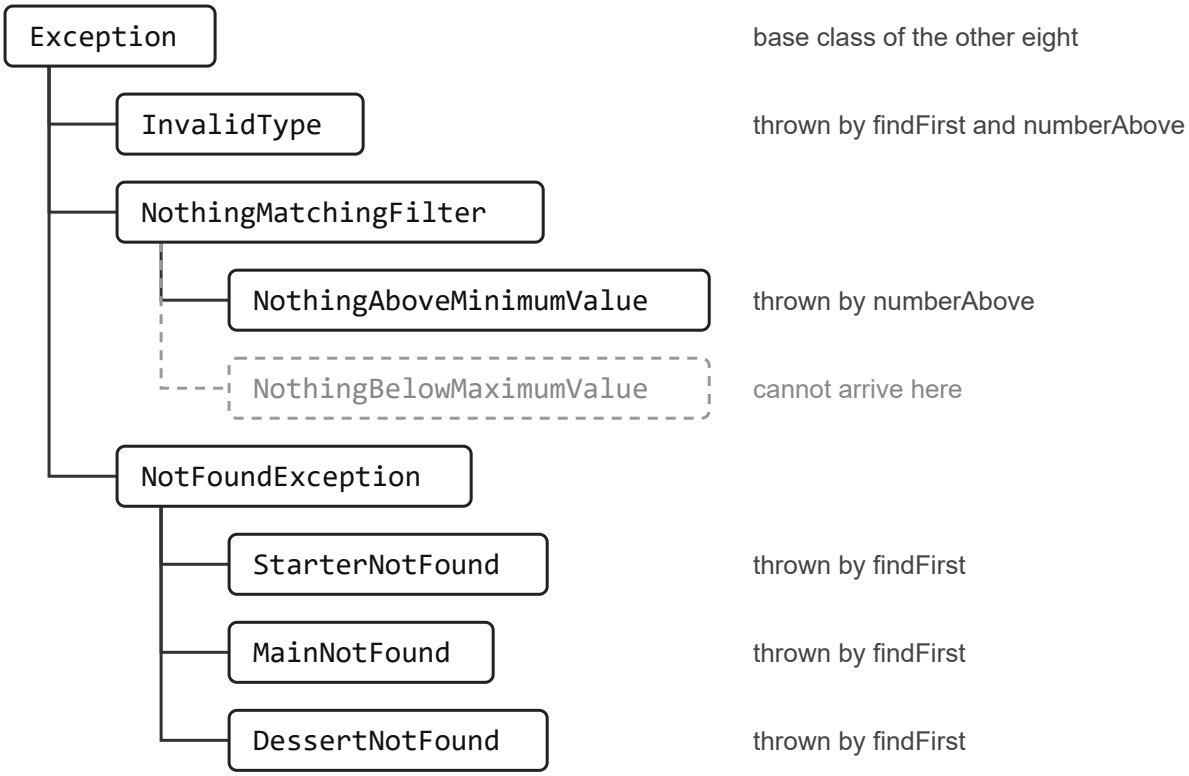
`printSummary(type, minPrice)` is the caller side of the exceptions. It calls two given functions that can throw, `findFirst` and `numberAbove`. If both calls work, it prints one summary line and returns `true`. If either call throws, it catches the exception, prints a line that depends on the exception's class, and returns `false`. No exception may leave the function.

The function is not in the exam paper. The exam's catching code needs the same skill: `catch` blocks by reference, in the right order.

The classes that can arrive

Call	Can throw
`findFirst(type)`	`InvalidType`, `StarterNotFound`, `MainNotFound`, `DessertNotFound`
`numberAbove(type, minPrice)`	`InvalidType`, `NothingAboveMinimumValue`

The family tree of all nine exception classes is below. `NothingBelowMaximumValue` is drawn in grey: neither call throws it, so it cannot arrive in `printSummary`.



An object of a derived class is also an object of every class above it. A `DessertNotFound` object is a `NotFoundException` and an `Exception` too.

How the solution works

The `try` block.

```
try {
    Item first = findFirst(type);
    int count = numberAbove(type, minPrice);
    cout << type << ": first is " << first << ", " << count << " above R" << minPrice << endl;
    return true;
}
```

- The two calls run in order. If `findFirst` throws, `numberAbove` never runs. If `numberAbove` throws, the `cout` line never runs.
- The `cout` line comes after both calls. So the summary line appears only when both calls worked. Nothing of it is printed for a failed call.
- `Item first = findFirst(type);` declares and fills `first` in one statement. `Item` has no default constructor, so `Item first;` on its own line does not compile.
- `return true;` is inside the `try` block. It runs only when nothing was thrown.

The `catch` blocks. When an exception is thrown, C++ looks at the `catch` blocks from top to bottom. It runs the first block whose class fits the object, and skips the rest.

```
} catch (DessertNotFound& e) {
    cout << "No desserts yet. ";
    e.printException();
} catch (NotFoundException& e) {
    cout << "Not on the menu. ";
```

```
        e.printException();
    } catch (NothingAboveMinimumValue& e) {
        cout << "Nothing above R" << minPrice << ". ";
        e.printException();
    } catch (Exception& e) {
        cout << "Request failed. ";
        e.printException();
    }
    return false;
```

- `DessertNotFound&` comes before `NotFoundException&`. In the other order, the `NotFoundException&` block would catch every `DessertNotFound`, and the `DessertNotFound&` block could never run. g++ reports this as "exception of type 'DessertNotFound' will be caught by earlier handler", and `-Werror` turns it into a build error.
- `NotFoundException&` catches the other two not-found classes: `StarterNotFound` and `MainNotFound`.
- `NothingAboveMinimumValue&` is not related to the not-found classes, so its place among them does not matter. It must come before `Exception&`.
- A `catch (NothingMatchingFilter& e)` block would also catch it here, because no other filter exception can reach `printSummary`. The question asks for `NothingAboveMinimumValue&`.
- `Exception&` comes last. It catches everything that the blocks above did not catch. Here that is `InvalidType`.
- Each block catches by reference (`&`). The object is not copied. Here, `Exception` has a virtual destructor. So catching it by value gives the warning "catching polymorphic type 'class Exception' by value", and `-Werror` turns it into a build error.
- `e.printException()` prints `Exception:`, the message and the newline. The line starts with the block's own prefix, so the two parts form one line.
- When a `catch` block ends, the function continues after the last `catch` block. So `return false;` runs.

Trace

The calls below are in `main.cpp`. Open it to see the full code.

The menu in `main.cpp` has `maxSize` 5: slot 0 Garlic bread (S) R45.5, slot 1 Beef burger (M) R120, slot 2 Veg curry (M) R95.5, slots 3 and 4 `NULL`. The helper `check` calls `printSummary` and then prints `returned true` or `returned false`.

Call	What happens in the `try` block	First `catch` that fits	Line printed
`("Main", 100)`	`findFirst` returns Beef burger. `numberAbove` counts Beef burger (120 > 100); Veg curry fails (95.5 > 100 is false). The count is 1.	none	`Main: first is Beef burger (M) R120, 1 above R100`
`("Starter", 45.5)`	`findFirst` returns Garlic bread. `numberAbove`: 45.5 > 45.5 is false, the count is 0, it throws `NothingAboveMinimumValue`.	`NothingAboveMinimumValue&`	`Nothing above R45.5. Exception: Nothing matching filter: above`

Call	What happens in the `try` block	First `catch` that fits	Line printed
			`minimum value`
`("Dessert", 10)`	`findFirst` finds no `'D'` and throws `DessertNotFound`.	`DessertNotFound&`	`No desserts yet. Exception: Item not found`
`("Drink", 10)`	`findFirst` throws `InvalidType("Drink")`.	`Exception&` (the three blocks above do not fit)	`Request failed. Exception: Invalid type: Drink`
`empty, ("Starter", 0)`	`findFirst` throws `StarterNotFound`.	`NotFoundException&` (`DessertNotFound&` does not fit)	`Not on the menu. Exception: Item not found`
`empty, ("Main", 0)`	`findFirst` throws `MainNotFound`.	`NotFoundException&`	`Not on the menu. Exception: Item not found`

For the `"Drink"` call, C++ checks the blocks in order. An `InvalidType` object is not a `DessertNotFound`, not a `NotFoundException`, not a `NothingAboveMinimumValue`. It is an `Exception`, so the last block runs.

Common mistakes

- **`catch (Exception& e)` first.** No later block can ever run. g++ reports "exception of type 'DessertNotFound' will be caught by earlier handler", with a note "for type 'Exception'". With `-Werror` the build stops, and the mark is 0.
- **`catch (NotFoundException& e)` before `catch (DessertNotFound& e)`.** Same problem, same build error: the `DessertNotFound` block can never run.
- **Catching by value**, as in `catch (Exception e)`. In this question, g++ reports "catching polymorphic type 'class Exception' by value" under `-Wall`, and with `-Werror` the build stops. The warning appears only because `Exception` has a virtual destructor here. If the exam's `Exception` has no virtual function, `catch (Exception e)` compiles without a warning. It still copies only the `Exception` part of the object. Always catch by reference.
- **No `catch (Exception& e)` block.** `InvalidType` then leaves `printSummary`. In `main.cpp`, the call `check(menu, "Drink", 10)` ends the program with "terminate called after throwing an instance of 'InvalidType'". The remaining lines are missing, and valgrind reports the abnormal end. The mark is 0.
- **Printing part of the summary before `numberAbove`.** For example, `cout << type << ": first is " << first << ", ";` placed between the two calls. When `numberAbove` throws, that text is already printed, and the `catch` line continues it: `Starter: first is Garlic bread (S) R45.5,`

`Nothing above R45.5. Exception: ...`. With this mistake, the output of `main.cpp` does not match the `ExpectedOutput` block.

- **`catch (const Exception& e).`** `printException()` is not a `const` member function, so `e.printException()` does not compile on a `const` reference. The error says "passing 'const DessertNotFound' as 'this' argument discards qualifiers".
- **Rethrowing or printing nothing.** The question requires a line for every case and no exception leaving the function.

A_2 Operator overloading: solutions

A2-01. Operator overloading: `operator<<` for `Item` (100)

File: A_2.cpp

```
#include "Item.h"

ostream& operator<<(ostream& out, const Item& item) {
    // A friend may read the private members. Write to out, not to cout, and print no newline.
    out << item.name << " (" << item.type << ") R" << item.price;
    return out;
}
```

Expected output of the given main.cpp

```
Garlic bread (S) R45.5
Beef burger (M) R120 | Malva pudding (D) R60.25
Menu (max 3):
  [0] Garlic bread (S) R45.5
  [1] Veg curry (M) R95.5
  [2] empty
```

Explanation

What the function must do

The function defines what `cout << anItem` prints. For one item it prints the name, the type in round brackets and the price after an `R`:

```
Garlic bread (S) R45.5
```

It prints no newline. It writes to the stream it is given and returns that stream.

How the solution works

```
ostream& operator<<(ostream& out, const Item& item) {
    out << item.name << " (" << item.type << ") R" << item.price;
    return out;
}
```

The signature.

- `operator<<` is a normal function, not a member of `Item`. For `cout << bread`, C++ calls `operator<<(cout, bread)`. The stream is the left operand, so it is the first parameter.
- `ostream& out` is a reference to the caller's stream. The function writes into that stream. A copy of a stream is not allowed, so it must be a reference.
- `const Item& item` is a reference to the item. `const` means the function only reads it. The reference avoids a copy and also accepts a temporary object, such as `Item('S', "Bread of the`

`day", 0)`.

- The function is a `friend` of `Item` (see `Item.h`). A friend may read the private members `item.name`, `item.type` and `item.price`. The word `friend` stays in the class. It is not written in `A_2.cpp`.

The body.

- `out << item.name` prints the name. The name may contain spaces, such as `Garlic bread`. A `string` prints all its characters.
- `" ("` prints a space and an opening bracket.
- `item.type` is a `char`. It prints as one letter, such as `S`.
- `") R"` prints the closing bracket, a space and the letter `R`.
- `item.price` is a `float`. The stream prints it in its default format: no zeros at the end of the number and no more than 6 digits in total. So 120 prints as `120`, 45.5 as `45.5`, and 60.25 as `60.25`.
- No `endl`. The caller decides about the newline.
- The body reads `item.name`, `item.type` and `item.price` directly. This works with any `Item.h`, because the function is a friend. Calling `item.getName()` and the other getters also works here, because they are `const` in this `Item.h`. If a getter is not `const`, it cannot be called on the `const Item&`, and the build fails. Check the header in the exam.

`return out;` returns the same stream. This makes chains work. `cout << burger << " | " << pudding` runs as `operator<<(operator<<(operator<<(cout, burger), " | "), pudding)`. Each call returns `cout`, and the next `<<` writes to it.

Trace

The calls below are in `main.cpp`. Open it to see the full code.

`main.cpp` makes three items and prints them.

Statement	Calls of this function	Printed
`cout << bread << endl;`	`out` is `cout`, `item` is bread	`Garlic bread (S) R45.5`, then `endl` from `main`
`cout << burger << " | " << pudding << endl;`	first with burger, returns `cout`; then `" | "`; then with pudding	`Beef burger (M) R120 | Malva pudding (D) R60.25`

For the first call, the pieces are:

Piece	Output so far
`item.name`	`Garlic bread`
`" ("`	`Garlic bread (`
`item.type`	`Garlic bread (S`
`") R"`	`Garlic bread (S) R`
`item.price`	`Garlic bread (S) R45.5`

Then `cout << menu;` runs the given `operator<<` for `Menu`. It calls this function once for each slot that holds an item: slot 0 (the copy of `bread` made by `new Item(bread)`) and slot 1 (Veg curry). Slot 2 is `NULL`, so the menu prints `empty` and does not call this function.

Common mistakes

- **Ending with `endl`.** Every item then ends its own line. `cout << burger << " | " << pudding` prints `Beef burger (M) R120` and `| Malva pudding (D) R60.25` on two lines, and `main`'s own `endl` adds empty lines. With this mistake, the output of `main.cpp` does not match the `ExpectedOutput` block.
- **Writing to `cout` instead of `out`.** The output of `main.cpp` looks right, because `main` only prints to `cout`. `main.cpp` does not show this mistake. Code that writes items into an `ostringstream` does, for example `text << Item('D', "Ice cream", 35) << ";" << Item('S', "Soup", 50);` and then `cout << "[" << text.str() << "]" << endl;`. With `cout` inside the function, the items appear on the screen at once, and the string stream gets only the `;`. The line becomes `Ice cream (D) R35Soup (S) R50[;]` instead of `[Ice cream (D) R35;Soup (S) R50]`.
- **Printing the price with `fixed << setprecision(2).`** The prices then print as `R45.50` and `R120.00`. With this mistake, the output of `main.cpp` does not match the `ExpectedOutput` block. The settings also stay on the stream after the function returns, so later output changes too.
- **Writing `friend` in `A_2.cpp`.** `friend` is allowed only inside a class. Outside the class it is a compile error.
- **A missing `return out;`.** g++ warns that a non-void function does not return a value, and `-Werror` stops the build.

A2-02. Operator overloading: `operator<<` for `Menu` (prints empty slots) (100)

File: A_2.cpp

```
#include "Menu.h"

ostream& operator<<(ostream& out, const Menu& menu) {
    out << "Menu (max " << menu.maxSize << "):" << endl;
    for (int i = 0; i < menu.maxSize; i++) {
        out << "  [" << i << "] ";
        // An empty slot is NULL. Never dereference it.
        if (menu.items[i] == NULL) {
            out << "empty";
        } else {
            out << *menu.items[i];
        }
        out << endl;
    }
    return out;
}
```

Expected output of the given main.cpp

```
Menu (max 4):
  [0] empty
  [1] empty
  [2] empty
  [3] empty
Menu (max 4):
  [0] Garlic bread (S) R45.5
  [1] Beef burger (M) R120
  [2] empty
  [3] empty
Menu (max 4):
  [0] Garlic bread (S) R45.5
  [1] Beef burger (M) R120
  [2] Malva pudding (D) R60
  [3] Veg curry (M) R95.5
(end)
```

Explanation

What the function must do

The function prints a whole menu: a heading with the array size, then one line per slot. A slot with an item shows the item. A `NULL` slot shows the word `empty`. Every slot gets a line, so the output also shows where the free places are.

```
Menu (max 3):
  [0] Garlic bread (S) R45.5
  [1] Beef burger (M) R120
  [2] empty
```

How the solution works

```
ostream& operator<<(ostream& out, const Menu& menu) {
    out << "Menu (max " << menu.maxSize << "):" << endl;
    for (int i = 0; i < menu.maxSize; i++) {
        out << "  [" << i << "] ";
        if (menu.items[i] == NULL) {
            out << "empty";
        } else {
            out << *menu.items[i];
        }
        out << endl;
    }
    return out;
}
```

The signature. `out` is the caller's stream. `menu` is a `const` reference: the function reads the menu and changes nothing. The function is a `friend` of `Menu`, so it may read the private members `menu.maxSize` and `menu.items`.

The heading. `out << "Menu (max " << menu.maxSize << "):" << endl;` prints the size of the array, not the number of items. It ends with `endl`.

The loop.

- `for (int i = 0; i < menu.maxSize; i++)` visits every slot from 0 to `maxSize - 1`. For a menu of size 0, the condition `0 < 0` is false at once, so only the heading is printed.
- `out << "  [" << i << "] ";` prints the start of the line: two spaces, the index in square brackets, a space. This part is the same for a full slot and an empty slot, so it comes before the `if`.

The condition.

- `menu.items[i] == NULL` is true for an empty slot. The function then prints `empty`.
- Otherwise the slot holds an address. `*menu.items[i]` follows the pointer and gives the `Item`. `out << *menu.items[i]` then calls the given `operator<<` for `Item`, which prints `<name> (<type>) R<price>`.
- The star is required. `out << menu.items[i]` without the star prints the pointer, which is a memory address such as `0x4e1f0c0`.
- The `NULL` check must come before the star. Using `*` on a `NULL` pointer is undefined behaviour and usually crashes the program.

The end of the line. `out << endl;` comes after the `if`, so both kinds of line end the same way.

The return. `return out;` gives the stream back, so `cout << menu << "(end)" << endl;` works.

Trace

The calls below are in `main.cpp`. Open it to see the full code.

`main.cpp` makes a menu of size 4 and prints it three times.

First `cout << menu;`. All four slots are `NULL`.

i	menu.items[i] == NULL	line
0	true	`[0] empty`
1	true	`[1] empty`
2	true	`[2] empty`
3	true	`[3] empty`

Second `cout << menu;`. `operator+` has stored Garlic bread in slot 0 and Beef burger in slot 1.

i	menu.items[i] == NULL	line
0	false	`[0] Garlic bread (S) R45.5`
1	false	`[1] Beef burger (M) R120`
2	true	`[2] empty`
3	true	`[3] empty`

Third print, `cout << menu << "(end)" << endl;`. After `main` adds two more items, all four slots hold items, so the condition is false four times. After the loop, `return out;` returns `cout`, and `main` prints `(end)` on the next line.

Common mistakes

- **No `NULL` check**, as in `out << " [" << i << "] " << *menu.items[i] << endl;` for every slot. The first print of `main.cpp` already reaches a `NULL` slot. Valgrind reports an invalid read, and the program stops with a segmentation fault after `Menu (max 4):`. The mark is 0.
- **Skipping the empty slots.** If the function prints only the slots that hold items, the lines `[i] empty` are missing. The first print of `main.cpp` shows only the heading. With this mistake, the output of `main.cpp` does not match the `ExpectedOutput` block.
- **Printing the pointer.** `out << menu.items[i];` prints an address in hexadecimal instead of the item. With this mistake, the output of `main.cpp` does not match the `ExpectedOutput` block.
- **Writing to `cout` instead of `out`.** `main.cpp` looks right, because it prints to `cout`. `main.cpp` does not show this mistake. Code that writes a menu into an `ostringstream` does: with `cout`, the text appears on the screen at once, and the string stream stays empty.
- **`i <= menu.maxSize`.** The loop reads `menu.items[maxSize]`, one slot past the array. Valgrind reports an invalid read.
- **Printing a count of items in the heading.** The heading shows `maxSize`, the length of the array, even for an empty menu: `Menu (max 4):`.

A2-03. Operator overloading: `operator<<` for `Exception` (100)

File: A_2.cpp

```
#include "Exception.h"

ostream& operator<<(ostream& out, const Exception& e) {
    // msg is private; a friend may read it. No newline, and no call to printException():
    // e is const and printException() is not a const member function.
    out << "Exception: " << e.msg;
    return out;
}
```

Expected output of the given main.cpp

```
Exception: Kitchen closed
Exception: Invalid type: Drink (InvalidType)
Caught: Exception: Nothing matching filter: below maximum value
Caught: Exception: Item not found
Exception: Invalid type: Drink!
```

Explanation

What the function must do

The function prints an exception: `Exception:` and then the message. It prints no newline. It is the text of `printException()` without the `endl`, written to the stream it is given.

```
Exception: Invalid type: Drink
```

How the solution works

```
ostream& operator<<(ostream& out, const Exception& e) {
    out << "Exception: " << e.msg;
    return out;
}
```

The signature.

- `ostream& out` is the caller's stream. The function writes into it and returns it.
- `const Exception& e` is a reference to the exception. It is `const`, so the function may only read it.
- The function is a `friend` of `Exception` (see `Exception.h`). A friend may read the private member `e.msg`. The word `friend` is written only in the class.

The body. `out << "Exception: " << e.msg;` prints the fixed text and then the stored message. There is no `endl`.

Why it works for every subclass. The parameter type is `const Exception&`. A reference to a base class can refer to an object of any derived class. So `cout << InvalidType("Drink")`, `cout << DessertNotFound()` and `cout << e` inside `catch (NotFoundException& e)` all call this one function.

A subclass does not need its own `operator<<`. Every subclass object contains the `msg` of `Exception`, and the subclass constructor already filled it.

Why it does not call `printException()`. `printException()` is not a `const` member function. `e` is a `const` reference, so `e.printException()` does not compile. `printException()` also prints to `cout` and ends the line, so it cannot write to `out`.

`return out;` returns the stream for chains such as `cout << bad << " (InvalidType)" << endl;`.

Trace

The calls below are in `main.cpp`. Open it to see the full code.

Statement in `main.cpp`	Object that `e` refers to	`e.msg`	Printed
`cout << plain << endl;`	`Exception("Kitchen closed")`	`Kitchen closed`	`Exception: Kitchen closed`
`cout << bad << " (InvalidType)" << endl;`	`InvalidType("Drink")`	`Invalid type: Drink`	`Exception: Invalid type: Drink (InvalidType)`
`catch (Exception& e) { cout << "Caught: " << e << endl; }`	a thrown `NothingBelowMaximumValue`	`Nothing matching filter: below maximum value`	`Caught: Exception: Nothing matching filter: below maximum value`
`catch (NotFoundException& e) { cout << "Caught: " << e << endl; }`	a thrown `DessertNotFound`	`Item not found`	`Caught: Exception: Item not found`
`cout << ref << "!" << endl;`	`bad`, through a `const Exception&`	`Invalid type: Drink`	`Exception: Invalid type: Drink!`

In the second row, the function returns `cout`. Then `" (InvalidType)"` is printed on the same line, because the function printed no newline. The last row works the same way with `"!"`.

In the `catch` rows, `e` has the type `Exception&` or `NotFoundException&`. Both convert to `const Exception&` for the call.

Common mistakes

- **Calling `e.printException()`.** `e` is `const` and `printException()` is not a `const` member function. The build stops with the error "passing 'const Exception' as 'this' argument discards qualifiers". The mark is 0.
- **Ending with `endl`.** In `main.cpp`, `(InvalidType)` and `!` then start new lines, and every `endl` in `main` adds an empty line. With this mistake, the output of `main.cpp` does not match the `ExpectedOutput` block.

- **Writing to `cout` instead of `out`.** `main.cpp` prints only to `cout`, so its output looks right. `main.cpp` does not show this mistake. Code that writes two exceptions into an `ostringstream`, with `/` between them, does: with `cout`, the exceptions appear on the screen at once, and the string stream holds only `/`.
- **Printing only the message.** Without `Exception:`, every line misses the prefix. With this mistake, the output of `main.cpp` does not match the `ExpectedOutput` block.
- **Writing one `operator<<` per subclass.** It is not needed and not declared in the header. A definition without a declaration in the class is not a friend, so it cannot read the private `msg`.

A2-04. Operator overloading: `Menu::operator+(Item*)` adds an item or deletes it (100)

File: A_2.cpp

```
#include "Menu.h"

Menu& Menu::operator+(Item* item) {
    for (int i = 0; i < maxSize; i++) {
        if (items[i] == NULL) {
            items[i] = item;
            // Stop at the first empty slot; the item must be stored only once.
            return *this;
        }
    }
    // The menu is full. The menu owns the item, so it must delete it.
    delete item;
    return *this;
}
```

Expected output of the given main.cpp

```
Menu (max 3):
  [0] Garlic bread (S) R45.5
  [1] empty
  [2] empty
Menu (max 3):
  [0] Garlic bread (S) R45.5
  [1] Beef burger (M) R120
  [2] Malva pudding (D) R60
Menu (max 3):
  [0] Garlic bread (S) R45.5
  [1] Beef burger (M) R120
  [2] Malva pudding (D) R60
```

Explanation

What the function must do

`menu + item` adds one item to the menu. The item goes into the first empty (`NULL`) slot. If there is no empty slot, the menu is full: the function deletes the item, and the menu stays as it was. In both cases the function returns the menu itself, so that `menu + a + b;` adds two items.

The menu owns every item that is passed to it. The caller writes `menu + new Item(...)` and never deletes that item. So every item must end up either in the array (and `~Menu()` deletes it later) or deleted by this function. Any other path leaks memory.

How the solution works

```
Menu& Menu::operator+(Item* item) {
    for (int i = 0; i < maxSize; i++) {
        if (items[i] == NULL) {
            items[i] = item;
```

```
            return *this;
        }
    }
    delete item;
    return *this;
}
```

The signature. `operator+` is a member function. In `menu + p`, the left operand `menu` is the object (`this`), and the right operand `p` is the parameter `item`. The return type `Menu&` is a reference to a menu. The function returns the same menu, not a new one.

The loop.

- `for (int i = 0; i < maxSize; i++)` visits the slots from the front. The first `NULL` slot it meets has the lowest index.
- `items[i] == NULL` is true for an empty slot.
- `items[i] = item;` stores the pointer. The item itself is not copied. The array now holds its address.
- `return *this;` ends the function at once. This is what makes the item go into one slot only. `this` is a pointer to the menu, and `*this` is the menu itself. Returning it as `Menu&` returns a reference to the menu.

After the loop. The loop ends without a `return` only when no slot was `NULL`. The menu is full. `delete item;` frees the item, because no slot will ever hold it and the caller will not delete it. Then the function returns the menu.

How the chain works. `menu + a + b` is read as `(menu + a) + b`. The first call returns `menu` by reference, so the second call is again `menu + b`. If the return type were `Menu` (a copy), the second item would go into a temporary copy of the menu.

Trace

The calls below are in `main.cpp`. Open it to see the full code.

`main.cpp` makes `Menu menu(3);`. All three slots start as `NULL`.

`menu + new Item('S', "Garlic bread", 45.5);`

i	items[i] == NULL	action
0	true	store Garlic bread in slot 0, return

`menu + new Item('M', "Beef burger", 120) + new Item('D', "Malva pudding", 60);` The first call:

i	items[i] == NULL	action
0	false	next slot
1	true	store Beef burger in slot 1, return

It returns `menu`, and the second call runs on it:

i	items[i] == NULL	action
0	false	next slot
1	false	next slot
2	true	store Malva pudding in slot 2, return

`menu + new Item('M', "Veg curry", 95.5);`

i	items[i] == NULL	action
0	false	next slot
1	false	next slot
2	false	next slot

`i` becomes 3, and `3 < 3` is false. The loop ends. The function deletes Veg curry and returns. The third print of the menu is the same as the second.

At the end of `main`, `~Menu()` deletes the three items in the array. Veg curry was already deleted by `operator+`. Valgrind reports "All heap blocks were freed".

Common mistakes

- **No `delete item;` when the menu is full.** Veg curry is then never freed. The output is right, but valgrind reports `definitely lost` memory for the `new Item('M', "Veg curry", 95.5)` in `main.cpp`. A leak costs the memory marks.
- **No `return` inside the loop.** The loop then stores the same pointer in every `NULL` slot. The first call puts Garlic bread in slots 0, 1 and 2. The menu prints Garlic bread three times, and `~Menu()` deletes the same item three times. Valgrind reports memory errors (an `Invalid read of size 8` of the item that was already deleted). With this mistake, the output of `main.cpp` does not match the `ExpectedOutput` block.
- **Replacing the last item when the menu is full**, as in `delete items[maxSize - 1]; items[maxSize - 1] = item;`. The question says the menu does not change. The third print then shows Veg curry instead of Malva pudding. With this mistake, the output of `main.cpp` does not match the `ExpectedOutput` block.
- **Returning a copy (`Menu` instead of `Menu&`).** The header fixes the return type as `Menu&`, so a different return type in `A_2.cpp` does not compile.
- **Making a copy of the item (`new Item(*item)`).** The original item is then never deleted, and valgrind reports a leak for each call.

A2-05. Operator overloading: `Menu::operator[]` returns `Item*&` (100)

File: A_2.cpp

```
#include "Menu.h"

Item*& Menu::operator[](int index) {
    // Return the slot itself (a reference to the pointer in the array), not a copy of it.
    return items[index];
}
```

Expected output of the given main.cpp

```
Slot 1: Beef burger (M) R120
Slot 2 is empty
Menu (max 3):
  [0] Soup (S) R50
  [1] Beef burger (M) R120
  [2] Malva pudding (D) R60
```

Explanation

What the function must do

`menu[i]` gives access to slot `i` of the array `items`. The return type `Item*&` is a reference to a pointer. A reference is another name for an existing variable. So `menu[i]` is another name for the array element `items[i]` itself. The caller can read it (`*menu[1]`, `menu[2] == NULL`) and can also store a new pointer in it (`menu[2] = new Item(...)`).

The function is not in the exam paper.

How the solution works

```
Item*& Menu::operator[](int index) {
    return items[index];
}
```

Reading the return type. Read `Item*&` from the right: a reference (`&`) to a pointer (`*`) to an `Item`. The array `items` has the type `Item**`, so each element `items[index]` has the type `Item*`. Returning it as `Item*&` returns the element, not a copy of the address in it.

Why one line is enough. `items[index]` is an element of the array. The array lives on the heap until `~Menu()` deletes it, so the reference stays valid after the function returns.

What the caller can do with it.

Expression in `main.cpp`	Meaning
`*menu[1]`	`menu[1]` is the pointer in slot 1; the star gives the `Item`
`menu[2] == NULL`	reads the pointer in slot 2 and compares it with `NULL`

Expression in `main.cpp`	Meaning
`menu[2] = new Item(...)`	stores a new address in slot 2 of the array
`delete menu[0];`	deletes the item that slot 0 points to; the slot still holds the old address
`menu[0] = new Item(...)`	stores a new address in slot 0

The assignment `menu[2] = ...` works only because the function returns a reference. With a return type of `Item*`, `menu[2]` would be a temporary copy of the address, and assigning to it would not compile.

What the function does not do. It does not check the index (the question says the index is always valid). It does not make or delete anything. Managing the items stays with the caller and with `~Menu()`.

Trace

The calls below are in `main.cpp`. Open it to see the full code.

`main.cpp` makes `Menu menu(3);` and adds Garlic bread (slot 0) and Beef burger (slot 1). Slot 2 is `NULL`.

Step	Statement	Slot 0	Slot 1	Slot 2	Printed
1	`cout << "Slot 1: " << *menu[1] << endl;`	Garlic bread	Beef burger	`NULL`	`Slot 1: Beef burger (M) R120`
2	`if (menu[2] == NULL)` is true	Garlic bread	Beef burger	`NULL`	`Slot 2 is empty`
3	`menu[2] = new Item('D', "Malva pudding", 60);`	Garlic bread	Beef burger	Malva pudding	
4	`delete menu[0];`	(deleted; old address)	Beef burger	Malva pudding	
5	`menu[0] = new Item('S', "Soup", 50);`	Soup	Beef burger	Malva pudding	
6	`cout << menu;`				the three items

In step 3, the function returns a reference to `items[2]`, and the assignment writes the address of the new item into the array. In step 4, `menu[0]` gives the pointer in slot 0, and `delete` frees Garlic bread. Step 5 overwrites the old address at once, so the slot never keeps a pointer to freed memory.

At the end, `~Menu()` deletes Soup, Beef burger and Malva pudding. Every `new` has one `delete`.

Common mistakes

- **Returning a reference to a local variable**, as in `Item* slot = items[index]; return slot;`. `slot` is destroyed when the function ends, so the reference would point to nothing. g++ reports "reference to local variable 'slot' returned" (`-Wreturn-local-addr`), and `-Werror` stops the build. The mark is 0.
- **Returning a separate `static` pointer for an empty slot**, as in `static Item* none = NULL; if (items[index] == NULL) return none;`. Reading still looks right. But `menu[2] = new Item(...)` then writes into `none`, not into the array. Malva pudding never reaches the menu: the print shows `[2] empty`, and the item is never deleted. With this mistake, the output of `main.cpp` does not match the `ExpectedOutput` block. `main.cpp` does not show the leak. Two assignments to

empty slots do: the second assignment overwrites `none`, and valgrind reports the first item as `definitely lost`.

- **Return type `Item*` in `A_2.cpp`.** It does not match the declaration `Item*& operator[](int index);` in `Menu.h`, so the build fails.
- **Returning `*items[index]`.** That is an `Item`, not an `Item*`. The types do not match, and the build fails.
- **Checking the range and returning `NULL`.** A reference cannot refer to the literal `NULL`. If a range check is needed, the function must throw an exception instead.

A2-06. Operator overloading: `Item::operator==` and `Item::operator<` (100)

File: A_2.cpp

```
#include "Item.h"

bool Item::operator==(const Item& other) const {
    // All three members must match.
    return type == other.type && name == other.name && price == other.price;
}

bool Item::operator<(const Item& other) const {
    // Only the price decides. Equal prices give false.
    return price < other.price;
}
```

Expected output of the given main.cpp

```
Beef burger (M) R120 vs Beef burger (M) R120: == true, < false
Beef burger (M) R120 vs Beef burger (M) R99.5: == false, < false
Beef burger (M) R99.5 vs Beef burger (M) R120: == false, < true
Veg curry (M) R95.5 vs Soup (S) R95.5: == false, < false
Soup (S) R95.5 vs Veg curry (M) R95.5: == false, < false
Cheapest: Veg curry (M) R95.5
```

Hints

A2-06 hints

Hint 1

Both operators are member functions. The left operand is the object itself (`this`). The right operand is the parameter `other`. Inside a member function you may read the private members of `other` too.

Hint 2

Both functions end in `const` in `Item.h`. Write `const` after the parameter list in `A_2.cpp` as well, or the definition does not match the declaration.

Hint 3

`operator==` joins three comparisons with `&&`. `operator<` compares one member only. For two equal prices, `a < b` and `b < a` are both false.

Explanation

What the functions must do

- `a == b` is `true` when the two items have the same type, the same name and the same price.

- `a < b` is `true` when the price of `a` is less than the price of `b`. The type and the name play no part. Equal prices give `false`.

Both operators are `const` member functions. They are not in the exam paper.

How the solution works

```
bool Item::operator==(const Item& other) const {
    return type == other.type && name == other.name && price == other.price;
}

bool Item::operator<(const Item& other) const {
    return price < other.price;
}
```

Member operators. For `a == b`, C++ calls `a.operator==(b)`. Inside the function, `type`, `name` and `price` are the members of `a` (the object `this` points to). `other` is `b`. A member function may read the private members of any `Item`, so `other.type` is allowed.

The parameter. `const Item& other` avoids a copy, and it promises that `other` is not changed. It also accepts a temporary item, as in a call such as `compare(Item('S', "Soup", 50), Item('S', "Soup", 50))`.

The `const` at the end. `) const {` promises that the function does not change `this` object. It must be written in `A_2.cpp` as well, because it is part of the function's signature. It is also needed for the calls in `main.cpp`: there the items are `const Item&` parameters, and only `const` member functions can be called on them.

`operator==`, condition by condition.

- `type == other.type` compares two `char` values.
- `name == other.name` compares two `string` values character by character.
- `price == other.price` compares two `float` values exactly. The prices in this question have an exact `float` value, so this is safe here.
- `&&` makes the result `true` only when all three parts are `true`. It stops at the first `false` part.

`operator<`. `price < other.price` is already a `bool`, so the function returns it directly. For equal prices, `<` gives `false`, as the question requires.

Trace

The calls below are in `main.cpp`. Open it to see the full code.

`main.cpp` makes five items: burger (M, Beef burger, 120), curry (M, Veg curry, 95.5), burgerCopy (a copy of burger), cheapBurger (M, Beef burger, 99.5) and soup (S, Soup, 95.5). `compare(a, b)` prints both items, then `a == b` and `a < b`.

Call	type equal	name equal	price equal	==	a.price < b.price	<
`compare(burger, burgerCopy)`	true	true	true	true	120 < 120: false	false

Call	type equal	name equal	price equal	==	a.price < b.price	<
`compare(burger, cheapBurger)`	true	true	false	false	120 < 99.5: false	false
`compare(cheapBurger, burger)`	true	true	false	false	99.5 < 120: true	true
`compare(curry, soup)`	false	not evaluated	not evaluated	false	95.5 < 95.5: false	false
`compare(soup, curry)`	false	not evaluated	not evaluated	false	95.5 < 95.5: false	false

The last part of `main.cpp` finds the cheapest item with `<`. `list` holds burger, curry, cheapBurger and soup, and `cheapest` starts at 0.

i	test *list[i] < *list[cheapest]	result	cheapest after
1	curry 95.5 < burger 120	true	1
2	cheapBurger 99.5 < curry 95.5	false	1
3	soup 95.5 < curry 95.5	false	1

The output is `Cheapest: Veg curry (M) R95.5`. Soup has the same price, but `<` is false for equal prices, so the first cheapest item stays.

Common mistakes

- **`operator==` compares only the names.** Burger (R120) and cheapBurger (R99.5) then count as equal. The line `compare(burger, cheapBurger)` shows `== true`. With this mistake, the output of `main.cpp` does not match the `ExpectedOutput` block.
- **`operator==` leaves out the type.** `main.cpp` gives the right output, because no two items there share a name and a price. `main.cpp` does not show this mistake. A call such as `compare(Item('S', "Chocolate", 30), Item('D', "Chocolate", 30))` does: this answer says `== true` instead of `== false`.
- **`operator<` uses `<=`.** Equal prices then give `true`. `compare(burger, burgerCopy)` shows `< true`, and the cheapest item becomes Soup. With this mistake, the output of `main.cpp` does not match the `ExpectedOutput` block.
- **Leaving out `const` after the parameter list in `A_2.cpp`.** The definition then does not match the declaration in `Item.h`. g++ reports "no declaration matches", and the build stops. The mark is 0.
- **Comparing addresses**, as in `return this == &other;`. Two different objects with the same values (burger and burgerCopy) are then not equal. `compare(burger, burgerCopy)` then prints `== false` instead of `== true`.
- **Writing the operators as free functions** (`bool operator==(const Item& a, const Item& b)`). The header declares member functions, so the free functions are never used, and the members stay undefined. The build fails at link time.

A2-07. Operator overloading: `Menu::operator+=` grows the array when the menu is full (100)

File: A_2.cpp

```
#include "Menu.h"

Menu& Menu::operator+=(Item* item) {
    // Use a free slot if there is one.
    for (int i = 0; i < maxSize; i++) {
        if (items[i] == NULL) {
            items[i] = item;
            return *this;
        }
    }
    // The menu is full: make an array with one more slot.
    Item** bigger = new Item*[maxSize + 1];
    for (int i = 0; i < maxSize; i++) {
        bigger[i] = items[i];
    }
    bigger[maxSize] = item;
    // Delete the old array of pointers only. The items now belong to the new array.
    delete[] items;
    items = bigger;
    maxSize++;
    return *this;
}
```

Expected output of the given main.cpp

```
Menu (max 2):
  [0] Garlic bread (S) R45.5
  [1] Beef burger (M) R120
Menu (max 3):
  [0] Garlic bread (S) R45.5
  [1] Beef burger (M) R120
  [2] Malva pudding (D) R60
Menu (max 5):
  [0] Garlic bread (S) R45.5
  [1] Beef burger (M) R120
  [2] Malva pudding (D) R60
  [3] Veg curry (M) R95.5
  [4] Ice cream (D) R35
```

Explanation

What the function must do

`menu += item` adds one item, like `operator+`. The difference is the full menu. `operator+` deletes the item. `operator+=` makes room: it replaces the array with one that is one slot longer and puts the item in the new last slot.

The function is not in the exam paper. A growing array always needs four steps: make a new array, copy the pointers, free the old array, update the size.

How the solution works

Part 1: use a free slot.

```
    for (int i = 0; i < maxSize; i++) {
        if (items[i] == NULL) {
            items[i] = item;
            return *this;
        }
    }
```

This is the loop of `operator+`. If a slot is `NULL`, the item goes there and the function ends. The array keeps its size.

Part 2: the menu is full, so grow.

```
    Item** bigger = new Item*[maxSize + 1];
```

`new Item*[maxSize + 1]` makes an array of `maxSize + 1` pointers on the heap. `bigger` points to it. The pointers inside are not set yet.

```
    for (int i = 0; i < maxSize; i++) {
        bigger[i] = items[i];
    }
    bigger[maxSize] = item;
```

- The loop copies the addresses from slots 0 to `maxSize - 1`. It does not copy the items. Both arrays now point to the same items.
- `bigger[maxSize]` is the new last slot. The old array had no slot with this index. The new item goes there. Every slot of `bigger` now has a value.

```
    delete[] items;
    items = bigger;
    maxSize++;
    return *this;
```

- `delete[] items;` frees the old array of pointers. It uses `delete[]` because the array was made with `new[]`. It does not free the items, because `bigger` still points to them.
- `items = bigger;` makes the menu use the new array.
- `maxSize++` records the new length. `operator<<` and `~Menu()` both loop up to `maxSize`, so they now see the new slot.
- `return *this;` returns the menu as `Menu&`, so `(menu += a) += b;` works.

Do the steps in this order: copy, then `delete[] items`. The copy loop reads `items[i]`, so the old array must still exist.

Trace

The calls below are in `main.cpp`. Open it to see the full code.

`main.cpp` makes `Menu menu(2);`.

`menu += Garlic bread` and `menu += Beef burger`. Part 1 finds slot 0, then slot 1. The array does not grow. The first print shows `Menu (max 2):` with two items.

`menu += new Item('D', "Malva pudding", 60);` Part 1 finds no `NULL` slot. Part 2 runs with `maxSize` 2:

Step	items (old array)	bigger	maxSize
`new Item*[3]`	[Garlic, Beef]	[?, ?, ?]	2
copy, `i = 0`	[Garlic, Beef]	[Garlic, ?, ?]	2
copy, `i = 1`	[Garlic, Beef]	[Garlic, Beef, ?]	2
`bigger[2] = item`	[Garlic, Beef]	[Garlic, Beef, Malva]	2
`delete[] items`	freed	[Garlic, Beef, Malva]	2
`items = bigger; maxSize++`	(now the new array)		3

The second print shows `Menu (max 3):` with three items.

`(menu += Veg curry) += Ice cream;` The menu is full again. The first call grows the array from 3 to 4 slots and puts Veg curry in slot 3. It returns `menu`. The second call finds no `NULL` slot either, grows the array from 4 to 5 slots and puts Ice cream in slot 4. The third print shows `Menu (max 5):` with five items.

At the end, `~Menu()` deletes the five items and the last array. Each old array was freed during its own `operator+=` call.

Common mistakes

- **No `delete[] items;`.** The old arrays of pointers are never freed. The output is right, but valgrind reports `definitely lost` memory: one block for each time the menu grew. Memory marks are lost.
- **Deleting the items as well**, with a loop of `delete items[i];` before `delete[] items;`. The new array then points to freed items. The next print reads them, and at the end `~Menu()` deletes them a second time. The freed memory can still hold the old values, so the printed output can look right. Valgrind still reports many errors, starting with `Invalid read`.
- **Forgetting `maxSize++`.** The new last slot is outside the loops of `operator<<` and `~Menu()`. The new item is not printed, and it is never deleted. The second print shows `Menu (max 2):` again, and valgrind reports a leak.
- **Growing every time, even when a slot is free.** Without Part 1, each of the first two `+=` calls adds a slot to the new menu of size 2. The first print then shows `Menu (max 4):`, with two empty slots in front and the two items in slots 2 and 3. With this mistake, the output of `main.cpp` does not match the `ExpectedOutput` block.
- **`delete items;` instead of `delete[] items;`.** Memory made with `new[]` must be freed with `delete[]`. Valgrind reports `Mismatched free() / delete / delete []`.
- **Copying the items with `new Item(*items[i])`.** The old items are then never deleted, and valgrind reports a leak for each one.

A2-08. Operator overloading: `Menu::operator-(string)` removes an item by name (100)

File: A_2.cpp

```
#include "Menu.h"

Menu& Menu::operator-(string name) {
    for (int i = 0; i < maxSize; i++) {
        if (items[i] != NULL && items[i]->getName() == name) {
            delete items[i];
            // Move every later pointer one slot to the front.
            for (int j = i; j < maxSize - 1; j++) {
                items[j] = items[j + 1];
            }
            // The last slot now has no item of its own.
            items[maxSize - 1] = NULL;
            return *this;
        }
    }
    // No item has this name: the menu does not change.
    return *this;
}
```

Expected output of the given main.cpp

```
Menu (max 4):
  [0] Garlic bread (S) R45.5
  [1] Malva pudding (D) R60
  [2] empty
  [3] empty
Menu (max 4):
  [0] Malva pudding (D) R60
  [1] empty
  [2] empty
  [3] empty
Menu (max 4):
  [0] Malva pudding (D) R60
  [1] Veg curry (M) R95.5
  [2] empty
  [3] empty
```

Explanation

What the function must do

`menu - "Beef burger"` removes the first item with that name. The function deletes the item, then moves every later pointer one slot to the front, so that the filled slots stay together at the front of the array. The last slot becomes `NULL`. If no item has the name, nothing changes. The function returns the menu, so `menu - "a" - "b"` works.

The function is not in the exam paper. Removing from a pointer array needs three steps: free the object, close the gap, and leave no slot pointing to freed memory.

How the solution works

Find the item.

```
    for (int i = 0; i < maxSize; i++) {
        if (items[i] != NULL && items[i]->getName() == name) {
```

- The loop goes from the front, so the first match has the lowest index.
- `items[i] != NULL` comes first. For an empty slot, `&&` stops, and `getName()` is never called through `NULL`.
- `items[i]->getName() == name` compares two `string` values exactly.

Delete it.

```
            delete items[i];
```

The menu owns the item, so the menu must free it. After this line, `items[i]` still holds the old address, but the memory there is freed. The next steps overwrite that slot before anything reads it.

Close the gap.

```
            for (int j = i; j < maxSize - 1; j++) {
                items[j] = items[j + 1];
            }
```

- `j` starts at the slot of the removed item.
- Each step copies the pointer from the next slot into slot `j`. This moves the pointers, not the items.
- The condition `j < maxSize - 1` stops at the second-last slot. The last step reads `items[maxSize - 1]`, the last valid slot. With `j < maxSize`, the last step would read `items[maxSize]`, outside the array.
- If the removed item is in the last slot, `j` starts at `maxSize - 1`, the condition is false at once, and nothing moves.

Clear the last slot.

```
            items[maxSize - 1] = NULL;
            return *this;
```

After the moves, the last two slots hold the same pointer, or both hold `NULL`. The last slot must become `NULL`. Otherwise one item would be in two slots, and `~Menu()` would delete it twice. The function then returns and removes no other item.

No match. If the loop ends without a match, the function reaches the last `return *this;` and the menu is unchanged.

Trace

The calls below are in `main.cpp`. Open it to see the full code.

`main.cpp` makes `Menu menu(4);` and adds three items:

Slot	0	1	2	3
Item	Garlic bread	Beef burger	Malva pudding	`NULL`

`menu - "Beef burger";` The outer loop: slot 0 is Garlic bread, no match. Slot 1 matches, so `i` is 1. The function deletes Beef burger. The inner loop runs while `j < 3`:

j	**`items[j] = items[j + 1]`**	**slots 0 to 3 after the step**
1	`items[1] = items[2]` (Malva pudding)	Garlic, Malva, Malva, `NULL`
2	`items[2] = items[3]` (`NULL`)	Garlic, Malva, `NULL`, `NULL`

Then `items[3] = NULL;` (it is already `NULL`) and the function returns. The first print shows Garlic bread, Malva pudding and two empty slots.

`menu - "Pizza";` Slots 0 and 1 hold other names. Slots 2 and 3 are `NULL`, so the left part of `&&` is false. The loop ends without a match. The menu does not change.

`menu - "Garlic bread";` Slot 0 matches, so `i` is 0. After the delete:

j	**`items[j] = items[j + 1]`**	**slots 0 to 3 after the step**
0	`items[0] = items[1]` (Malva pudding)	Malva, Malva, `NULL`, `NULL`
1	`items[1] = items[2]` (`NULL`)	Malva, `NULL`, `NULL`, `NULL`
2	`items[2] = items[3]` (`NULL`)	Malva, `NULL`, `NULL`, `NULL`

Then `items[3] = NULL;`. The second print shows Malva pudding in slot 0.

`menu + new Item('M', "Veg curry", 95.5);` The given `operator+` finds the first `NULL` slot, slot 1. Because the gap was closed, the new item goes right after Malva pudding.

Common mistakes

- **No `delete items[i];`.** The pointer to the item is overwritten by the shift, so the item can never be freed. The output is right, but valgrind reports `definitely lost` memory for Beef burger and Garlic bread. Memory marks are lost.
- **Not setting the last slot to `NULL`.** In `main.cpp` the last slot is already `NULL` every time, so the output is right. `main.cpp` does not show this mistake. Removing the first item from a full menu of size 3 does. The last two slots then hold the same pointer to Ice cream. The print shows Ice cream twice (`[2] Ice cream (D) R35` instead of `[2] empty`), and `~Menu()` deletes it twice. Valgrind reports memory errors (`Invalid read of size 8`).
- **Setting the slot to `NULL` without moving the later items.** The menu then has a gap. After `menu - "Beef burger"`, the print shows `[1] empty` and `[2] Malva pudding`, and the next `operator+` fills the gap. With this mistake, the output of `main.cpp` does not match the `ExpectedOutput` block.
- **Shift loop with `j < maxSize`.** The last step reads `items[maxSize]`, one slot past the array. Valgrind reports `Invalid read of size 8`.
- **Removing every item with the name.** The question asks for the first one only. `main.cpp` does not show this mistake. A menu with two items called `Soup` does: after `menu - "Soup"`, one `Soup` must stay.
- **Continuing the outer loop after the removal.** Without `return`, the loop goes on with the next index. The item that moved into slot `i` is then skipped, and a second item with the same name may

also be removed.

A_3 Assignment operator: solutions

A3-01. Assignment operator: `Item::operator=` (100)

File: A_3.cpp

```
#include "Item.h"

Item& Item::operator=(const Item& other) {
    if (this == &other) {
        return *this;
    }
    type = other.type;
    name = other.name;
    price = other.price;
    return *this;
}
```

Expected output of the given main.cpp

```
a: Soup (S) R50
b: Beef burger (M) R120
after a = b, a: Beef burger (M) R120
type M, name Beef burger, price 120
after a = a, a: Beef burger (M) R120
after a = b = c, a: Malva pudding (D) R60
b: Malva pudding (D) R60
```

Explanation

The answer

```
#include "Item.h"

Item& Item::operator=(const Item& other) {
    if (this == &other) {
        return *this;
    }
    type = other.type;
    name = other.name;
    price = other.price;
    return *this;
}
```

- `Item& Item::operator=(const Item& other)`: the return type is a reference to an `Item`, the class name and `::` come before `operator=`, and the parameter is a `const` reference (copying the parameter would call the copy constructor for nothing).
- `if (this == &other)`: `this` is the address of the item on the left of `=`; `&other` is the address of the item on the right. They are equal only for `a = a;`. For an `Item` nothing goes wrong without this check (nothing is deleted), but writing it every time builds the habit that `Menu::operator=` needs.

- The three assignments copy each member. `name` is a `string`, and `string` has its own `operator=`, so this copies the characters.
- `return *this;`: `this` is a pointer, `*this` is the item itself. Returning it lets `a = b = c;` work: `b = c` runs first and returns `b`, then `a = b` runs.

The test

`main.cpp` checks `a = b` (all three members, printed with `<<` and with the getters), `a = a`, and `a = b = c`. `main_edge.cpp` assigns from a temporary returned by a function, through pointers, and an empty name.

Mistakes that cost marks

- No `return *this;`: `-Wall -Werror` turns "control reaches end of non-void function" into a compile error, so the mark is 0.
- Forgetting one member (often `price`): the program runs, but the printed price is wrong.
- Writing `Item::operator=(...)` with no return type, or leaving out `Item::`: a compile error.

A3-02. Assignment operator: `Menu::operator=` (100)

File: A_3.cpp

```
#include "Menu.h"

Menu& Menu::operator=(const Menu& other) {
    if (this == &other) {
        return *this;
    }

    for (int i = 0; i < maxSize; i++) {
        delete items[i];
    }
    delete[] items;

    maxSize = other.maxSize;
    items = new Item*[maxSize];
    for (int i = 0; i < maxSize; i++) {
        if (other.items[i] == NULL) {
            items[i] = NULL;
        } else {
            items[i] = new Item(*other.items[i]);
        }
    }

    return *this;
}
```

Expected output of the given main.cpp

```
--- 1. a = b, where b is bigger ---
a before:
Menu (max 2):
  [0] Soup (S) R50
  [1] empty
b:
Menu (max 4):
  [0] Garlic bread (S) R45.5
  [1] Beef burger (M) R120
  [2] Malva pudding (D) R60
  [3] empty
a after a = b:
Menu (max 4):
  [0] Garlic bread (S) R45.5
  [1] Beef burger (M) R120
  [2] Malva pudding (D) R60
  [3] empty
--- 2. the copy is deep ---
b after adding Veg curry:
Menu (max 4):
  [0] Garlic bread (S) R45.5
  [1] Beef burger (M) R120
  [2] Malva pudding (D) R60
  [3] Veg curry (M) R95.5
a has not changed:
Menu (max 4):
  [0] Garlic bread (S) R45.5
  [1] Beef burger (M) R120
```

```
  [2] Malva pudding (D) R60
  [3] empty
--- 3. self-assignment ---
a after a = a:
Menu (max 4):
  [0] Garlic bread (S) R45.5
  [1] Beef burger (M) R120
  [2] Malva pudding (D) R60
  [3] empty
--- 4. chaining ---
a after a = b = c:
Menu (max 1):
  [0] Lemon tart (D) R55
b:
Menu (max 1):
  [0] Lemon tart (D) R55
```

Explanation

The answer

```
#include "Menu.h"

Menu& Menu::operator=(const Menu& other) {
    if (this == &other) {
        return *this;
    }

    for (int i = 0; i < maxSize; i++) {
        delete items[i];
    }
    delete[] items;

    maxSize = other.maxSize;
    items = new Item*[maxSize];
    for (int i = 0; i < maxSize; i++) {
        if (other.items[i] == NULL) {
            items[i] = NULL;
        } else {
            items[i] = new Item(*other.items[i]);
        }
    }

    return *this;
}
```

The four steps:

1. **Self-assignment check.** `this == &other` is true only for `a = a;`. Without it, step 2 deletes `a`'s items and array. In step 3, `other` is `a`, so `other.items` is the new array, whose slots were never set. valgrind reports `Conditional jump or move depends on uninitialised value(s)`, and `a`'s items are lost.
2. **Free the old memory.** The first loop deletes every item (`delete` on a `NULL` slot does nothing, so no `if` is needed). Then `delete[] items;` frees the array of pointers. Deleting only the array leaks every item.

3. **Deep copy.** `maxSize = other.maxSize;` must come before `new Item*[maxSize]`, otherwise the new array has the old size. In the loop, the condition `other.items[i] == NULL` is true for an empty slot: the copy gets `NULL` too. Otherwise `new Item(*other.items[i])` calls the `Item` copy constructor, so the copy gets its own item. `items[i] = other.items[i];` would copy only the address, so both menus would delete the same item.
4. `return *this;` so that `a = b = c;` works.

A trace of `a = b;` in main (part 1)

Before: `a` has `maxSize` 2 (`Soup`, empty). `b` has `maxSize` 4 (`Garlic bread`, `Beef burger`, `Malva pudding`, empty).

- Step 1: `this` is `&a`, `&other` is `&b`: different, carry on.
- Step 2: `i = 0` deletes `Soup`; `i = 1` deletes `NULL` (nothing happens). Then the 2-slot array is freed.
- Step 3: `maxSize` becomes 4, a new 4-slot array is made. `i = 0`: `b`'s slot holds Garlic bread, so a new copy goes in. `i = 1`: Beef burger, copied. `i = 2`: Malva pudding, copied. `i = 3`: `NULL`, so `items[3] = NULL`.
- Step 4: returns `a`.

Part 2 adds `Veg curry` to `b`; `a` still prints `[3] empty`, which shows the arrays are separate.

Mistakes that cost marks

- `items = other.items;` (shallow array): both menus share one array. Part 2 shows `Veg curry` in `a` too, and at the end both destructors delete the same items: valgrind reports invalid frees.
- `items[i] = other.items[i];` (shallow slots): each item is deleted twice at the end.
- No self-assignment check: "uninitialised value(s)" errors in part 3, and part 3 prints every slot as `empty`.
- Only `delete[] items;`: every old item leaks.
- Forgetting `maxSize = other.maxSize;`: the new array has the old size. With `a` (size 2) = `b` (size 4), Malva pudding (slot 2) is not copied: wrong output. In part 4, the loop then reads past the end of `c`'s 1-slot array: invalid reads.

A3-03. Copy constructor: `Menu::Menu(const Menu&)` (100)

File: A_3.cpp

```
#include "Menu.h"

Menu::Menu(const Menu& other) {
    maxSize = other.maxSize;
    items = new Item*[maxSize];
    for (int i = 0; i < maxSize; i++) {
        if (other.items[i] == NULL) {
            items[i] = NULL;
        } else {
            items[i] = new Item(*other.items[i]);
        }
    }
}
```

Expected output of the given main.cpp

```
a:
Menu (max 3):
  [0] Garlic bread (S) R45.5
  [1] Beef burger (M) R120
  [2] empty
b, made with Menu b(a):
Menu (max 3):
  [0] Garlic bread (S) R45.5
  [1] Beef burger (M) R120
  [2] empty
a after adding Malva pudding:
Menu (max 3):
  [0] Garlic bread (S) R45.5
  [1] Beef burger (M) R120
  [2] Malva pudding (D) R60
b has not changed:
Menu (max 3):
  [0] Garlic bread (S) R45.5
  [1] Beef burger (M) R120
  [2] empty
c, made with Menu c = b:
Menu (max 3):
  [0] Garlic bread (S) R45.5
  [1] Beef burger (M) R120
  [2] empty
d, a copy of an empty menu:
Menu (max 2):
  [0] empty
  [1] empty
```

Explanation

The answer

```
#include "Menu.h"

Menu::Menu(const Menu& other) {
    maxSize = other.maxSize;
    items = new Item*[maxSize];
    for (int i = 0; i < maxSize; i++) {
        if (other.items[i] == NULL) {
            items[i] = NULL;
        } else {
            items[i] = new Item(*other.items[i]);
        }
    }
}
```

A copy constructor builds a new object, so it has no old memory to free and no self-assignment to check. It is step 3 of `operator=` alone: set `maxSize`, make a new array, and copy each slot. The `if` matters here as it does in `operator=`: `new Item*[maxSize]` does not set the slots to `NULL`, so an empty slot that is skipped holds a random address, and `~Menu()` then deletes it.

`Menu c = b;` looks like an assignment, but `c` is being created on that line, so the copy constructor runs, not `operator=`.

Mistakes that cost marks

- `items = other.items;`: both menus share one array; the second destructor frees everything again (invalid frees).
- Skipping `NULL` slots without setting them to `NULL`: valgrind reports uninitialised values, and the destructor deletes random addresses.
- Writing a return type on a constructor: a compile error.

A3-04. Assignment operator: `Menu::operator=` that keeps its own size (100)

File: A_3.cpp

```
#include "Menu.h"

Menu& Menu::operator=(const Menu& other) {
    if (this == &other) {
        return *this;
    }

    // Empty this menu. The array stays: its size (maxSize) does not change.
    for (int i = 0; i < maxSize; i++) {
        delete items[i];
        items[i] = NULL;
    }

    // Copy slot i of other into slot i of this menu, for every slot that both menus have.
    for (int i = 0; i < maxSize && i < other.maxSize; i++) {
        if (other.items[i] != NULL) {
            items[i] = new Item(*other.items[i]);
        }
    }

    return *this;
}
```

Expected output of the given main.cpp

```
--- 1. b has more items than a can hold ---
a after a = b:
Menu (max 2):
  [0] Garlic bread (S) R45.5
  [1] Beef burger (M) R120
--- 2. c has fewer items than b ---
b after b = c:
Menu (max 4):
  [0] Lemon tart (D) R55
  [1] empty
  [2] empty
  [3] empty
--- 3. the copy is deep ---
b after adding Veg curry:
Menu (max 4):
  [0] Lemon tart (D) R55
  [1] Veg curry (M) R95.5
  [2] empty
  [3] empty
c has not changed:
Menu (max 1):
  [0] Lemon tart (D) R55
--- 4. self-assignment ---
a after a = a:
Menu (max 2):
  [0] Garlic bread (S) R45.5
  [1] Beef burger (M) R120
--- 5. chaining ---
```

```
a after a = b = c:
Menu (max 2):
  [0] Lemon tart (D) R55
  [1] empty
```

Explanation

The answer

```
#include "Menu.h"

Menu& Menu::operator=(const Menu& other) {
    if (this == &other) {
        return *this;
    }

    // Empty this menu. The array stays: its size (maxSize) does not change.
    for (int i = 0; i < maxSize; i++) {
        delete items[i];
        items[i] = NULL;
    }

    // Copy slot i of other into slot i of this menu, for every slot that both menus have.
    for (int i = 0; i < maxSize && i < other.maxSize; i++) {
        if (other.items[i] != NULL) {
            items[i] = new Item(*other.items[i]);
        }
    }

    return *this;
}
```

- The self-assignment check is the same as always.
- The first loop deletes each item **and sets the slot to `NULL`**. The array itself is kept, so its slots would otherwise still hold the addresses of deleted items. If `b` has fewer items than `a` has slots, those slots are printed and later deleted again.
- The second loop runs while `i` is a slot of **both** menus: `i < maxSize && i < other.maxSize`. The first half stops it at the end of this menu's array; the second half stops it at the end of `other`'s array. Inside, the `if` skips `other`'s empty slots: those slots of this menu are already `NULL` from the first loop.
- `return *this;`

A trace of `a = b;` in main (part 1)

`a` has 2 slots (`Soup`, empty). `b` has 4 (`Garlic bread`, `Beef burger`, `Malva pudding`, empty).

- First loop: `Soup` deleted, slot 0 set to `NULL`; slot 1 already `NULL`.
- `i = 0`: `0 < 2` and `0 < 4`, so copy Garlic bread to slot 0. `i = 1`: `1 < 2` and `1 < 4`, so copy Beef burger to slot 1. `i = 2`: `2 < 2` is false, so the loop stops. Malva pudding is not copied.

Mistakes that cost marks

- Not setting the slots to `NULL` after deleting: part 2 (`b = c`, with `b` keeping 4 slots and `c` holding one item) prints deleted items: invalid reads.

- A loop condition with only `i < other.maxSize`: when `other` is bigger, the code writes past the end of this menu's array (invalid write).
- Replacing the array and copying `maxSize` (the usual version): `a` prints 4 slots instead of 2, so the output is wrong.

A3-05. Assignment operator: `Binary::operator=` (a `bool*` array) (100)

File: A_3.cpp

```
#include "Binary.h"

Binary& Binary::operator=(const Binary& other) {
    if (this == &other) {
        return *this;
    }

    delete[] array;

    size = other.size;
    array = new bool[size];
    for (int i = 0; i < size; i++) {
        array[i] = other.array[i];
    }

    return *this;
}
```

Expected output of the given main.cpp

```
a: 01, b: 1101
after a = b, a: 1101 (size 4)
after b[0] = false, b: 0101, a: 1101
after a = a, a: 1101
after a = b = c, a: 000, b: 000
```

Explanation

The answer

```
#include "Binary.h"

Binary& Binary::operator=(const Binary& other) {
    if (this == &other) {
        return *this;
    }

    delete[] array;

    size = other.size;
    array = new bool[size];
    for (int i = 0; i < size; i++) {
        array[i] = other.array[i];
    }

    return *this;
}
```

The same four steps as `Menu::operator=`, with one array instead of an array of pointers:

1. `if (this == &other) return *this;`
2. `delete[] array;` (one `delete[]`, because the array holds `bool`s, not pointers).
3. `size = other.size;`, `array = new bool[size];`, then copy each bit.
4. `return *this;`

Mistakes that cost marks

- No `delete[] array;`: the old array leaks (valgrind "definitely lost").
- `array = other.array;`: both objects share one array, so `b[0] = false;` also changes `a`, and the array is freed twice at the end.
- No self-assignment check: after `delete[] array;`, `array = new bool[size]` is a new array with unset values, and the loop copies it onto itself. valgrind reports uninitialised values when the bits are printed.

B_1 Recursion: solutions

B1-01. Recursion: `Binary::toDecimal` (100)

File: B_1.cpp

```
#include "Binary.h"

// Returns the decimal value of the bits from index 0 up to and including index.
// array[0] is the leftmost bit (the most significant bit).
int Binary::toDecimal(int index) {
    // Invalid index, and also the stop case: index -1 means "no bits left".
    if (index < 0 || index >= size) {
        return 0;
    }
    // Recursive case: the value of the bits before index, shifted one place left,
    // plus this bit.
    if (array[index]) {
        return 2 * toDecimal(index - 1) + 1;
    }
    return 2 * toDecimal(index - 1);
}
```

Expected output of the given main.cpp

```
1011 up to index 3: 11
1011 up to index 1: 2
1011 up to index 0: 1
00000101 whole string: 5
invalid index 4: 0
invalid index -1: 0
```

Explanation

What the function must do

`toDecimal(index)` returns the value of the bits from index 0 up to and including `index`, read as an unsigned binary number. `array[0]` is the leftmost bit and the most significant bit. For `1011`, `toDecimal(3)` is 11. An index below 0 or at `size` or above returns 0. Loops and helper functions score 0.

The idea

Reading a binary number from the left, each new bit doubles the value so far and then adds the new bit. For `1011`: start at 0, then 0×2+1 = 1, 1×2+0 = 2, 2×2+1 = 5, 5×2+1 = 11.

So the value of the bits up to `index` is two times the value of the bits up to `index - 1`, plus the bit at `index`. That sentence is the recursive case. The recursion walks from `index` down to 0. One step further, `index` is -1, which means "no bits": the value is 0. That is the base case.

How the solution works

```
int Binary::toDecimal(int index) {
```

The function is a member of `Binary`, so it can use `array` and `size` directly. It allocates nothing and frees nothing.

```
    if (index < 0 || index >= size) {
        return 0;
    }
```

This one test does two jobs.

- `index < 0` is the base case. Every recursive chain ends with a call on `index - 1 = -1`. No bits are left, so the value is 0. The same test also covers a negative argument from the caller, which the question says returns 0.
- `index >= size` is the invalid-input test. `array` has `size` elements, at indexes 0 to `size - 1`. Without this test, `toDecimal(4)` on a 4-bit string reads `array[4]`, which is outside the array.
- The operator `||` stops at the first true test. Both tests return the same value, so their order does not matter here.
- An empty string has `size` 0. The call `toDecimal(getSize() - 1)` is `toDecimal(-1)`, so it returns 0.

```
    if (array[index]) {
        return 2 * toDecimal(index - 1) + 1;
    }
    return 2 * toDecimal(index - 1);
```

This is the recursive case.

- `array[index]` is a `bool`. The `if` asks: is this bit a 1?
- If it is, the result is twice the value of the bits before it, plus 1. If not, the result is twice that value, plus 0.
- `index - 1` moves one bit to the left. Each call is one step closer to -1, so the recursion ends.
- The function never changes `array`.

Trace

`main.cpp` calls `a.toDecimal(3)` with `a` = `1011` (`array[0]` = 1, `array[1]` = 0, `array[2]` = 1, `array[3]` = 1).

Call	index	Test `index < 0 || index >= 4`	`array[index]`	Waits for	Returns
1	3	false	1	call 2	2 × 5 + 1 = 11
2	2	false	1	call 3	2 × 2 + 1 = 5
3	1	false	0	call 4	2 × 1 = 2
4	0	false	1	call 5	2 × 0 + 1 = 1
5	-1	true (base case)	not read	nothing	0

Calls 1 to 5 go down. The return values come back up: call 5 gives 0, call 4 gives 1, call 3 gives 2, call 2 gives 5, and call 1 gives 11.

The other calls in `main.cpp`:

- `a.toDecimal(1)` runs calls 3, 4 and 5 of the table and returns 2 (the bits `10`).
- `a.toDecimal(0)` returns 2 × 0 + 1 = 1.
- `b.toDecimal(b.getSize() - 1)` with `b` = `00000101` is `toDecimal(7)`. The five leading zeros keep the value at 0. Then 1, 2, 5. It returns 5.
- `a.toDecimal(4)`: `4 >= 4` is true, so it returns 0 at once. No bit is read.
- `a.toDecimal(-1)`: `-1 < 0` is true, so it returns 0.

The output of `main.cpp` is:

```
1011 up to index 3: 11
1011 up to index 1: 2
1011 up to index 0: 1
00000101 whole string: 5
invalid index 4: 0
invalid index -1: 0
```

Common mistakes

- **Only `index < 0` in the test.** The call `toDecimal(4)` on `1011` then reads `array[4]`. valgrind reports `Invalid read of size 1`, and the mark for memory is lost.
- **Forgetting the factor 2.** `return toDecimal(index - 1) + 1;` adds the bits instead of building the number. `1011` gives 3, not 11. The output is wrong.
- **Reading from the wrong end.** A version that starts at index 0 and calls `toDecimal(index + 1)` treats the last bit as the most significant bit. `00000101` read from index 0 gives 160, not 5. In `main.cpp`, `toDecimal(7)` then prints 1, and `1011` up to index 3 prints 1 instead of 11.
- **A base case of `index == 0` that returns `array[0]`.** This works for valid indexes, but `toDecimal(-1)` and an empty string then skip the base case and read `array[-1]`. Keep the `index < 0` test.
- **No base case at all.** g++ may warn `infinite recursion detected`. Under `-Werror` the build fails, and the mark is 0.
- **A loop, or a helper such as `power(2, n)`.** Either one scores 0. The doubling idea avoids any power function.

B1-02. Recursion: `Binary::countOnes` (100)

File: B_1.cpp

```
#include "Binary.h"

// Returns the number of 1 bits from index to the end of the array.
int Binary::countOnes(int index) {
    // Invalid index (less than 0) or past the last bit: nothing to count.
    if (index < 0 || index >= size) {
        return 0;
    }
    // Recursive case: this bit (1 or 0) plus the 1 bits after it.
    if (array[index]) {
        return 1 + countOnes(index + 1);
    }
    return countOnes(index + 1);
}
```

Expected output of the given main.cpp

```
10110 from index 0: 3
10110 from index 2: 2
10110 from index 4: 0
10110 from index 5: 0
10110 from index -1: 0
```

Explanation

What the function must do

`countOnes(index)` returns the number of `true` bits from `index` to the end of the array. An index at the end or past it returns 0. A negative index is invalid and returns 0. Loops and helper functions score 0.

How the solution works

```
int Binary::countOnes(int index) {
```

The function is a member of `Binary`, so it uses `array` and `size` directly. The index moves forward by 1 in each call.

```
    if (index < 0 || index >= size) {
        return 0;
    }
```

- `index >= size` is the base case. The valid indexes are 0 to `size - 1`. When `index` reaches `size`, no bits are left, so the count is 0. The same test covers a caller who passes an index far past the end, such as 50.
- `index < 0` is the invalid-input test from the question. Without it, `countOnes(-1)` reads `array[-1]`, which is before the start of the array.
- An empty string has `size` 0, so any call returns 0 at once.

```
    if (array[index]) {
        return 1 + countOnes(index + 1);
    }
    return countOnes(index + 1);
```

This is the recursive case.

- If the bit at `index` is 1, the answer is 1 plus the number of 1 bits after it.
- If the bit is 0, the answer is the number of 1 bits after it.
- `index + 1` moves one bit to the right, toward `size`, so the recursion ends.

Trace

`main.cpp` calls `a.countOnes(0)` with `a` = `10110` (size 5).

Call	index	Base or invalid?	`array[index]`	Returns
1	0	no	1	1 + call 2 = 1 + 2 = 3
2	1	no	0	call 3 = 2
3	2	no	1	1 + call 4 = 1 + 1 = 2
4	3	no	1	1 + call 5 = 1 + 0 = 1
5	4	no	0	call 6 = 0
6	5	`5 >= 5`: base case	not read	0

The values return from call 6 up to call 1: 0, 0, 1, 2, 2, 3.

The other calls in `main.cpp`:

- `countOnes(2)` starts at call 3 of the table and returns 2.
- `countOnes(4)` starts at call 5 and returns 0 (the last bit is 0).
- `countOnes(5)` is the base case at once and returns 0.
- `countOnes(-1)`: `-1 < 0` is true, so it returns 0 without reading the array.

The output of `main.cpp` is:

```
10110 from index 0: 3
10110 from index 2: 2
10110 from index 4: 0
10110 from index 5: 0
10110 from index -1: 0
```

Common mistakes

- **No `index < 0` test.** `countOnes(-1)` reads `array[-1]`, and then `array[0]` onwards. valgrind reports `Invalid read of size 1` in `main.cpp`.
- **`index > size` instead of `index >= size`.** The call with `index == size` reads `array[size]`, one element past the end. valgrind reports an invalid read.
- **The `1 +` on the wrong branch.** The function then counts the 0 bits. For `10110` it returns 2 instead of 3.

- **Calling `countOnes(index)` instead of `countOnes(index + 1)`.** The argument never changes, so the recursion never ends. The program stops with a stack overflow (valgrind shows `Stack overflow`), and the mark is 0.
- **A loop or a helper function.** Either one scores 0.

B1-03. Recursion: `Binary::isPalindrome` (100)

File: B_1.cpp

```
#include "Binary.h"

// True when the bits from start to end (both included) read the same in both directions.
bool Binary::isPalindrome(int start, int end) {
    // Invalid input: start or end outside the array.
    if (start < 0 || end >= size) {
        return false;
    }
    // Base case: one bit or no bits left in the range.
    if (start >= end) {
        return true;
    }
    // The two outer bits differ: not a palindrome.
    if (array[start] != array[end]) {
        return false;
    }
    // Recursive case: check the range without the two outer bits.
    return isPalindrome(start + 1, end - 1);
}
```

Expected output of the given main.cpp

```
1001 [0, 3]: palindrome
1011 [0, 3]: not a palindrome
10101 [0, 4]: palindrome
1011 [0, 2]: palindrome
1011 [1, 2]: not a palindrome
1001 [0, 4]: not a palindrome
1001 [-1, 2]: not a palindrome
```

Explanation

What the function must do

`isPalindrome(start, end)` checks the bits from `start` to `end`, both included. It returns `true` when they read the same in both directions. A range of one bit or no bits is a palindrome. If `start < 0` or `end >= size`, the input is invalid and the result is `false`. Loops and helper functions score 0.

The idea

Compare the two outer bits. If they differ, the range is not a palindrome. If they are equal, the answer depends only on the range inside them: `start + 1` to `end - 1`. Each call removes two bits, so the range becomes empty or one bit long. That is the base case.

This is the "start, end" index-pair style of the recursive binary search in Lecture 2 (`search(arr, start, end, value)`).

How the solution works

```
bool Binary::isPalindrome(int start, int end) {
    if (start < 0 || end >= size) {
        return false;
    }
```

The invalid-input test comes first, as the question says.

- `start < 0`: an index before the array.
- `end >= size`: an index after the array. The last valid index is `size - 1`.
- For `isPalindrome(4, 3)` on a 3-bit string, `end >= size` is `3 >= 3`, so the input is invalid and the result must be `false`. If the base case `start >= end` (`4 >= 3`) came first, the function would return `true`. So the order matters for invalid input.
- The recursive calls only move inward (`start` up, `end` down), so once the first call passes this test, every later call passes it too.

```
    if (start >= end) {
        return true;
    }
```

This is the base case. `start == end` means one bit is left, and one bit reads the same both ways. `start > end` means no bits are left. This happens for an even number of bits, for example `1001`: after (0, 3) and (1, 2), the next call is (2, 1). An empty string called with `(0, getSize() - 1)` is `(0, -1)`: `-1 >= 0` is false, so the input is valid, and `0 >= -1` is true, so the result is `true`.

```
    if (array[start] != array[end]) {
        return false;
    }
    return isPalindrome(start + 1, end - 1);
```

- If the two outer bits differ, the range is not a palindrome. The function returns `false` at once and makes no more calls.
- Otherwise the function calls itself on the inner range. Both indexes move one step toward the middle, so the range gets shorter by 2 each time and the base case is always reached.
- The function never changes `array`.

Trace

`main.cpp` calls `a.isPalindrome(0, 3)` with `a` = `1001`.

Call	start	end	Invalid?	`start >= end`?	`array[start]`, `array[end]`	Result
1	0	3	no	no	1, 1 (equal)	result of call 2
2	1	2	no	no	0, 0 (equal)	result of call 3
3	2	1	no	yes: base case	not read	`true`

`true` returns through calls 2 and 1, so the line prints `palindrome`.

`b.isPalindrome(0, 3)` with `b` = `1011`:

Call	start	end	`array[start]`, `array[end]`	Result
1	0	3	1, 1 (equal)	result of call 2
2	1	2	0, 1 (differ)	`false`

The other calls in `main.cpp`:

- `c` = `10101` with (0, 4): `array[0]` and `array[4]` are both 1. The next call is (1, 3): `array[1]` and `array[3]` are both 0. The next call is (2, 2): one bit, so `true`.
- `b` with (0, 2) checks `101`: `array[0]` and `array[2]` are both 1. The next call is (1, 1): one bit, so `true`.
- `b` with (1, 2) checks `01`: `array[1]` is 0 and `array[2]` is 1. They differ, so `false`.
- `a` with (0, 4): `4 >= 4`, so the input is invalid: `false`. No bit is read.
- `a` with (-1, 2): `-1 < 0`, so `false`.

The output of `main.cpp` is:

```
1001 [0, 3]: palindrome
1011 [0, 3]: not a palindrome
10101 [0, 4]: palindrome
1011 [0, 2]: palindrome
1011 [1, 2]: not a palindrome
1001 [0, 4]: not a palindrome
1001 [-1, 2]: not a palindrome
```

Common mistakes

- **No range test.** `isPalindrome(0, 4)` on `1001` reads `array[4]`. valgrind reports `Invalid read of size 1`.
- **Moving only one index**, as in `isPalindrome(start + 1, end)`. The function then compares every bit with the last bit. `1001` gives `false`.
- **`start == end` as the only base case.** For an even number of bits, `start` and `end` pass each other and are never equal. The calls continue with `start > end`: `start` runs past the end of the array and `end` runs below 0. The range test does not stop this, because it only checks `start < 0` and `end >= size`. In `main.cpp`, `1001` prints `not a palindrome`, and valgrind reports `Invalid read of size 1`. Use `start >= end`.
- **Returning `true` as soon as one pair is equal.** The inner bits are never checked. `1011` then gives `true`.
- **A loop or a helper function.** Either one scores 0.

B1-04. Recursion: `BinaryStrings::countNegative` (100)

File: B_1.cpp

```
#include "BinaryStrings.h"

// Counts the strings from curr to the end of the list whose first bit is 1.
int BinaryStrings::countNegative(Node<Binary>* curr) {
    // Base case: past the last node (this also covers an empty list).
    if (curr == NULL) {
        return 0;
    }
    // The size test comes first: an empty string has no bit 0 to read.
    if (curr->getData().getSize() > 0 && curr->getData()[0]) {
        return 1 + countNegative(curr->next());
    }
    return countNegative(curr->next());
}
```

Expected output of the given main.cpp

```
Empty: 0 negative
100 -> 0011 -> 1101 -> 0101: 2 negative
from 1101: 1 negative
```

Explanation

What the function must do

`countNegative(curr)` counts the strings from node `curr` to the end of the list whose bit 0 is 1. An empty string has no bit 0, so it does not count. `NULL` gives 0. Loops and helper functions score 0.

How the solution works

The parameter is the node to look at. Each call handles one node and passes the next node to the next call.

```
int BinaryStrings::countNegative(Node<Binary>* curr) {
    if (curr == NULL) {
        return 0;
    }
```

This is the base case. After the last node, `next()` is `NULL`. There are no strings left, so the count is 0. The same test covers an empty list, where `getHead()` is `NULL`. Without it, the next line would call `getData()` through `NULL` and the program would crash.

```
    if (curr->getData().getSize() > 0 && curr->getData()[0]) {
        return 1 + countNegative(curr->next());
    }
    return countNegative(curr->next());
```

- `curr->getData()` is the `Binary` in this node. `getData()` returns a reference (`T&`), so no copy is made.
- `getSize() > 0` asks: does the string have at least one bit? For an empty string the array has 0 elements, so `[0]` would read outside it.
- `curr->getData()[0]` calls `Binary::operator[]` and gives bit 0, the sign bit.
- The operator `&&` stops at the first false test. For an empty string, `getSize() > 0` is false, so bit 0 is never read. The order of the two tests matters.
- If both tests are true, this string is negative: 1 plus the count for the rest of the list.
- Otherwise: the count for the rest of the list.
- `curr->next()` moves one node forward, toward `NULL`, so the recursion ends.
- The function changes no pointer and no bit. It allocates nothing.

Trace

In `main.cpp`, the list is `100 -> 0011 -> 1101 -> 0101`. The call is `countNegative(list.getHead())`.

Call	curr	size > 0?	bit 0	Returns
1	`100`	yes	1	1 + call 2 = 1 + 1 = 2
2	`0011`	yes	0	call 3 = 1
3	`1101`	yes	1	1 + call 4 = 1 + 0 = 1
4	`0101`	yes	0	call 5 = 0
5	`NULL`	base case	-	0

The values return from call 5 up: 0, 0, 1, 1, 2.

- The first line of `main.cpp` uses the empty list: `getHead()` is `NULL`, so the base case returns 0 at once.
- `countNegative(third)` starts at `1101`. It runs calls 3, 4 and 5 and returns 1.

The output of `main.cpp` is:

```
Empty: 0 negative
100 -> 0011 -> 1101 -> 0101: 2 negative
from 1101: 1 negative
```

Common mistakes

- **No size test.** `curr->getData()[0]` on an empty string reads the first element of an array of size 0. `main.cpp` does not show this mistake, because its list has no empty string. A list made from `Binary("100")`, `Binary("")` and `Binary("0101")` does: the line still prints `1 negative`, but valgrind reports `Invalid read of size 1`.
- **The tests in the wrong order** (`curr->getData()[0] && curr->getData().getSize() > 0`). Bit 0 is read before the size is checked, which is the same invalid read.
- **Returning 1 at the first negative string**, without the recursive call. The count stops early. In `main.cpp`, the second line then prints `1 negative` instead of `2 negative`.
- **No `NULL` test.** The empty list in `main.cpp` calls `getData()` through `NULL`. valgrind reports `Invalid read`, the program stops with a segmentation fault (`SIGSEGV`), and the mark is 0.
- **Copying the Binary**, as in `Binary b = curr->getData();`. This compiles (it uses the copy constructor) and frees its copy at the end of the call, so it gives the correct count. It makes a new

array in every call, which is not needed. `Binary& b = curr->getData();` makes no copy.
- **A loop or a helper function.** Either one scores 0.

B1-05. Recursion: `BinaryStrings::calculatePos` with a node and a bit index (100)

File: B_1.cpp

```
#include "BinaryStrings.h"

// Counts the positive strings from curr to the end of the list.
// A string is positive when its first bit (the sign bit) is 0 and at least one
// later bit is 1. index is the bit of curr that is checked in this call.
int BinaryStrings::calculatePos(Node<Binary>* curr, int index) {
    // Base case: past the last node (this also covers an empty list).
    if (curr == NULL) {
        return 0;
    }
    // Invalid input: index 0 is the sign bit, so the search starts at 1 or later.
    if (index < 1) {
        return 0;
    }
    // This string is not positive: no 1 was found before the end of the string,
    // or the sign bit is 1. The size test comes first, so bit 0 of an empty
    // string is never read. Move to the next node and start again at bit 1.
    if (index >= curr->getData().getSize() || curr->getData()[0]) {
        return calculatePos(curr->next(), 1);
    }
    // A 1 after a 0 sign bit: this string is positive. Count it and move on.
    if (curr->getData()[index]) {
        return 1 + calculatePos(curr->next(), 1);
    }
    // This bit is 0: check the next bit of the same string.
    return calculatePos(curr, index + 1);
}
```

Expected output of the given main.cpp

```
Empty: 0 positive
000 -> 0011 -> 1101 -> 0101: 2 positive
from 1101: 1 positive
invalid index 0: 0
```

Explanation

What the function must do

`calculatePos(curr, index)` counts the positive strings from node `curr` to the end of the list. A string is positive when bit 0 is 0 and at least one other bit is 1. `index` is the bit of `curr` that this call checks. The first call is `calculatePos(getHead(), 1)`. `NULL` gives 0, and an `index` below 1 gives 0. Loops and helper functions score 0.

Why there are two parameters

A solution with loops needs two loops: one over the nodes and one over the bits of each string. With no loops, both walks become recursive calls of the same function:

- `calculatePos(curr, index + 1)` moves to the next **bit** of the same string.
- `calculatePos(curr->next(), 1)` moves to the next **node** and starts again at bit 1.

The same shape works for a grid. One call moves along a row, and another call jumps to the start of the next row. Here each node is like a row, and each bit is like a column.

How the solution works

```
int BinaryStrings::calculatePos(Node<Binary>* curr, int index) {
    if (curr == NULL) {
        return 0;
    }
```

Base case: no node is left, so no string is left to count. This also covers an empty list.

```
    if (index < 1) {
        return 0;
    }
```

Invalid input. Bit 0 is the sign bit, so the search for a 1 starts at bit 1 or later. The recursive calls always pass 1 or more, so only a wrong first call can reach this line.

```
    if (index >= curr->getData().getSize() || curr->getData()[0]) {
        return calculatePos(curr->next(), 1);
    }
```

This test finds a string that is **not** positive. The function then skips the string and moves to the next node at bit 1.

- `index >= curr->getData().getSize()`: the search has passed the last bit of the string without finding a 1. So the string is zero (`000`) or too short to hold a 1 after the sign bit (`0`, `1`, or an empty string).
- `curr->getData()[0]`: the sign bit is 1, so the string is negative.
- The operator `||` stops at the first true test. For an empty string, `index >= 0` is true, so bit 0 of an empty string is never read. The order of the two tests matters.

```
    if (curr->getData()[index]) {
        return 1 + calculatePos(curr->next(), 1);
    }
```

The sign bit is 0 (the test above was false), and bit `index` is 1. So this string is positive. Count it once with `1 +`, then move to the next node at bit 1. The function does not look at the rest of this string, so a string with many 1 bits is still counted once.

```
    return calculatePos(curr, index + 1);
}
```

Bit `index` is 0. The function checks the next bit of the **same** node. `index + 1` moves toward the size of the string, so this inner walk always ends at the test `index >= size`.

The function changes no pointer and no bit, and it allocates nothing.

Trace

In `main.cpp`, the list is `000 -> 0011 -> 1101 -> 0101`. The call is `calculatePos(list.getHead(), 1)`.

In calls 1 to 7, `curr == NULL` is false and `index < 1` is false. The table shows the other three tests. "Not read" means that an earlier test was true, so the call returned before this test.

Call	curr	index	`index >= size`?	Bit 0 is 1?	Bit `index` is 1?	Next call	Adds
1	`000`	1	`1 >= 3`: false	false	false (bit 1 is 0)	(`000`, 2)	0
2	`000`	2	`2 >= 3`: false	false	false (bit 2 is 0)	(`000`, 3)	0
3	`000`	3	`3 >= 3`: true, no 1 found, not positive	not read	not read	(`0011`, 1)	0
4	`0011`	1	`1 >= 4`: false	false	false (bit 1 is 0)	(`0011`, 2)	0
5	`0011`	2	`2 >= 4`: false	false	true (bit 2 is 1): positive	(`1101`, 1)	1
6	`1101`	1	`1 >= 4`: false	true: negative	not read	(`0101`, 1)	0
7	`0101`	1	`1 >= 4`: false	false	true (bit 1 is 1): positive	(`NULL`, 1)	1
8	`NULL`	1	not read (`curr == NULL` is true: base case)	not read	not read	none	returns 0

The return values come back up: call 8 gives 0, call 7 gives 1, call 6 gives 1, call 5 gives 2, and calls 4 to 1 pass the 2 up unchanged. The result is 2.

The other calls in `main.cpp`:

- The empty list: `getHead()` is `NULL`, so the result is 0.
- `calculatePos(third, 1)` starts at `1101`. It runs calls 6, 7 and 8 and returns 1.
- `calculatePos(getHead(), 0)`: `0 < 1`, so the result is 0. No node is visited.

The output of `main.cpp` is:

```
Empty: 0 positive
000 -> 0011 -> 1101 -> 0101: 2 positive
from 1101: 1 positive
invalid index 0: 0
```

Common mistakes

- **Reading the sign bit before the size test** (`curr->getData()[0] || index >= ...`). For an empty string this reads element 0 of an array of size 0. `main.cpp` does not show this mistake, because its list has no empty string. A list made from `Binary("0011")`, `Binary("")` and `Binary("01")` does: the line still prints `2 positive`, but valgrind reports `Invalid read of size 1`.

- **Keeping the old index when moving to the next node**, as in `calculatePos(curr->next(), index)`. The next string is then checked from the wrong bit. `main.cpp` does not show this mistake. The list `0000000001 -> 01` does: it gives 1 instead of 2. The search in `0000000001` ends at index 9, and `01` is then checked from bit 9, which is past its end.
- **Counting every 1 bit**, as in `return 1 + calculatePos(curr, index + 1);`. The string `0111` then counts 3 times.
- **Counting zero as positive.** If the function counts a string as soon as bit 0 is 0, `000` counts. In `main.cpp`, the second line then prints `3 positive` instead of `2 positive`.
- **Calling `calculatePos(curr, index)` with the same arguments.** Nothing changes between calls, so the recursion never ends. The program stops with a stack overflow, and the mark is 0.
- **A loop or a helper function.** Either one scores 0. A loop over the bits inside a recursive walk over the nodes is still a loop.

Note on the name

The exam paper does not explain `calculatePos(): int`. In this question "Pos" means "positive", with bit 0 as a sign bit (0 means not negative, 1 means negative). In the exam, read the question text: "Pos" may mean something else, such as a position. The two-parameter recursion pattern stays the same for any rule that needs a walk over nodes and over bits.

If the exam keeps `calculatePos(): int` with no parameters and its text allows a helper, put this same body in a helper such as `int calculatePos(Node<Binary>* curr, int index)` and call it once: `return calculatePos(head, 1);`. The exam question text decides whether helpers are allowed.

B1-06. Recursion: `BinaryStrings::printReverse` (100)

File: B_1.cpp

```
#include "BinaryStrings.h"

// Prints the strings from the last node back to curr, one string per line.
void BinaryStrings::printReverse(Node<Binary>* curr) {
    // Base case: past the last node, so there is nothing to print.
    if (curr == NULL) {
        return;
    }
    // Recursive case: print the nodes after this one first, then this node.
    printReverse(curr->next());
    cout << curr->getData() << endl;
}
```

Expected output of the given main.cpp

```
000 -> 0011 -> 1101 -> 0101
Reverse:
0101
1101
0011
000
Reverse from the second node:
0101
1101
0011
Reverse of an empty list:
(end)
```

Explanation

What the function must do

`printReverse(curr)` prints the strings of the list in reverse order: from the last node back to `curr`, one string per line, with `cout` and `endl`. A list has only forward links (`next()`), so the function cannot walk backward. If `curr` is `NULL`, it prints nothing. It must not change the list.

How the solution works

```
void BinaryStrings::printReverse(Node<Binary>* curr) {
    if (curr == NULL) {
        return;
    }
```

This is the base case. `NULL` comes after the last node. It is also the head of an empty list. There is nothing to print, so the function returns. The function returns `void`, so `return;` has no value.

```
    printReverse(curr->next());
    cout << curr->getData() << endl;
```

```
}
```

This is the recursive case. The order of the two lines is what reverses the output.

- First the function calls itself for the rest of the list, starting at `curr->next()`. That call prints every later node, from the last one back.
- Only after that call returns does the function print its own string. `curr->getData()` is the `Binary` in this node, and the given `operator<<` prints its bits.
- `curr->next()` moves one node forward, toward `NULL`, so the recursion ends.
- The function changes no pointer, so the list stays the same.

This is the recursive answer to the lecture's `printInReverse`, which uses two nested loops. The call stack remembers the nodes on the way down, and the prints happen on the way back up.

Trace

In `main.cpp`, the list is `000 -> 0011 -> 1101 -> 0101`. The call is `printReverse(list.getHead())`.

Call	curr	`curr == NULL`?	Branch	Prints after the next call returns
1	`000`	false	recursive, calls call 2 first	`000`
2	`0011`	false	recursive, calls call 3 first	`0011`
3	`1101`	false	recursive, calls call 4 first	`1101`
4	`0101`	false	recursive, calls call 5 first	`0101`
5	`NULL`	true	base case, returns	nothing (it makes no call)

The calls return in the order 5, 4, 3, 2, 1. After each return, the caller runs its print line:

- Call 4 prints `0101`.
- Call 3 prints `1101`.
- Call 2 prints `0011`.
- Call 1 prints `000`.

`printReverse(list.getHead()->next())` starts at call 2 and prints `0101`, `1101` and `0011`. The empty list's head is `NULL`, so that call prints nothing.

The output of `main.cpp` is:

```
000 -> 0011 -> 1101 -> 0101
Reverse:
0101
1101
0011
000
Reverse from the second node:
0101
1101
0011
Reverse of an empty list:
(end)
```

Common mistakes

- **Printing before the recursive call.** The strings then appear in forward order: `000`, `0011`, `1101`, `0101`. The output is wrong.
- **Calling `printReverse(curr)` instead of `printReverse(curr->next())`.** The argument never changes, so the recursion never ends. The program compiles, but valgrind reports a stack overflow and the program stops with `SIGSEGV`. Nothing after `Reverse:` is printed, and the mark is 0.
- **No `NULL` test.** Every call makes another call. g++ may warn `infinite recursion detected`. Under `-Werror` the build fails, and the mark is 0.
- **Printing without `endl`, or with a separator.** The strings then run together on one line, for example `010111010011000`. Each string must be on its own line.
- **Printing the bits one by one in reverse.** The question reverses the order of the strings, not the bits inside a string. `0011` stays `0011`.
- **A loop or a helper function.** Either one scores 0.

B1-07. Recursion: `Binary::setBits` (100)

File: B_1.cpp

```
#include "Binary.h"

// Sets array[index], array[index + 1], ... from the same positions in bits.
// A '1' gives true; any other character gives false.
void Binary::setBits(string bits, int index) {
    // Invalid input (index < 0), or the end of the array or of the string: stop.
    // The (int) cast stops the signed/unsigned comparison warning (-Werror).
    if (index < 0 || index >= size || index >= (int)bits.length()) {
        return;
    }
    // Set this bit, then the rest.
    array[index] = (bits[index] == '1');
    setBits(bits, index + 1);
}
```

Expected output of the given main.cpp

```
a: 1011 (size 4)
after setBits("11", 0): 1100
after setBits("0110", 2): 1110
after setBits("111111", 0): 1111
after setBits("0000", -1): 1111
```

Explanation

What the function must do

`setBits(bits, index)` copies characters of `bits` into `array`, from position `index` onward, at the same positions. A `'1'` gives `true`; any other character gives `false`. It stops when `index` reaches the end of the array or the end of the string, the smaller of the two. A negative `index` changes nothing. The given constructor calls `setBits(bits, 0)`, so every `Binary` in the program is filled by this function. Loops and helper functions score 0.

How the solution works

```
void Binary::setBits(string bits, int index) {
```

The function is a member of `Binary`, so it uses `array` and `size` directly. `bits` is passed by value, so each call gets its own copy of the string. This costs a little time, but it does not leak memory: each copy is freed when its call ends.

```
    if (index < 0 || index >= size || index >= (int)bits.length()) {
        return;
    }
```

This one test has three parts. Any true part ends the call.

- `index < 0`: invalid input. Without it, `setBits("0000", -1)` writes to `array[-1]`, which is before the array.
- `index >= size`: the end of the array. This is the normal base case when the string is at least as long as the array. Without it, `setBits("111111", 0)` on a 4-bit `Binary` writes to `array[4]` and `array[5]`.
- `index >= (int)bits.length()`: the end of the string. Without it, a short string such as `"1"` is read past its last character.
- `bits.length()` returns an unsigned type. Comparing it with the `int index` gives the warning `comparison of integer expressions of different signedness`. Under `-Werror` that is a build error. The cast `(int)` removes the warning.
- The function returns `void`, so `return;` has no value.

```
    array[index] = (bits[index] == '1');
    setBits(bits, index + 1);
}
```

- `bits[index] == '1'` is a `bool`: `true` for the character `'1'`, `false` for anything else (`'0'`, `'x'`, a space).
- That value goes into `array[index]`. The array already exists. The constructor made it with `new bool[size]` before the call.
- `setBits(bits, index + 1)` sets the rest, one position to the right. Each call moves toward `size` and toward the string length, so the recursion ends.

Trace

`main.cpp` makes `Binary b("0000")` and then calls `b.setBits("11", 0)`. The array is `0000`, `size` is 4, and the string length is 2.

Call	index	`< 0`?	`>= 4`?	`>= 2`?	Action	Array after
1	0	no	no	no	`array[0]` = (`'1'` == `'1'`) = true	`1000`
2	1	no	no	no	`array[1]` = true	`1100`
3	2	no	no	yes	return	`1100`

Then `b.setBits("0110", 2)`, string length 4:

Call	index	Stops?	Action	Array after
1	2	no	`array[2]` = (`'1'` == `'1'`) = true	`1110`
2	3	no	`array[3]` = (`'0'` == `'1'`) = false	`1110`
3	4	`4 >= 4`	return	`1110`

Bits 0 and 1 are not touched, so they stay `11`.

- `b.setBits("111111", 0)`: calls 0 to 3 set four `true` bits. Call 4 stops at `4 >= size`. The result is `1111`. The last two characters of the string are never read.
- `b.setBits("0000", -1)`: `-1 < 0`, so the first call returns. The array stays `1111`.
- `Binary a("1011")` at the top of `main.cpp`: the constructor calls `setBits("1011", 0)`, which runs 4 setting calls and stops at index 4.

The output of `main.cpp` is:

```
a: 1011 (size 4)
after setBits("11", 0): 1100
after setBits("0110", 2): 1110
after setBits("111111", 0): 1111
after setBits("0000", -1): 1111
```

Common mistakes

- **`index >= bits.length()` without the cast.** g++ reports `comparison of integer expressions of different signedness: 'int' and 'std::__cxx11::basic_string<char>::size_type'`. Under `-Werror` the build fails, and the mark is 0.
- **No test for the end of the string.** The function reads characters after the end of `bits`. Those bytes are not part of the string. In `main.cpp` the output still looks right, but valgrind reports `Conditional jump or move depends on uninitialised value(s)`, so the memory mark is lost.
- **No `index < 0` test.** `setBits("0000", -1)` writes `array[-1]`. valgrind reports `Invalid write of size 1`.
- **Creating the array inside `setBits`** (`array = new bool[size];`). The constructor already did this. Every call then makes a new array and loses the old one: valgrind reports `definitely lost`.
- **Setting `array[index] = bits[index];`** without the comparison. A `char` converted to `bool` is `true` for every character except `'\0'`. So `'0'` becomes `true`, and `0000` prints `1111`.
- **A loop or a helper function.** Either one scores 0.

B_2 Linked lists: solutions

B2-01. Linked lists: `BinaryStrings::append` (100)

File: B_2.cpp

```
#include "BinaryStrings.h"

// Adds a copy of b at the end of the list.
void BinaryStrings::append(const Binary& b) {
    // The node makes its own copy of b (Node<Binary> stores a Binary).
    Node<Binary>* newNode = new Node<Binary>(b);
    if (head == NULL) {
        // Empty list: the new node is the only node.
        head = newNode;
    } else {
        // Walk to the last node: the node whose next() is NULL.
        Node<Binary>* curr = head;
        while (curr->next() != NULL) {
            curr = curr->next();
        }
        curr->next() = newNode;
    }
    size++;
}
```

Expected output of the given main.cpp

```
Empty (size 0)
0101 (size 1)
000 -> 0101 -> 11 -> 1 (size 4)
000 -> 0101 -> 11 -> 1 -> 0000 (size 5)
x is now 1000; the list keeps its own copy
```

Explanation

What the function must do

`append(b)` adds a copy of `b` at the end of the list and adds 1 to `size`. If the list is empty, the new node becomes the head. These are the "empty" and "append" cases of the insertion flowcharts in Lecture 22.

How the solution works

```
void BinaryStrings::append(const Binary& b) {
    Node<Binary>* newNode = new Node<Binary>(b);
```

- `b` is a `const Binary&`: a reference to the caller's object, which the function may not change.
- `new Node<Binary>(b)` creates a node on the heap. The `Node` constructor takes a `T` by value and stores it in `data`, so the node holds its **own** `Binary`, made with the `Binary` copy constructor. The copy has its own bool array.

- The `Node` constructor sets `nextNode` to `NULL`. So the new node is already a correct last node.
- The list now owns this node. `~BinaryStrings()` deletes it later.

```
    if (head == NULL) {
        head = newNode;
    } else {
```

The empty-list case. There is no last node to link from, so the new node becomes the head. Without this case, the walk below would start with `curr` equal to `NULL`.

```
        Node<Binary>* curr = head;
        while (curr->next() != NULL) {
            curr = curr->next();
        }
        curr->next() = newNode;
    }
```

- `curr` starts at the head. The function never moves `head` itself; the lecture warns that moving `head` to walk the list loses the list.
- The loop condition `curr->next() != NULL` is true while `curr` is not the last node. Lecture 22 detects the end in the same way: "the traversal node's next is pointing to NULL".
- The loop stops **on** the last node, not after it. This matters: the function needs the last node to change its link.
- `next()` returns a reference to the link (`Node<T>*&`). So `curr->next() = newNode;` changes the last node's link itself, not a copy.

```
    size++;
}
```

Every path adds one node, so `size` goes up by 1 in both cases.

Trace

`main.cpp` starts with an empty list.

1. `append("0101")`: `head == NULL` is true, so `head = newNode`. The list is `0101`, size 1.
2. `append("11")`: the list is not empty. `curr` = node `0101`. `curr->next()` is `NULL`, so the loop does not run. `0101`'s link is set to the new node. The list is `0101 -> 11`, size 2.
3. `prepend("000")` (given): the list is `000 -> 0101 -> 11`, size 3.
4. `append("1")`:

Loop check	curr	curr->next()	Continue?
1	`000`	`0101`	yes, `curr` = `0101`
2	`0101`	`11`	yes, `curr` = `11`
3	`11`	`NULL`	no

`11`'s link is set to the new node. The list is `000 -> 0101 -> 11 -> 1`, size 4. 5. `append(x)` with `x` = `0000`: the walk goes to `1`, and the new node holds a copy of `x`. Then `main.cpp` sets `x[0] = true`. Only `x` changes, because the node has its own array.

The output of `main.cpp` is:

```
Empty (size 0)
0101 (size 1)
000 -> 0101 -> 11 -> 1 (size 4)
000 -> 0101 -> 11 -> 1 -> 0000 (size 5)
x is now 1000; the list keeps its own copy
```

Common mistakes

- **No empty-list case.** `curr` starts as `NULL`, and `curr->next()` reads through `NULL`. valgrind reports `Invalid read of size 8`, and the program stops with a segmentation fault (`SIGSEGV`). The mark is 0.
- **Forgetting `size++`.** The list prints correctly but `getSize()` is wrong: `0101 (size 0)`.
- **Walking until `curr == NULL`** (`while (curr != NULL)`). The loop ends after the last node, with `curr` equal to `NULL`. The line `curr->next() = newNode;` then writes through `NULL`, and the program crashes.
- **Moving `head` to walk the list.** After the loop, `head` points to the last node, and every earlier node is lost. valgrind reports `definitely lost`.
- **Storing the caller's object instead of a copy.** With `data: T` this is not possible. Because `data` is a `T` (not a `T*`), the node always holds its own copy. In the HRMS practical, `data` was a `T*`, so a node there was made with `new T(value)`.

B2-02. Linked lists: `BinaryStrings::insertAt` (100)

File: B_2.cpp

```
#include "BinaryStrings.h"

// Inserts a copy of b so that it becomes the node at position pos (the head is position 0).
// pos <= 0 inserts at the front. A pos past the end appends.
void BinaryStrings::insertAt(const Binary& b, int pos) {
    Node<Binary>* newNode = new Node<Binary>(b);
    // Front case: also covers the empty list.
    if (pos <= 0 || head == NULL) {
        newNode->next() = head;
        head = newNode;
        size++;
        return;
    }
    // Walk to the node before position pos, or to the last node.
    Node<Binary>* prev = head;
    int count = 1;
    while (prev->next() != NULL && count < pos) {
        prev = prev->next();
        count++;
    }
    // Link the new node in after prev.
    newNode->next() = prev->next();
    prev->next() = newNode;
    size++;
}
```

Expected output of the given main.cpp

```
000 -> 0011 -> 1101 (size 3)
insertAt(1, 0):   1 -> 000 -> 0011 -> 1101 (size 4)
insertAt(01, 2):  1 -> 000 -> 01 -> 0011 -> 1101 (size 5)
insertAt(111, 99): 1 -> 000 -> 01 -> 0011 -> 1101 -> 111 (size 6)
insertAt(10, -3): 10 -> 1 -> 000 -> 01 -> 0011 -> 1101 -> 111 (size 7)
```

Explanation

What the function must do

`insertAt(b, pos)` inserts a copy of `b` so that it becomes the node at position `pos` (the head is position 0). A `pos` of 0 or less inserts at the front. A `pos` at or past the end appends. An empty list gets the new node as its only node. `size` goes up by 1 in every case. These are the rules of the combined `insert(val, pos)` in Lecture 22.

How the solution works

```
void BinaryStrings::insertAt(const Binary& b, int pos) {
    Node<Binary>* newNode = new Node<Binary>(b);
```

The node holds its own copy of `b` (the `Node` constructor copies the `Binary`). Its link starts as `NULL`.

```
    if (pos <= 0 || head == NULL) {
        newNode->next() = head;
        head = newNode;
        size++;
        return;
    }
```

The front case.

- `pos <= 0`: the question puts every position of 0 or less at the front.
- `head == NULL`: in an empty list, the only place is the front, for any value of `pos`.
- `newNode->next() = head;` links the new node to the old first node (or to `NULL` for an empty list).
- `head = newNode;` makes it the first node. The order matters: if `head` changed first, the old first node would be lost.
- `return;` stops here, so the code below runs only for a non-empty list and `pos >= 1`.

```
    Node<Binary>* prev = head;
    int count = 1;
    while (prev->next() != NULL && count < pos) {
        prev = prev->next();
        count++;
    }
```

The walk finds the node **before** position `pos`. The new node goes after it.

- `prev` starts at the head (position 0). `count` is the position right after `prev`, so it starts at 1.
- `count < pos`: while the place after `prev` is still before `pos`, move on. When `count == pos`, the new node belongs right after `prev`.
- `prev->next() != NULL`: stop at the last node. If `pos` is past the end, the walk ends on the last node, and the new node is appended.
- The `&&` checks `prev->next() != NULL` first. Both tests are needed: without the first, a large `pos` walks `prev` into `NULL`.

```
    newNode->next() = prev->next();
    prev->next() = newNode;
    size++;
}
```

- First, the new node points to the node after `prev` (or to `NULL` at the end of the list).
- Then `prev` points to the new node.
- The order matters. If `prev->next() = newNode;` came first, the rest of the list after `prev` would be lost.

Trace

`main.cpp` starts with `000 -> 0011 -> 1101` (size 3).

1. `insertAt("1", 0)`: `0 <= 0`, front case. The list is `1 -> 000 -> 0011 -> 1101`, size 4.
2. `insertAt("01", 2)`:

Loop check	prev	count	prev->next() != NULL	count < 2	Continue?
1	`1`	1	yes	yes	yes: `prev` = `000`, count = 2

Loop check	prev	count	prev->next() != NULL	count < 2	Continue?
2	000	2	yes	no	no

The new node goes after 000: 1 -> 000 -> 01 -> 0011 -> 1101, size 5. 01 is at position 2. 3. insertAt("111", 99) on 1 -> 000 -> 01 -> 0011 -> 1101: 99 <= 0 is false and head is not NULL, so the walk runs.

Loop check	prev	count	prev->next() != NULL	count < 99	Continue?
1	1	1	yes	yes	yes: prev = 000, count = 2
2	000	2	yes	yes	yes: prev = 01, count = 3
3	01	3	yes	yes	yes: prev = 0011, count = 4
4	0011	4	yes	yes	yes: prev = 1101, count = 5
5	1101	5	no (next() is NULL)	yes	no

The walk stops because prev->next() is NULL, not because of count. The new node goes after 1101: 1 -> 000 -> 01 -> 0011 -> 1101 -> 111, size 6. 4. insertAt("10", -3): -3 <= 0, front case: 10 -> 1 -> 000 -> 01 -> 0011 -> 1101 -> 111, size 7.

One case is not in main.cpp: insertAt(b, 5) on an empty list. 5 <= 0 is false, but head == NULL is true, so the front case runs. newNode->next() is NULL, head is newNode, and size is 1.

The output of main.cpp is:

```
000 -> 0011 -> 1101 (size 3)
insertAt(1, 0):   1 -> 000 -> 0011 -> 1101 (size 4)
insertAt(01, 2):  1 -> 000 -> 01 -> 0011 -> 1101 (size 5)
insertAt(111, 99): 1 -> 000 -> 01 -> 0011 -> 1101 -> 111 (size 6)
insertAt(10, -3): 10 -> 1 -> 000 -> 01 -> 0011 -> 1101 -> 111 (size 7)
```

Common mistakes

- **No end-of-list test in the walk** (while (count < pos)). For pos 99, prev becomes NULL and prev->next() reads through NULL. valgrind reports Invalid read of size 8, and the program stops with SIGSEGV. The mark is 0.
- **Starting count at 0.** The walk takes one step too many, so 01 lands at position 3: 1 -> 000 -> 0011 -> 01 -> 1101. The output is wrong.
- **Linking in the wrong order** (prev->next() = newNode; first). The nodes after prev are lost. The list prints short, and valgrind reports definitely lost.
- **No head == NULL in the front test.** For an empty list and pos 5, the walk starts with prev equal to NULL and crashes.
- **Forgetting size++ in the front case.** The return; skips the size++ at the end, so the size is wrong after every front insert.

B2-03. Linked lists: `BinaryStrings::removeAt` (100)

File: B_2.cpp

```
#include "BinaryStrings.h"

// Removes the node at position pos (the head is position 0).
// Returns false, and changes nothing, if pos is not a valid position.
bool BinaryStrings::removeAt(int pos) {
    // Invalid position (this also covers the empty list, where size is 0).
    if (pos < 0 || pos >= size) {
        return false;
    }
    Node<Binary>* temp = NULL;
    if (pos == 0) {
        // Front case: the second node becomes the head.
        temp = head;
        head = head->next();
    } else {
        // Walk to the node before position pos.
        Node<Binary>* prev = head;
        for (int i = 0; i < pos - 1; i++) {
            prev = prev->next();
        }
        // Unlink the node after prev.
        temp = prev->next();
        prev->next() = temp->next();
    }
    // Deleting the node also destroys its Binary (and so its bool array).
    delete temp;
    size--;
    return true;
}
```

Expected output of the given main.cpp

```
1 -> 000 -> 0011 -> 1101 -> 0101 (size 5)
removeAt(2): true   1 -> 000 -> 1101 -> 0101 (size 4)
removeAt(0): true   000 -> 1101 -> 0101 (size 3)
removeAt(2): true   000 -> 1101 (size 2)
removeAt(5): false  000 -> 1101 (size 2)
removeAt(-1): false  000 -> 1101 (size 2)
removeAt(1): true   000 (size 1)
removeAt(0): true   Empty (size 0)
removeAt(0): false  Empty (size 0)
```

Explanation

What the function must do

`removeAt(pos)` removes the node at position `pos` (the head is position 0), deletes it, subtracts 1 from `size` and returns `true`. For a position outside 0 to `size - 1` it changes nothing and returns `false`. These are the "front" and "middle/rear" removal cases of Lecture 23.

How the solution works

```
bool BinaryStrings::removeAt(int pos) {
    if (pos < 0 || pos >= size) {
        return false;
    }
```

- The valid positions are 0 to `size - 1`. `pos >= size` covers the position right after the end (`pos == size`) and every larger one.
- An empty list has `size` 0, so every `pos` is invalid there. No separate `head == NULL` test is needed.
- This test trusts `size`. The given code keeps `size` equal to the number of nodes, and this function keeps it that way.

```
    Node<Binary>* temp = NULL;
    if (pos == 0) {
        temp = head;
        head = head->next();
    } else {
```

The front case (Lecture 23, RemoveFromFront): `temp` remembers the first node, and the head moves to the second node. If the list had one node, `head` becomes `NULL`, which is an empty list.

```
        Node<Binary>* prev = head;
        for (int i = 0; i < pos - 1; i++) {
            prev = prev->next();
        }
        temp = prev->next();
        prev->next() = temp->next();
    }
```

The middle and rear cases.

- The loop moves `prev` `pos - 1` times, so `prev` stops on the node **before** position `pos`. A single forward link cannot reach back, so the function must stop one node early.
- `temp = prev->next();` is the node to remove. It exists, because `pos <= size - 1`.
- `prev->next() = temp->next();` makes `prev` skip `temp`. If `temp` was the last node, `temp->next()` is `NULL`, so `prev` becomes the new last node with a `NULL` link.

```
    delete temp;
    size--;
    return true;
}
```

- `delete temp;` runs `~Node()`, which is empty, and then destroys the node's `data`. That runs `~Binary()`, which frees the bool array. So one `delete` frees both the node and its string.
- `~Node()` does not delete the next node, so the rest of the list is safe. The node is unlinked first, so nothing points to the deleted memory.
- `size--` keeps `size` equal to the number of nodes.

Trace

`main.cpp` starts with `1 -> 000 -> 0011 -> 1101 -> 0101` (size 5).

Call	Valid?	Case	prev	temp	List after	Returns
`removeAt(2)`	0 <= 2 < 5	middle, loop runs once	`000`	`0011`	`1 -> 000 -> 1101 -> 0101`	true
`removeAt(0)`	yes	front	-	`1`	`000 -> 1101 -> 0101`	true
`removeAt(2)`	2 < 3	rear, loop runs once	`1101`	`0101`	`000 -> 1101`	true
`removeAt(5)`	5 >= 2	-	-	-	unchanged	false
`removeAt(-1)`	-1 < 0	-	-	-	unchanged	false
`removeAt(1)`	1 < 2	rear, loop runs 0 times (`pos - 1` is 0)	`000`	`1101`	`000`	true
`removeAt(0)`	yes	front	-	`000`	`Empty`	true
`removeAt(0)`	0 >= 0	-	-	-	`Empty`	false

For the first call, the loop `for (i = 0; i < 1; i++)` runs once and moves `prev` from `1` to `000`. Then `temp` is `0011`, and `000` is linked to `1101`.

The loop can also run more than once. For a call such as `removeAt(3)` on a list `a -> b -> c -> d`, `pos - 1` is 2:

- `i` = 0: `0 < 2` is true, so `prev` moves from `a` to `b`.
- `i` = 1: `1 < 2` is true, so `prev` moves from `b` to `c`.
- `i` = 2: `2 < 2` is false, so the loop ends with `prev` on `c`.

Then `temp` is `d`, the node at position 3.

The output of `main.cpp` is:

```
1 -> 000 -> 0011 -> 1101 -> 0101 (size 5)
removeAt(2): true   1 -> 000 -> 1101 -> 0101 (size 4)
removeAt(0): true   000 -> 1101 -> 0101 (size 3)
removeAt(2): true   000 -> 1101 (size 2)
removeAt(5): false  000 -> 1101 (size 2)
removeAt(-1): false  000 -> 1101 (size 2)
removeAt(1): true   000 (size 1)
removeAt(0): true   Empty (size 0)
removeAt(0): false  Empty (size 0)
```

Common mistakes

- **Unlinking without `delete`.** The output is right, but every removed node and its bool array are lost. valgrind reports `definitely lost`, and the memory mark is lost.
- **`pos > size` instead of `pos >= size`.** With an empty list, `removeAt(0)` gets through (0 is not greater than 0), and `head = head->next();` reads through `NULL`. With a non-empty list, `removeAt(size)` (for example, `removeAt(2)` on a list of 2 nodes) leaves `temp` as `NULL`, and `temp->next()` reads through `NULL`. Both stop with `SIGSEGV`, mark 0. `main.cpp` ends with `removeAt(0)` on an empty list to catch this.
- **Deleting before unlinking.** `delete temp;` followed by `prev->next() = temp->next();` reads a deleted node. valgrind reports `Invalid read`.

- **Deleting the Binary's array as well**, for example through a pointer to `temp->getData()`. The node's destructor already frees it, so this is a double free (`Invalid free()`).
- **Walking `pos` steps instead of `pos - 1`.** `prev` then stops on the node to remove, and the wrong node is deleted.
- **Forgetting `size--`.** The printed list is right, but `getSize()` is wrong, and later calls trust a wrong `size`.

B2-04. Linked lists: `BinaryStrings::remove` the first equal string (100)

File: B_2.cpp

```
#include "BinaryStrings.h"

// Removes the first string in the list that is equal to b: the same size and the
// same bit at every index. Returns true if a node was removed, false otherwise.
bool BinaryStrings::remove(const Binary& b) {
    Node<Binary>* prev = NULL;
    Node<Binary>* curr = head;
    while (curr != NULL) {
        // Compare curr's string with b, bit by bit (Binary has no operator==).
        Binary& data = curr->getData();
        bool same = (data.getSize() == b.getSize());
        for (int i = 0; same && i < b.getSize(); i++) {
            if (data[i] != b[i]) {
                same = false;
            }
        }
        if (same) {
            // Unlink curr: at the front, head moves; otherwise prev skips curr.
            if (prev == NULL) {
                head = curr->next();
            } else {
                prev->next() = curr->next();
            }
            // Deleting the node also destroys its Binary (and so its bool array).
            delete curr;
            size--;
            return true;
        }
        prev = curr;
        curr = curr->next();
    }
    return false;
}
```

Expected output of the given main.cpp

```
0101 -> 0011 -> 101 -> 0011 (size 4)
remove(0011): true   0101 -> 101 -> 0011 (size 3)
remove(11): false  0101 -> 101 -> 0011 (size 3)
remove(101): true   0101 -> 0011 (size 2)
remove(0101): true   0011 (size 1)
remove(0011): true   Empty (size 0)
remove(0011): false  Empty (size 0)
```

Explanation

What the function must do

`remove(b)` finds the first node whose string equals `b`, unlinks it, deletes it, subtracts 1 from `size` and returns `true`. If no string is equal, it returns `false` and changes nothing. `Binary` has no `operator==`, so

the function compares the strings itself: equal means the same size and the same bit at every index. This follows the combined `remove(val)` flowchart of Lecture 23, with a bit-by-bit comparison in place of `==`.

How the solution works

```
bool BinaryStrings::remove(const Binary& b) {
    Node<Binary>* prev = NULL;
    Node<Binary>* curr = head;
```

Two pointers walk the list together. `curr` is the node being checked. `prev` is the node before it, or `NULL` while `curr` is the head. Lecture 23 uses two pointers "such that it is possible to relink the rest of the list correctly".

```
    while (curr != NULL) {
        Binary& data = curr->getData();
        bool same = (data.getSize() == b.getSize());
```

- The loop runs once for each node and ends after the last one. For an empty list it does not run, and the function returns `false`.
- `data` is a reference to the string in the node. No copy is made.
- `same` starts as the size test. Strings of different sizes are never equal, so `011` and `11` are different even though both have the value 3.

```
        for (int i = 0; same && i < b.getSize(); i++) {
            if (data[i] != b[i]) {
                same = false;
            }
        }
```

- The inner loop runs only while `same` is true. If the sizes differ, `same` is already false and the loop does not run at all. This is what stops the loop from reading a bit past the end of the shorter string.
- The loop stops at the first bit that differs, because `same` becomes false.
- `b` is a `const Binary&`, so `b[i]` uses the `const` version of `operator[]`. `data[i]` uses the normal version.
- For two empty strings, the sizes match (0 and 0), the loop does not run, and `same` stays true.

```
        if (same) {
            if (prev == NULL) {
                head = curr->next();
            } else {
                prev->next() = curr->next();
            }
            delete curr;
            size--;
            return true;
        }
```

- `prev == NULL` means `curr` is the head. The head moves to the second node.
- Otherwise `prev` skips `curr` and links to the node after it. If `curr` was the last node, `prev` gets a `NULL` link.
- `delete curr;` frees the node, and the node's `Binary` destructor frees its bool array.
- `return true;` stops at the first match, so later equal strings stay.

```
        prev = curr;
        curr = curr->next();
    }
    return false;
}
```

No match at this node: both pointers move one node forward. If the loop ends, no string matched.

Trace

`main.cpp` starts with `0101 -> 0011 -> 101 -> 0011` (size 4). The first call is `remove("0011")`.

Step	prev	curr	Size test	Bit test	Result
1	`NULL`	`0101`	4 == 4	i = 0: 0 and 0, same; i = 1: `0101` has 1, `0011` has 0, differ, so `same` = false and the loop stops	move on
2	`0101`	`0011`	4 == 4	i = 0: 0 and 0; i = 1: 0 and 0; i = 2: 1 and 1; i = 3: 1 and 1; `same` stays true; i = 4: `4 < 4` is false, so the loop ends	match

`prev` is `0101`, so `0101` is linked to `101`. The first `0011` is deleted. The list is `0101 -> 101 -> 0011`, size 3, and the second `0011` stays.

`remove("11")`: the list holds sizes 4, 3 and 4, and `11` has size 2. So `same` is false at once for every node. The bit loop never runs. The result is `false`.

`remove("101")`: `0101` has size 4 (no), then `101` matches. The list is `0101 -> 0011`.

`remove("0101")`: the head matches. `prev` is `NULL`, so `head` moves to `0011`.

`remove("0011")`: the only node matches. `head` becomes `NULL`. The next `remove("0011")` finds an empty list and returns `false`.

The output of `main.cpp` is:

```
0101 -> 0011 -> 101 -> 0011 (size 4)
remove(0011): true   0101 -> 101 -> 0011 (size 3)
remove(11): false  0101 -> 101 -> 0011 (size 3)
remove(101): true   0101 -> 0011 (size 2)
remove(0101): true   0011 (size 1)
remove(0011): true   Empty (size 0)
remove(0011): false  Empty (size 0)
```

Common mistakes

- **No size test.** The bit loop runs to `b.getSize()`. When the stored string is shorter than `b`, `data[i]` reads past its array. `main.cpp` does not show this mistake. A call such as `remove("11")` on a list that holds only `1` does: the result is still `false`, but valgrind reports `Invalid read of size 1`.
- **Comparing only up to the shorter size.** No invalid read, but `1` and `11` count as equal, and the wrong node is removed.
- **Comparing decimal values.** `011` and `11` then count as equal. The question defines equal as the same size and the same bits.
- **Unlinking without `delete`.** valgrind reports `definitely lost`.

- **Removing every equal string.** The function must stop at the first match. `main.cpp` has `0011` twice, and the second one must stay. With this mistake, the first `remove(0011)` line prints `0101 -> 101 (size 2)`, so the output does not match the `ExpectedOutput` block.
- **Forgetting the head case.** If the function always runs `prev->next() = ...`, removing the head writes through `NULL` and the program crashes.
- **Writing `curr->getData() == b`.** `Binary` has no `operator==`, so this does not compile. The build fails and the mark is 0.

B2-05. Linked lists: `BinaryStrings::reverse` (100)

File: B_2.cpp

```
#include "BinaryStrings.h"

// Reverses the order of the nodes by turning every link around.
// No node is created or deleted.
void BinaryStrings::reverse() {
    Node<Binary>* prev = NULL;
    Node<Binary>* curr = head;
    while (curr != NULL) {
        Node<Binary>* temp = curr->next();   // remember the rest of the list
        curr->next() = prev;                 // turn this link around
        prev = curr;
        curr = temp;
    }
    // prev is the old last node: it is the new head.
    head = prev;
}
```

Expected output of the given main.cpp

```
000 -> 0011 -> 1101 -> 0101 (size 4)
0101 -> 1101 -> 0011 -> 000 (size 4)
000 -> 0011 -> 1101 -> 0101 (size 4)
Empty (size 0)
```

Explanation

What the function must do

`reverse()` puts the nodes in the opposite order by reversing every link. It creates no node and deletes no node. The strings themselves do not change, and `size` does not change.

How the solution works

The function walks the list once with three pointers:

- `curr`: the node whose link is being reversed.
- `prev`: the node before `curr` in the old order. After the link is reversed, `curr` points to it.
- `temp`: the node after `curr` in the old order. It is saved before the link changes.

```
void BinaryStrings::reverse() {
    Node<Binary>* prev = NULL;
    Node<Binary>* curr = head;
```

`prev` starts as `NULL`, because the old head becomes the new last node, and the last node's link must be `NULL`.

```
    while (curr != NULL) {
        Node<Binary>* temp = curr->next();
```

```
        curr->next() = prev;
        prev = curr;
        curr = temp;
    }
```

- `curr != NULL`: the loop runs once per node. For an empty list it does not run.
- `temp = curr->next();` saves the rest of the list. The next line overwrites `curr`'s link, and without `temp` the rest of the list would be lost.
- `curr->next() = prev;` reverses the link. `next()` returns a reference, so this changes the node's own link.
- `prev = curr;` and `curr = temp;` move both pointers one node forward in the old order.

```
    head = prev;
}
```

When the loop ends, `curr` is `NULL` and `prev` is the old last node. That node is the new head. For an empty list, `prev` is still `NULL`, so `head` stays `NULL`.

No `new` and no `delete`: the same nodes are reused, so the memory does not change.

Trace

`main.cpp` starts with `000 -> 0011 -> 1101 -> 0101`.

Iteration	curr	temp	Link set	prev after	Reversed part
1	`000`	`0011`	`000 -> NULL`	`000`	`000`
2	`0011`	`1101`	`0011 -> 000`	`0011`	`0011 -> 000`
3	`1101`	`0101`	`1101 -> 0011`	`1101`	`1101 -> 0011 -> 000`
4	`0101`	`NULL`	`0101 -> 1101`	`0101`	`0101 -> 1101 -> 0011 -> 000`

After iteration 4, `curr` is `NULL` and the loop ends. `head = prev` makes `0101` the head.

The second `reverse()` in `main.cpp` puts the list back in the first order. The empty list stays `Empty`.

A list with one node `a` is not in `main.cpp`, but the loop handles it. Iteration 1: `curr != NULL` is true. `temp` = `NULL`, `a->next()` = `NULL`, `prev` = `a`, and `curr` = `NULL`. Next check: `curr != NULL` is false, so the loop ends. `head` = `a`, so the list is unchanged.

The output of `main.cpp` is:

```
000 -> 0011 -> 1101 -> 0101 (size 4)
0101 -> 1101 -> 0011 -> 000 (size 4)
000 -> 0011 -> 1101 -> 0101 (size 4)
Empty (size 0)
```

Common mistakes

- **Forgetting `head = prev;`.** `head` still points to the old first node, which is now the last node with a `NULL` link. The list prints as `000` only, and the other three nodes are lost: valgrind reports `definitely lost`.

- **No `temp` (`curr = curr->next();` after the link has changed).** `curr->next()` is now the previous node, or `NULL` on the first pass. The loop ends after one node, and the rest of the list is lost. valgrind reports `definitely lost`.
- **Making a new reversed list with `new` and not deleting the old nodes.** The output can look right, but the old nodes leak. The question says the same nodes must be used, and the marking program checks this.
- **Swapping the strings instead of the links.** `Binary`'s `operator=` is private here, so `a->getData() = b->getData();` does not compile. If `Binary` did not declare `operator=`, the compiler would make one. That one copies the array pointer, so two strings would delete the same array (`Invalid free()`).
- **Reversing the bits inside each string.** The question reverses the order of the nodes only.

B2-06. Linked lists: `BinaryStrings::sort` by decimal value (100)

File: B_2.cpp

```
#include "BinaryStrings.h"

// Helper (allowed in B_2): the decimal value of a binary string.
// array[0] is the leftmost, most significant bit. An empty string has value 0.
static int decimalValue(const Binary& b) {
    int value = 0;
    for (int i = 0; i < b.getSize(); i++) {
        value = value * 2;
        if (b[i]) {
            value = value + 1;
        }
    }
    return value;
}

// Sorts the strings by decimal value, smallest first, by relinking the nodes
// (insertion sort). Strings with equal values keep their original order.
void BinaryStrings::sort() {
    Node<Binary>* sorted = NULL;            // head of the sorted part
    Node<Binary>* curr = head;
    while (curr != NULL) {
        Node<Binary>* temp = curr->next();  // remember the rest of the unsorted part
        int value = decimalValue(curr->getData());
        // Case 1: curr goes in front of the sorted part.
        if (sorted == NULL || value < decimalValue(sorted->getData())) {
            curr->next() = sorted;
            sorted = curr;
        } else {
            // Case 2: find the last sorted node with a value <= curr's value,
            // and link curr after it.
            Node<Binary>* prev = sorted;
            while (prev->next() != NULL && decimalValue(prev->next()->getData()) <= value) {
                prev = prev->next();
            }
            curr->next() = prev->next();
            prev->next() = curr;
        }
        curr = temp;
    }
    head = sorted;
}
```

Expected output of the given main.cpp

```
0101 -> 11 -> 1000 -> 011 -> 0 (size 5)
0 -> 11 -> 011 -> 0101 -> 1000 (size 5)
Empty (size 0)
```

Explanation

What the function must do

`sort()` puts the strings in ascending order of decimal value, smallest at the head. `array[0]` is the most significant bit, so `0101` is 5 and `011` is 3. Equal values keep their original order (a stable sort). The function moves whole nodes by changing links. It creates no node, deletes no node, and does not copy strings between nodes. `size` does not change.

Part 1: the helper

```
static int decimalValue(const Binary& b) {
    int value = 0;
    for (int i = 0; i < b.getSize(); i++) {
        value = value * 2;
        if (b[i]) {
            value = value + 1;
        }
    }
    return value;
}
```

- The question allows a helper in `B_2.cpp`. `static` keeps the name inside this file, so it cannot clash with a name in another `.cpp` file.
- The loop reads from the left. Each new bit doubles the value so far and adds the bit. For `0101`: 0, 0, 1, 2, 5.
- `b` is a `const Binary&`, so `b[i]` uses the `const` `operator[]`, and `getSize()` is `const`. Both exist in the given class.
- An empty string never enters the loop, so its value is 0.

Part 2: the sort (insertion sort by relinking)

The function builds a second chain, the **sorted part**. It takes the nodes of the old list one at a time and links each one into the right place in the sorted part.

```
    Node<Binary>* sorted = NULL;
    Node<Binary>* curr = head;
    while (curr != NULL) {
        Node<Binary>* temp = curr->next();
        int value = decimalValue(curr->getData());
```

- `sorted` is the first node of the sorted part. It starts empty (`NULL`).
- `curr` is the node being moved. For an empty list the loop does not run.
- `temp` saves the rest of the old list. The code below changes `curr->next()`, and without `temp` the rest would be lost.
- `value` is `curr`'s value, worked out once.

```
        if (sorted == NULL || value < decimalValue(sorted->getData())) {
            curr->next() = sorted;
            sorted = curr;
        } else {
```

Case 1: `curr` goes in front of the sorted part.

- `sorted == NULL`: the sorted part is empty, so `curr` becomes its only node.
- `value < ...`: `curr` is **strictly** smaller than the first sorted value. If it is equal, it must not go in front, because an equal value that came earlier must stay earlier.

- `||` stops at the first true test, so `sorted->getData()` is never read through `NULL`.
- `curr->next() = sorted;` then `sorted = curr;`. In the other order, the old sorted part would be lost.

```
            Node<Binary>* prev = sorted;
            while (prev->next() != NULL && decimalValue(prev->next()->getData()) <= value) {
                prev = prev->next();
            }
            curr->next() = prev->next();
            prev->next() = curr;
        }
        curr = temp;
    }
    head = sorted;
```

Case 2: `curr` goes after some sorted node.

- `prev` starts at the first sorted node. Case 1 was false, so that node's value is `<= value`.
- `prev->next() != NULL`: stop at the last sorted node. Without it, the next test reads through `NULL`.
- `<= value`: move past every node with a smaller **or equal** value. So `curr` lands after the equal values that came before it. This `<=` and the strict `<` in case 1 make the sort stable.
- `curr->next() = prev->next();` and then `prev->next() = curr;` link `curr` in after `prev`. At the end of the sorted part, `prev->next()` is `NULL`, so `curr` gets a `NULL` link and becomes the last node.
- `curr = temp;` moves to the next node of the old list.
- After the loop, every node is in the sorted part. `head = sorted;` makes it the list.

Trace

`main.cpp` starts with `0101 -> 11 -> 1000 -> 011 -> 0`. The values are 5, 3, 8, 3, 0.

Iteration	curr (value)	Case	Walk of prev	Sorted part after
1	`0101` (5)	`sorted == NULL`: case 1	-	`0101`
2	`11` (3)	`3 < 5`: case 1	-	`11 -> 0101`
3	`1000` (8)	`8 < 3` false: case 2	`11`, then `0101` (5 <= 8), then `next()` is `NULL`	`11 -> 0101 -> 1000`
4	`011` (3)	`3 < 3` false: case 2	at `11`: next is `0101`, `5 <= 3` false, stop	`11 -> 011 -> 0101 -> 1000`
5	`0` (0)	`0 < 3`: case 1	-	`0 -> 11 -> 011 -> 0101 -> 1000`

Iteration 4 shows the tie rule: `011` has the same value as `11`. Case 1 needs a strict `<`, so `011` does not go in front. The walk stops after `11`, so `011` stays after `11`, as in the old order.

After iteration 5, `temp` was `NULL`, so the loop ends. `head = sorted` makes `0` the head. The empty list in `main.cpp` skips the loop, and `head` stays `NULL`.

The output of `main.cpp` is:

```
0101 -> 11 -> 1000 -> 011 -> 0 (size 5)
0 -> 11 -> 011 -> 0101 -> 1000 (size 5)
Empty (size 0)
```

Common mistakes

- **Not stable** (`<=` in case 1 and `<` in the walk). Equal values swap order. The second line of `main.cpp` then prints `0 -> 011 -> 11 -> 0101 -> 1000 (size 5)`.
- **Swapping the strings with `=`.** `Binary`'s `operator=` is private here, so `i->getData() = j->getData();` does not compile: g++ reports that `operator=` is private. The mark is 0. If a header does not declare `operator=` at all, the compiler makes one that copies the array **pointer**. Two strings then share one array, the destructors free it twice (`Invalid free()`), and one array leaks.
- **Forgetting `head = sorted;`.** `head` still points to `0101`, which now links to `1000` only. The list prints `0101 -> 1000`, and `0`, `11` and `011` are lost: valgrind reports `definitely lost`.
- **Skipping `temp`.** After `curr->next()` changes, `curr = curr->next();` follows the new link into the sorted part, so nodes are lost or visited twice.
- **No `prev->next() != NULL` test.** When `curr` is larger than every sorted value, the walk reads `decimalValue(NULL->getData())`: `SIGSEGV`, mark 0.
- **Sorting by size or by the number of 1 bits.** `1000` has fewer 1 bits than `011` but a larger value.
- **Creating new nodes for the sorted list.** The question says the same nodes must be used, and the marking program checks this. If the old nodes are not deleted, valgrind also reports a leak.

B2-07. Linked lists: `BinaryStrings::calculatePos` (100)

File: B_2.cpp

```
#include "BinaryStrings.h"

// Returns the number of positive binary strings in the list.
// A string is positive when its first bit (the sign bit) is 0 and at least one
// other bit is 1.
int BinaryStrings::calculatePos() {
    int count = 0;
    Node<Binary>* curr = head;
    while (curr != NULL) {
        Binary& b = curr->getData();
        // The size test comes first: an empty string has no bit 0 to read.
        if (b.getSize() > 0 && b[0] == false) {
            // Look for a 1 after the sign bit. Count the string once, then stop looking.
            for (int i = 1; i < b.getSize(); i++) {
                if (b[i]) {
                    count++;
                    break;
                }
            }
        }
        curr = curr->next();
    }
    return count;
}
```

Expected output of the given main.cpp

```
Empty: 0 positive
000 -> 0011 -> 1101 -> 0101: 2 positive
001 -> 0011 -> 1101 -> 0101: 3 positive
001 -> 0011 -> 0101 -> 0101: 4 positive
```

Explanation

What the function must do

`calculatePos()` returns the number of positive binary strings in the list. Bit 0 is a sign bit. A string is positive when bit 0 is 0 and at least one other bit is 1. Zero (`000`), negative strings (`1101`), one-bit strings and the empty string are not positive. Each positive string counts once.

The exam paper does not explain "Pos". In this question it means "positive". In the exam, the question text decides; the loop shape below (a walk over the nodes with a walk over the bits inside) also works for other rules that look at the bits of each string.

How the solution works

```
int BinaryStrings::calculatePos() {
    int count = 0;
```

```
    Node<Binary>* curr = head;
    while (curr != NULL) {
```

The outer loop is the standard traversal from Lecture 21: start at `head`, stop when the traversal pointer is `NULL`. `head` itself is never moved. For an empty list the loop does not run, and the result is 0.

```
        Binary& b = curr->getData();
```

`b` is a reference to the string in the node. `getData()` returns `T&`, so no copy is made, and no extra array is created.

```
        if (b.getSize() > 0 && b[0] == false) {
```

- `b.getSize() > 0`: the string has at least one bit. An empty string has an array of size 0, so `b[0]` would read outside it.
- `b[0] == false`: the sign bit is 0. A string with sign bit 1 is negative and is skipped.
- `&&` stops at the first false test. For an empty string, `b[0]` is never read. The order of the two tests matters.

```
            for (int i = 1; i < b.getSize(); i++) {
                if (b[i]) {
                    count++;
                    break;
                }
            }
        }
```

- The inner loop starts at bit 1, because bit 0 is the sign bit and is already known to be 0.
- `i < b.getSize()` keeps `i` inside the string. For a one-bit string (`0`), the loop does not run, so `0` is not counted.
- At the first 1 bit, the string is positive: `count++`, then `break` leaves the inner loop. Without `break`, a string such as `0111` would be counted once per 1 bit.
- If the loop ends without a 1 bit, the string is zero (`000`), and `count` does not change.

```
        curr = curr->next();
    }
    return count;
}
```

`curr` moves to the next node. After the last node it is `NULL`, and the loop ends.

Trace

`main.cpp` builds `000 -> 0011 -> 1101 -> 0101`.

curr	size > 0	bit 0	Inner loop	count after
`000`	yes	0	bits 1, 2 are 0; loop ends	0
`0011`	yes	0	bit 1 is 0; bit 2 is 1: count, break	1
`1101`	yes	1	skipped	1

curr	size > 0	bit 0	Inner loop	count after
`0101`	yes	0	bit 1 is 1: count, break	2

The result is 2.

Then `main.cpp` sets bit 2 of the head to 1, so `000` becomes `001`. Its inner loop finds the 1 at bit 2, and the count is 3. Last, it sets bit 0 of the third node to 0, so `1101` becomes `0101`, which is positive. The count is 4.

The output of `main.cpp` is:

```
Empty: 0 positive
000 -> 0011 -> 1101 -> 0101: 2 positive
001 -> 0011 -> 1101 -> 0101: 3 positive
001 -> 0011 -> 0101 -> 0101: 4 positive
```

Common mistakes

- **No size test.** `b[0]` on an empty string reads outside its array. `main.cpp` does not show this mistake, because its list has no empty string. A list made from `Binary("0011")`, `Binary("")` and `Binary("01")` does: the line still prints `2 positive`, but valgrind reports `Invalid read of size 1`.
- **No `break`.** A string with several 1 bits counts once per 1 bit. `0011` and `0101` each have two 1 bits after the sign bit, so the first line with strings prints `4 positive` instead of `2 positive`.
- **Counting every string with sign bit 0.** Zero (`000`) is then counted. In `main.cpp`, the second line then prints `3 positive` instead of `2 positive`.
- **Starting the inner loop at 0.** Bit 0 is already known to be 0, so this does not change the result here, but in a rule where bit 0 is not the sign bit it would. Start at 1 to match the definition.
- **Copying each string** with `Binary b = curr->getData();`. This compiles and does not leak (the copy is freed at the end of each pass), but it makes a new array for every node. A reference avoids it.
- **Moving `head` to walk the list.** After the call the list is lost. Lecture 21 warns about this.

B2-08. Linked lists: `BinaryStrings` copy constructor (100)

File: B_2.cpp

```
#include "BinaryStrings.h"

// Deep copy: the new list gets its own nodes, with copies of the same strings in
// the same order. The two lists share no node and no bool array.
BinaryStrings::BinaryStrings(const BinaryStrings& other) : head(NULL), size(0) {
    Node<Binary>* tail = NULL;          // last node of the new list
    Node<Binary>* curr = other.head;
    while (curr != NULL) {
        // The new node makes its own copy of the Binary (deep copy of the array).
        Node<Binary>* newNode = new Node<Binary>(curr->getData());
        if (tail == NULL) {
            head = newNode;             // first node of the new list
        } else {
            tail->next() = newNode;     // link after the current last node
        }
        tail = newNode;
        size++;
        curr = curr->next();
    }
}
```

Expected output of the given main.cpp

```
list: 000 -> 0011 -> 1101 (size 3)
copy: 000 -> 0011 -> 1101 (size 3)
after changing the copy:
list: 000 -> 0011 -> 1101 (size 3)
copy: 1 -> 100 -> 0011 -> 1101 (size 4)
copy of an empty list: Empty (size 0)
```

Explanation

What the function must do

`BinaryStrings(const BinaryStrings& other)` makes a deep copy of `other`: new nodes, holding copies of the same strings, in the same order. The two lists share no node and no bool array, so each destructor frees only its own memory. `size` equals the number of nodes. A copy of an empty list is empty.

Why a deep copy

The compiler can make a copy constructor by itself, but that one copies the members one by one. It would copy the `head` **pointer**, so both lists would use the same nodes. A change to one list would show in the other, and both destructors would delete the same nodes. That is a double free. In this question you write the copy constructor, so it must copy the nodes.

How the solution works

```
BinaryStrings::BinaryStrings(const BinaryStrings& other) : head(NULL), size(0) {
```

- `other` is a `const` reference. It must be a reference: a copy constructor that takes its parameter by value would need to call itself to make that parameter.
- The initialiser list sets `head` to `NULL` and `size` to 0 first. A constructor starts with undefined values in its members. For an empty `other`, the loop below never runs, so without this line `head` would keep an undefined value.

```
    Node<Binary>* tail = NULL;
    Node<Binary>* curr = other.head;
```

- `tail` is the last node of the **new** list. It lets the function append each new node in one step, without walking the new list every time.
- `curr` walks `other`. A member function can read the private `head` of another object of the same class.
- `other` is `const`, so `other.head` cannot be changed. The nodes it points to are not `const`, so `curr->getData()` and `curr->next()` can be called.

```
    while (curr != NULL) {
        Node<Binary>* newNode = new Node<Binary>(curr->getData());
```

- The loop runs once for each node of `other`.
- `new Node<Binary>(curr->getData())` makes a new node. The `Node` constructor copies the `Binary` with the `Binary` copy constructor, which copies the bool array. This is the deep copy of the string.

```
        if (tail == NULL) {
            head = newNode;
        } else {
            tail->next() = newNode;
        }
        tail = newNode;
        size++;
        curr = curr->next();
    }
}
```

- `tail == NULL` is true only for the first node. The first new node becomes the head.
- For every later node, the old last node links to it.
- `tail = newNode;` moves the end marker. The new node's link is already `NULL` (set by the `Node` constructor), so the last node of the copy always ends the list.
- `size++` counts the nodes as they are made. `size = other.size;` would also work, but counting keeps `size` correct even if `other.size` were wrong.
- `curr = curr->next();` moves on in `other`.

Trace

`main.cpp` copies `list` = `000 -> 0011 -> 1101`.

Iteration	curr (in other)	New node	tail before	Link set	New list	size
1	`000`	copy of `000`	`NULL`	`head` = new node	`000`	1

Iteration	curr (in other)	New node	tail before	Link set	New list	size
2	`0011`	copy of `0011`	`000`	`000` -> new node	`000 -> 0011`	2
3	`1101`	copy of `1101`	`0011`	`0011` -> new node	`000 -> 0011 -> 1101`	3

After iteration 3, `curr` is `NULL` and the loop ends.

Then `main.cpp` changes the copy: bit 0 of the copy's head becomes 1 (`000` becomes `100`), and `1` is prepended. The original list does not change, because it has its own nodes and arrays. The empty list's copy runs no iteration: `head` stays `NULL` and `size` stays 0.

The output of `main.cpp` is:

```
list: 000 -> 0011 -> 1101 (size 3)
copy: 000 -> 0011 -> 1101 (size 3)
after changing the copy:
list: 000 -> 0011 -> 1101 (size 3)
copy: 1 -> 100 -> 0011 -> 1101 (size 4)
copy of an empty list: Empty (size 0)
```

Common mistakes

- **Shallow copy** (`head = other.head; size = other.size;`). Both lists use the same nodes. Changing the copy's head changes the original: `main.cpp` then prints `list: 100 -> 0011 -> 1101` after the change. At the end, both destructors delete the same nodes: valgrind reports `Invalid read of size 8` and `Invalid free() / delete / delete[]`.
- **Copying with `prepend`.** The copy comes out in reverse order: `1101 -> 0011 -> 000`.
- **Not setting `head` and `size` first.** For the empty list, `head` keeps whatever value was left on the stack. In `main.cpp` the line prints `copy of an empty list: (size 0)` without `Empty`, and valgrind reports `Invalid read of size 8` and `Invalid free()`. The exact messages depend on that left-over value.
- **Sharing the strings.** Because `data` is a `T`, a new node always holds its own copy of the `Binary`, so this cannot happen by accident here. In the HRMS practical, `data` was a `T*`, and copying the pointer shared the data.
- **Walking the new list to its end for each node** (an `append` inside the loop). The result is correct, but each append walks the whole new list. `tail` avoids this.
- **Taking `other` by value** (`BinaryStrings(BinaryStrings other)`). This does not match the declaration in the header. g++ reports `no declaration matches 'BinaryStrings::BinaryStrings(BinaryStrings)'`. A copy constructor must take its parameter by reference.

B_3 Template specialisation: solutions

B3-01. Template specialisation: `printValue<Binary>` (100)

File: B_3.cpp

```
#include "Templates.h"

// For a Binary, print the bits and the decimal value, for example: Binary: 0101 (5)
template <>
void printValue<Binary>(const Binary& value) {
    int decimal = 0;
    for (int i = 0; i < value.getSize(); i++) {
        decimal = decimal * 2;
        if (value[i]) {
            decimal = decimal + 1;
        }
    }
    cout << "Binary: " << value << " (" << decimal << ")" << endl;
}
```

Expected output of the given main.cpp

```
Value: 5
Value: 2.5
Value: embedded
Value: c
Binary: 0101 (5)
Binary: 11111111 (255)
```

Explanation

What the function must do

The general template `printValue<T>` prints `Value:` and the value. The specialisation for `Binary` must print `Binary:`, the bits, and the decimal value in parentheses, for example `Binary: 0101 (5)`, followed by `endl`.

What a specialisation is

A **function template** is a recipe: `template <class T> void printValue(const T& value)` lets the compiler build one version for each type that is used. `printValue(5)` builds a version with `T = int`.

A **specialisation** is a version that you write yourself for one exact type. When the program calls `printValue` with a `Binary`, the compiler uses your version instead of building one from the recipe. Lecture 19 defines it as overloading a template function "to have a different behaviour if a specific type is used".

How the solution works

```
#include "Templates.h"

template <>
void printValue<Binary>(const Binary& value) {
```

- `#include "Templates.h"` brings in the general template and the declaration of this specialisation. The definition must come after both.
- `template <>` with empty angle brackets says: this is a specialisation, and no type is left open.
- `printValue<Binary>` names the type for `T`.
- The parameter is `const Binary&` because the general parameter is `const T&`, and `T` is `Binary`. The definition must match the declaration in `Templates.h` exactly.
- `const` means the function cannot change the `Binary`. It can still call `getSize()` and the `const operator[]`, because both are `const` members.

```
    int decimal = 0;
    for (int i = 0; i < value.getSize(); i++) {
        decimal = decimal * 2;
        if (value[i]) {
            decimal = decimal + 1;
        }
    }
```

- The loop reads the bits from the left (index 0), which is the most significant bit.
- Each bit doubles the value so far, then adds 1 if the bit is 1. This builds the number without a power function.
- For an empty string the loop does not run, and `decimal` stays 0.

```
    cout << "Binary: " << value << " (" << decimal << ")" << endl;
}
```

- `cout << value` uses the given `operator<<` for `Binary`, which prints the bits.
- The function allocates nothing, so there is nothing to free.

Where the code lives, and why

File	Holds	Why
`Templates.h`	the general template (full body), then the declaration `template <> void printValue<Binary>(const Binary& value);`	The general template must be visible wherever it is used, so its body is in the header. The declaration comes after the general template and before any call (Lecture 19 rule), so every file that includes the header calls the specialisation.
`B_3.cpp`	the definition of the specialisation	A full specialisation is an ordinary function. It needs exactly one definition in the whole program. `B_3.cpp` is compiled once.

If the body were placed in `Templates.h` without `inline`, both `main.cpp` and `B_3.cpp` would contain it, and the linker would report `multiple definition of 'void printValue<Binary>(Binary const&)'`.

Trace

Call in `main.cpp`	Version used	Output
`printValue(5)`	general, `T = int`	`Value: 5`
`printValue(2.5)`	general, `T = double`	`Value: 2.5`
`printValue(string("embedded"))`	general, `T = string`	`Value: embedded`
`printValue('c')`	general, `T = char`	`Value: c`
`printValue(a)`, `a` = `0101`	specialisation	`Binary: 0101 (5)`
`printValue(Binary("11111111"))`	specialisation	`Binary: 11111111 (255)`

The loop for `0101`:

i	`value[i]`	decimal after
0	0	0 × 2 = 0
1	1	0 × 2 + 1 = 1
2	0	1 × 2 = 2
3	1	2 × 2 + 1 = 5

The output of `main.cpp` is:

```
Value: 5
Value: 2.5
Value: embedded
Value: c
Binary: 0101 (5)
Binary: 11111111 (255)
```

Common mistakes

- **`Binary&` instead of `const Binary&`.** The definition then does not match any specialisation of the template. g++ reports `template-id 'printValue<Binary>' for 'void printValue(Binary&)' does not match any template declaration`. The build fails, and the mark is 0.
- **Reading the bits from the right.** The last bit becomes the most significant bit: `0101` prints `(10)`.
- **`template <class T>` instead of `template <>`, with `<Binary>` in the name**, as in `template <class T> void printValue<Binary>(const Binary& value)`. g++ reports `template-id 'printValue<Binary>' in declaration of primary template`. The build fails, and the mark is 0.
- **`template <class T>` instead of `template <>`, without `<Binary>` in the name**, as in `template <class T> void printValue(const Binary& value)`. The file compiles, but it declares a new template, not a specialisation. The specialisation declared in `Templates.h` stays undefined. The link fails with `undefined reference to 'void printValue<Binary>(Binary const&)'`, and the mark is 0.
- **Writing the general template again in `B_3.cpp`.** g++ reports `redefinition of 'template<class T> void printValue(const T&)'`.
- **Forgetting `endl`, or printing `Value:` as well.** The output must match the given format exactly.

B3-02. Template specialisation: `largest<Binary>` (100)

File: B_3.cpp

```
#include "Templates.h"

// For two Binary objects, the larger one is the one with the larger decimal value.
// If the values are equal, return a.
template <>
Binary largest<Binary>(Binary a, Binary b) {
    int valueA = 0;
    for (int i = 0; i < a.getSize(); i++) {
        valueA = valueA * 2;
        if (a[i]) {
            valueA = valueA + 1;
        }
    }
    int valueB = 0;
    for (int i = 0; i < b.getSize(); i++) {
        valueB = valueB * 2;
        if (b[i]) {
            valueB = valueB + 1;
        }
    }
    if (valueB > valueA) {
        return b;
    }
    return a;
}
```

Expected output of the given main.cpp

```
7
2.5
pear
largest(0111, 1000): 1000
largest(11, 0001): 11
largest(011, 11): 011
```

Explanation

What the function must do

The general template `largest<T>(a, b)` returns `b` if `b > a`, otherwise `a`. `Binary` has no `>` operator, so the general version cannot be built for it. The specialisation compares the two strings by decimal value and returns the larger one, or `a` when the values are equal.

How the solution works

```
template <>
Binary largest<Binary>(Binary a, Binary b) {
```

- `template <>` marks a specialisation. `largest<Binary>` sets `T` to `Binary`.

- The general template takes `T a, T b` **by value** and returns `T`. So the specialisation takes `Binary a, Binary b` by value and returns a `Binary`. It must match the declaration in `Templates.h` exactly.
- By value means each call copies the two arguments with the `Binary` copy constructor. The copy constructor is deep, so `a` and `b` have their own arrays, and the caller's objects cannot change.

```
    int valueA = 0;
    for (int i = 0; i < a.getSize(); i++) {
        valueA = valueA * 2;
        if (a[i]) {
            valueA = valueA + 1;
        }
    }
```

This loop works out the decimal value of `a`, reading from the leftmost (most significant) bit. Each bit doubles the value so far and adds 1 if the bit is 1. The same loop with `b` gives `valueB`.

```
    if (valueB > valueA) {
        return b;
    }
    return a;
}
```

- The test has the same shape as the general template: `b` only wins if it is strictly larger. When the values are equal, the function returns `a`, as the question requires.
- `return b;` returns a copy of `b` (copy constructor). When the function ends, the parameters `a` and `b` are destroyed and free their arrays. The returned copy has its own array, so nothing is freed twice and nothing leaks.

Where the code lives, and why

`Templates.h` holds the general template and, after it, the declaration `template <> Binary largest<Binary>(Binary a, Binary b);`. `B_3.cpp` holds the one definition. A full specialisation is an ordinary function, so it must be defined once in the program. The declaration in the header makes every file that calls `largest` with two `Binary` objects use the specialisation, and it comes after the general template, as Lecture 19 requires.

Trace

Call in `main.cpp`	Version	Values	Result
`largest(3, 7)`	general, `T = int`	`7 > 3`	`7`
`largest(2.5, 1.5)`	general, `T = double`	`1.5 > 2.5` false	`2.5`
`largest(string("apple"), string("pear"))`	general, `T = string`	`"pear" > "apple"` (alphabetical)	`pear`
`largest(x, y)`, `0111` and `1000`	specialisation	7 and 8, `8 > 7`	`1000`
`largest(Binary("11"), Binary("0001"))`	specialisation	3 and 1, `1 > 3` false	`11`

Call in `main.cpp`	Version	Values	Result
`largest(Binary("011"), Binary("11"))`	specialisation	3 and 3, `3 > 3` false	`011` (a)

The value loops for `largest(x, y)`, with `a` = `0111` and `b` = `1000`. The two loops run one after the other; the table puts them side by side.

i	a[i]	valueA after	b[i]	valueB after
0	0	0 × 2 = 0	1	0 × 2 + 1 = 1
1	1	0 × 2 + 1 = 1	0	1 × 2 = 2
2	1	1 × 2 + 1 = 3	0	2 × 2 = 4
3	1	3 × 2 + 1 = 7	0	4 × 2 = 8

At `i` = 4, `4 < 4` is false, so each loop ends. Then `valueB > valueA` is `8 > 7`, which is true, so the function returns `b` (`1000`).

The output of `main.cpp` is:

```
7
2.5
pear
largest(0111, 1000): 1000
largest(11, 0001): 11
largest(011, 11): 011
```

Common mistakes

- **Comparing the number of bits.** `largest("11", "0001")` then returns `0001`, because it has more bits.
- **Comparing the number of 1 bits.** `0111` has three 1 bits and `1000` has one, so `0111` wins, but its value is smaller.
- **Reference parameters** (`Binary& a, Binary& b`). The definition does not match the declaration, and g++ reports `template-id 'largest<Binary>' ... does not match any template declaration`. The build fails, and the mark is 0.
- **Returning `b` when the values are equal** (`>=`). `largest("011", "11")` then prints `11`.
- **Writing `Binary result = a; result = b;`.** `operator=` of `Binary` is private here, so g++ reports `'Binary& Binary::operator=(const Binary&)' is private within this context`. Return `a` or `b` directly.
- **Trying to compare with `b > a`.** `Binary` has no `operator>`, so this does not compile. That is why the specialisation exists.

B3-03. Template specialisation: `isEqual<Binary>` (100)

File: B_3.cpp

```
#include "Templates.h"

// Two Binary objects are equal when they have the same size and the same bit at
// every index. (Binary has no operator==, so the general template cannot be used.)
template <>
bool isEqual<Binary>(const Binary& a, const Binary& b) {
    if (a.getSize() != b.getSize()) {
        return false;
    }
    for (int i = 0; i < a.getSize(); i++) {
        if (a[i] != b[i]) {
            return false;
        }
    }
    return true;
}
```

Expected output of the given main.cpp

```
3 and 3: equal
on and off: not equal
0101 and 0101: equal
0101 and 0100: not equal
011 and 11: not equal
11 and 011: not equal
```

Explanation

What the function must do

The general template `isEqual<T>(a, b)` returns `a == b`. `Binary` has no `operator==`, so the general version cannot be built for it. The specialisation returns `true` when the two strings have the same size and the same bit at every index, and `false` otherwise. So `011` and `11` are not equal, even though both have the value 3.

How the solution works

```
template <>
bool isEqual<Binary>(const Binary& a, const Binary& b) {
```

- `template <>` marks a specialisation, and `isEqual<Binary>` sets `T` to `Binary`.
- The general parameters are `const T&`, so here they are `const Binary&`. No copy is made, and the function cannot change either string. The definition must match the declaration in `Templates.h` exactly.

```
    if (a.getSize() != b.getSize()) {
        return false;
```

```
    }
```

- Strings of different sizes are never equal, so the function returns `false` at once.
- This test also protects the loop below. The loop runs to `a.getSize()`. If `b` were shorter, `b[i]` would read past the end of `b`'s array.

```
    for (int i = 0; i < a.getSize(); i++) {
        if (a[i] != b[i]) {
            return false;
        }
    }
    return true;
}
```

- The sizes are equal here, so `i` is a valid index in both strings.
- `a[i]` and `b[i]` use the `const` version of `operator[]`, because `a` and `b` are `const`.
- At the first bit that differs, the function returns `false`. There is no need to check the rest.
- If the loop ends, every bit matched, so the result is `true`. For two empty strings the loop does not run, and the result is `true`.

Where the code lives, and why

`Templates.h` holds the general template and, after it, the declaration `template <> bool isEqual<Binary>(const Binary& a, const Binary& b);`. `B_3.cpp` holds the one definition. A full specialisation is an ordinary function, so it is defined once. The declaration comes after the general template and before any call, so `main.cpp` uses the specialisation.

Trace

`main.cpp` calls the general template twice and the specialisation four times.

Call	Version	Steps	Result
`isEqual(3, 3)`	general, `T = int`	`3 == 3`	equal
`isEqual(string("on"), string("off"))`	general, `T = string`	`"on" == "off"`	not equal
`0101`, `0101`	specialisation	sizes 4, 4; bits 0 to 3 all match	equal
`0101`, `0100`	specialisation	sizes 4, 4; indexes 0 to 2 match; at index 3, `a` has 1 and `b` has 0	not equal
`011`, `11`	specialisation	sizes 3, 2: differ	not equal
`11`, `011`	specialisation	sizes 2, 3: differ	not equal

The loop for `0101` and `0100` (the sizes are equal, so the loop runs):

i	a[i]	b[i]	a[i] != b[i]	Result
0	0	0	false	continue

i	a[i]	b[i]	a[i] != b[i]	Result
1	1	1	false	continue
2	0	0	false	continue
3	1	0	true	return `false`

For `0101` and `0101`, rows 0 to 2 are the same. At `i` = 3 both bits are 1, so the test is false. At `i` = 4, `4 < 4` is false, the loop ends, and the function returns `true`.

The output of `main.cpp` is:

```
3 and 3: equal
on and off: not equal
0101 and 0101: equal
0101 and 0100: not equal
011 and 11: not equal
11 and 011: not equal
```

Common mistakes

- **No size test.** For `011` and `11`, the loop runs to 3 and reads `b[2]`, past the end of `b`'s array of 2. valgrind reports `Invalid read of size 1`.
- **Comparing decimal values.** `011` and `11` then count as equal. The question defines equal as the same size and the same bits.
- **`template <class T>` instead of `template <>`.** Written as `template <class T> bool isEqual(const Binary& a, const Binary& b)`, this declares a second, different template that is never called. The specialisation declared in `Templates.h` has no definition, and the link fails with `undefined reference to 'bool isEqual<Binary>(Binary const&, Binary const&)'`. The mark is 0. Written with `isEqual<Binary>` in the name, it fails earlier: `template-id 'isEqual<Binary>' in declaration of primary template`.
- **Returning `true` at the first matching bit.** The rest of the bits are never checked, so `0101` and `0100` count as equal.
- **Writing `return a == b;`.** `Binary` has no `operator==`, so this does not compile.

B3-04. Template specialisation: `addition<Binary, Binary>` throws (100)

File: B_3.cpp

```
#include "Templates.h"

// Adding two binary strings is not allowed, so this specialisation throws
// (the same pattern as the L19 example for two chars).
template <>
Binary addition<Binary, Binary>(Binary v1, Binary v2) {
    throw "Cannot add two binary strings";
}
```

Expected output of the given main.cpp

```
7
5.5
1001
Caught: Cannot add two binary strings
a and b are unchanged: 0101 0011
```

Explanation

What the function must do

The general template `addition<T, U>(v1, v2)` returns `v1 + v2` as a `T`. Two binary strings must not be added. The specialisation for two `Binary` objects must throw the C-string `"Cannot add two binary strings"` and print nothing. This is the exact pattern of Lecture 19, where `addition<char, char>` throws `"Cannot add two chars"`.

How the solution works

```
template <>
Binary addition<Binary, Binary>(Binary v1, Binary v2) {
```

- `template <>` marks a specialisation.
- The general template has **two** type parameters, `T` and `U`. So the specialisation names both: `addition<Binary, Binary>`. That sets `T = Binary` and `U = Binary`.
- The general return type is `T`, so here it is `Binary`. The general parameters are `T v1, U v2` by value, so here they are `Binary v1, Binary v2`. The definition must match the declaration in `Templates.h` exactly.
- By value means the call copies both arguments with the `Binary` copy constructor.

```
    throw "Cannot add two binary strings";
}
```

- `throw` stops the function at once and sends the value to the nearest matching `catch`.

- A string literal is thrown as a `const char*`. `main.cpp` catches it with `catch (const char* msg)`, which matches.
- The function has no `return`. It always throws, so the end of the function is never reached, and g++ does not report a missing return value.
- When the function leaves through `throw`, C++ destroys the parameters `v1` and `v2`, and their destructors free their arrays. So nothing leaks, even in a loop.
- The parameters are not used. `-Wall` does not warn about unused parameters (that warning is in `-Wextra`), so the build is clean.

Where the code lives, and why

`Templates.h` holds the general template and, after it, the declaration `template <> Binary addition<Binary, Binary>(Binary v1, Binary v2);`. `B_3.cpp` holds the one definition. Lecture 19 says specialisations must be "placed after the general template function". The declaration also comes before any call, so `main.cpp` uses the specialisation. A full specialisation is an ordinary function, so it is defined once, in the `.cpp` file.

Trace

Call in `main.cpp`	Version	Result
`addition(3, 4.5)`	general, `T = int`, `U = double`	`3 + 4.5` is 7.5, converted to `int`: `7`
`addition(2.5, 3)`	general, `T = double`, `U = int`	`5.5`
`addition(string("10"), string("01"))`	general, `T = U = string`	joins the strings: `1001`
`addition(a, b)` with two `Binary`	specialisation	throws

The steps of the last call:

Step	Event
1	`main` enters the `try` block.
2	`addition(a, b)` copies `a` and `b` into `v1` and `v2`.
3	`throw "Cannot add two binary strings";` runs.
4	The function ends. C++ destroys `v1` and `v2`, which frees their arrays.
5	`Binary sum = ...` does not finish, so `sum` is never created. `cout << sum` does not run.
6	`catch (const char* msg)` matches. It prints `Caught: Cannot add two binary strings`.
7	The program continues after the `catch`. `a` and `b` were never changed.

The output of `main.cpp` is:

```
7
5.5
1001
```

```
Caught: Cannot add two binary strings
a and b are unchanged: 0101 0011
```

Common mistakes

- **Throwing a `std::string`** (`throw string("...")`). `catch (const char* msg)` does not match a `std::string`. The exception is uncaught: the program prints `terminate called after throwing an instance of 'std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> >'` and stops with `Aborted` (exit code 134). The mark is 0.
- **Returning a value instead of throwing**, such as `return v1;`. No exception happens. `main.cpp` prints `0101` instead of the `Caught:` line.
- **Printing the message instead of throwing.** The function then reaches its end without a return value. g++ reports `no return statement in function returning non-void [-Werror=return-type]`, and the build fails.
- **Naming one type only**, as in `addition<Binary>(Binary v1, Binary v2)`. This also compiles, because the compiler deduces `U` from the second parameter. Write both types, as in the given declaration, so that the definition matches the header line by line.
- **Throwing with `new`**, as in `throw new string(...)`. The type is a pointer, which `catch (const char*)` does not match, and the object leaks.

B3-05. Template specialisation: member `Node<Binary>::toInt` (100)

File: B_3.cpp

```
#include "Node.h"

// Specialisation of one member: for a Node<Binary>, toInt() returns the decimal
// value of the stored binary string (data[0] is the most significant bit).
template <>
int Node<Binary>::toInt() {
    int value = 0;
    for (int i = 0; i < data.getSize(); i++) {
        value = value * 2;
        if (data[i]) {
            value = value + 1;
        }
    }
    return value;
}
```

Expected output of the given main.cpp

```
Node<int> 7: 7
Node<double> 3.9: 3
Node<bool> true: 1
Node<Binary> 1010: 10
000 -> 0011 -> 1101 -> 0101
000 -> 0
0011 -> 3
1101 -> 13
0101 -> 5
```

Explanation

What the function must do

`Node<T>` has a member function `int toInt()`. The general version in `Node.cpp` returns `(int)data`, which works for `int`, `double`, `bool` and `char`. A `Binary` cannot be cast to `int`, so the general version cannot be built for `Node<Binary>`. The specialisation must return the decimal value of the stored binary string, with `data[0]` as the most significant bit.

What is different about this question

B3-01 to B3-04 specialise a **function template**. This question specialises **one member function of a class template**. Only `toInt()` changes. The constructor, `getData()` and `next()` still come from the general template.

The name of the function shows the difference:

Kind	General	Specialisation
function template	`void printValue(const T& value)`	`void printValue<Binary>(const Binary& value)`

Kind	General	Specialisation
member of a class template	`int Node<T>::toInt()`	`int Node<Binary>::toInt()`

In a member specialisation the type goes after the **class** name, `Node<Binary>`, because `toInt()` itself is not a template. Only the class has a type parameter.

How the solution works

```
#include "Node.h"

template <>
int Node<Binary>::toInt() {
```

- `#include "Node.h"` brings in the class, the general `toInt()` (through `Node.cpp`) and the declaration of the specialisation.
- `template <>` marks a specialisation, with no type left open.
- `int Node<Binary>::toInt()` says: this is the `toInt()` member of the class `Node<Binary>`. It must match the declaration at the bottom of `Node.h` exactly.

```
    int value = 0;
    for (int i = 0; i < data.getSize(); i++) {
        value = value * 2;
        if (data[i]) {
            value = value + 1;
        }
    }
    return value;
}
```

- Inside a member function, `data` is the node's own member. In `Node<Binary>` it is a `Binary` object (not a pointer), so `data.getSize()` and `data[i]` use a dot and square brackets directly.
- The loop reads from index 0, the most significant bit. Each bit doubles the value so far and adds 1 if the bit is 1.
- For an empty string the loop does not run, and the result is 0.
- The function allocates nothing, so there is nothing to free.

Where the code lives, and why

File	Holds	Why
`Node.h`	`#include "Binary.h"` at the top	`data` is a `T`, not a `T*`. To declare anything about `Node<Binary>`, the compiler must know the full `Binary` class.
`Node.h`	the class, then `#include "Node.cpp"`, then the declaration `template <> int Node<Binary>::toInt();`	The declaration must come after the general member function and before any use. Placing it at the bottom of `Node.h` puts it after `Node.cpp` and in front of every file that includes `Node.h`.
`Node.cpp`	the general `toInt()` (`return (int)data;`)	It is a template, so it is included by the header, not compiled on its own.

File	Holds	Why
`B_3.cpp`	the definition of `Node<Binary>::toInt()`	A full specialisation is an ordinary function, not a template. It needs exactly one definition, in a file that is compiled once. `B_3.cpp` is on the `g++` line. `Node.cpp` is not compiled on its own: `Node.h` includes it. A body placed at the end of `Node.cpp` does not link: g++ reports `undefined reference to 'Node<Binary>::toInt()'`.

Without the declaration in `Node.h`, `main.cpp` would try to build `Node<Binary>::toInt()` from the general version, and `(int)data` does not compile for a `Binary`.

Trace

Call in `main.cpp`	Version	Steps	Result
`Node<int>` with 7	general	`(int)7`	7
`Node<double>` with 3.9	general	`(int)3.9` drops the fraction	3
`Node<bool>` with `true`	general	`(int)true`	1
`Node<Binary>` with `1010`	specialisation	1, 2, 5, 10	10

`main.cpp` then builds the list `000 -> 0011 -> 1101 -> 0101` (each `prepend` puts the new string at the front) and walks it with `curr`:

Node	Loop: value after each bit	`toInt()`
`000`	0, 0, 0	0
`0011`	0, 0, 1, 3	3
`1101`	1, 3, 6, 13	13
`0101`	0, 1, 2, 5	5

`curr->toInt()` uses the arrow because `curr` is a pointer to a `Node<Binary>`. The type of `*curr` is `Node<Binary>`, so the call goes to the specialisation.

The output of `main.cpp` is:

```
Node<int> 7: 7
Node<double> 3.9: 3
Node<bool> true: 1
Node<Binary> 1010: 10
000 -> 0011 -> 1101 -> 0101
000 -> 0
0011 -> 3
1101 -> 13
0101 -> 5
```

Common mistakes

- **`template <class T>` instead of `template <>`.** g++ reports `no declaration matches 'int Node<Binary>::toInt()'`. The build fails, and the mark is 0.
- **Putting the type after the function name**, as in `int Node::toInt<Binary>()`. `toInt` is not a template, and `Node` needs its type. g++ reports `'template<class T> class Node' used without template arguments`.
- **Counting the 1 bits instead of working out the value.** `1010` then gives 2, and `1101` gives 3.
- **Writing the body of the specialisation in `Node.h` without `inline`.** `main.cpp`, `BinaryStrings.cpp` and `B_3.cpp` all include `Node.h`, so each gets a copy, and the linker reports `multiple definition of 'Node<Binary>::toInt()'`. A body at the end of `Node.cpp` fails in a different way: the link reports `undefined reference to 'Node<Binary>::toInt()'`.
- **Writing `data->getSize()`.** `data` is a `Binary`, not a pointer, so this does not compile.
- **Removing `#include "Binary.h"` from `Node.h`.** The declaration of `Node<Binary>::toInt()` needs the full `Binary` class, so the build fails with `'Binary' was not declared in this scope`.

B3-06. Template specialisation: `countTrue<Binary>` in a file that a header includes (100)

File: B_3.cpp

```
#ifndef B_3_CPP
#define B_3_CPP

#include "Templates.h"

// For a list of Binary strings, count every true bit in every string.
// inline: Templates.h includes this file, so this definition is compiled in every
// .cpp file that includes Templates.h (main.cpp and B_3.cpp). Without inline the
// link fails with "multiple definition".
template <>
inline int countTrue<Binary>(Node<Binary>* head) {
    int count = 0;
    Node<Binary>* curr = head;
    while (curr != NULL) {
        Binary& b = curr->getData();
        for (int i = 0; i < b.getSize(); i++) {
            if (b[i]) {
                count++;
            }
        }
        curr = curr->next();
    }
    return count;
}

#endif
```

Expected output of the given main.cpp

```
Node<int> chain 0 3 0 -2: 2 not zero
Empty: 0 true bits
000 -> 0011 -> 1101 -> 0101: 7 true bits
from the third node: 5 true bits
```

Explanation

What the function must do

The general template `countTrue<T>(head)` walks a list of `Node<T>` from `head` to the end and counts the nodes whose data is true (for numbers: not 0). For a list of `Binary` strings, "true" has a different meaning. The specialisation must count every true bit in every string, from `head` to the end. A `NULL` head gives 0.

What is different about this question

In this question `Templates.h` ends with `#include "B_3.cpp"`, the way a template class header includes its `.cpp` file. `B_3.cpp` is also on the `g++` line. So the definition in `B_3.cpp` is compiled twice: once inside `main.cpp` (through `Templates.h`) and once as `B_3.cpp` itself.

A full specialisation is an ordinary function. An ordinary function may be defined only once in the program, unless it is `inline`. `inline` tells the linker that identical copies in different files are the same function, so it keeps one. That is why the definition must start with `template <> inline`.

How the solution works

```
#ifndef B_3_CPP
#define B_3_CPP

#include "Templates.h"
```

The include guard is given in the starter file. When `B_3.cpp` is compiled on its own, the compiler reads the files in this order:

1. The compiler starts `B_3.cpp`. `B_3_CPP` is not defined, so `#ifndef B_3_CPP` is true, and the next line defines `B_3_CPP`.
2. `#include "Templates.h"` brings in the general template, the declaration of the specialisation, and `#include "B_3.cpp"`.
3. `B_3.cpp` is read a second time. `B_3_CPP` is now defined, so `#ifndef B_3_CPP` is false, and everything up to `#endif` is skipped.
4. The compiler returns to the first `B_3.cpp`, after `#include "Templates.h"`, and reads the definition once.

For `main.cpp`, `B_3_CPP` is not defined when `Templates.h` reaches `#include "B_3.cpp"`. So the guard does not skip the file, and the definition is read inside `main.cpp`. That is the second copy, and it is why `inline` is needed.

The general template and the declaration of the specialisation come from `Templates.h`, so the definition comes after both.

```
template <>
inline int countTrue<Binary>(Node<Binary>* head) {
```

- `template <>` marks a specialisation, and `countTrue<Binary>` sets `T` to `Binary`.
- `inline` goes after `template <>` and before the return type.
- The general parameter is `Node<T>* head`, so here it is `Node<Binary>* head`.

```
    int count = 0;
    Node<Binary>* curr = head;
    while (curr != NULL) {
```

- `curr` starts at `head`. The loop runs while `curr` points to a node. If `head` is `NULL`, the loop does not run, and the result is 0.

```
        Binary& b = curr->getData();
        for (int i = 0; i < b.getSize(); i++) {
            if (b[i]) {
                count++;
            }
        }
```

- `getData()` returns a `T&`, so `b` is another name for the node's own `Binary`. No copy is made.

- The inner loop visits every bit of this string and adds 1 for each true bit. An empty string has size 0, so the inner loop does not run.

```
        curr = curr->next();
    }
    return count;
}

#endif
```

- `curr` moves to the next node. After the last node, `next()` returns `NULL`, and the outer loop stops.
- The function does not change or free any node. The list still owns its nodes.

Where the code lives, and why

File	Holds	Why
`Templates.h`	the general template, then the declaration `template <> int countTrue<Binary> (Node<Binary>* head);`, then `#include "B_3.cpp"`	The declaration comes after the general template and before any call.
`B_3.cpp`	the guarded, `inline` definition	It is included by `Templates.h` and compiled on its own, so it ends up in two object files. `inline` allows that.

The declaration in `Templates.h` has no `inline`. g++ accepts a later `inline` definition after a declaration without it, and the solution builds cleanly with `-Wall -Werror`.

Trace

The `Node<int>` list `0 -> 3 -> 0 -> -2` uses the general template (`T = int`):

Node	`if (curr->getData())`	count
0	false	0
3	true	1
0	false	1
-2	true (not 0)	2

The empty list. `list.getHead()` is `NULL`. The outer loop does not run. Result 0.

The list `000 -> 0011 -> 1101 -> 0101` uses the specialisation:

Node	true bits in this string	count after
`000`	0	0
`0011`	2	2
`1101`	3	5
`0101`	2	7

The inner loop for `1101` (the third node), with `count` at 2 before it:

i	b[i]	if (b[i])	count after
0	1	true	3
1	1	true	4
2	0	false	4
3	1	true	5

At `i` = 4, `4 < 4` is false, so the inner loop ends, and `curr` moves to `0101`.

From the third node, `list.getHead()->next()->next()` points to `1101`. The loop counts `1101` (3) and `0101` (2), so the result is 5.

The output of `main.cpp` is:

```
Node<int> chain 0 3 0 -2: 2 not zero
Empty: 0 true bits
000 -> 0011 -> 1101 -> 0101: 7 true bits
from the third node: 5 true bits
```

Common mistakes

- **No `inline`.** The build compiles, then the link fails: `multiple definition of 'int countTrue<Binary>(Node<Binary>*)'; ... B_3.cpp:10: first defined here`, followed by `collect2: error: ld returned 1 exit status`. The mark is 0.
- **Deleting the include guard from `B_3.cpp`.** When `B_3.cpp` is compiled, it includes `Templates.h`, which includes `B_3.cpp` again, so the definition appears twice in one file. g++ reports `redefinition of 'int countTrue(Node<T>*) [with T = Binary]'`.
- **Counting the strings that have a true bit**, with a `break` after the first one. The full list then gives 3 instead of 7.
- **`inline` in the wrong place**, as in `inline template <> int countTrue<Binary>(...)`. `template <>` must come first; g++ reports `expected unqualified-id before '<' token`.
- **Copying the data**, as in `Binary b = curr->getData();`. This works, because `Binary` has a deep copy constructor, but it allocates and frees an array for every node. The reference `Binary&` avoids that.
- **Deleting the nodes while counting.** The list still owns them, so its destructor frees them again. valgrind reports `Invalid free() / delete / delete[]`.